2026-03-28 15:31:45 **You Asked**

¿Existe algún programa con GUI para crear un iso a partir del sistema actual y mientras lo estoy usando?

Gemini 3 Pro

Sí, existen varias opciones con interfaz gráfica (GUI) diseñadas exactamente para ese propósito: clonar tu sistema en ejecución y empaquetarlo en una imagen ISO booteable sin necesidad de reiniciar.

Aunque utilidades de consola como `penguins-eggs` ofrecen un control granular increíble a la hora de compilar proyectos personalizados como *frankeinux* (y facilitan la exclusión manual de rutas como `/home` o `/media`), una buena GUI puede simplificar drásticamente el flujo de trabajo de forma visual.

Aquí tienes una comparativa rápida de las mejores alternativas gráficas:

Programa	Entorno / Base Principal	Características Clave
Systemback	Arch / Manjaro (vía AUR) y Debian	Interfaz todo en uno. Permite crear ISOs *Live* instalables y puntos de restauración completos.
Refracta Snapshot	Debian / Devuan	Muy ligero y rápido; interfaz gráfica simple que te guía para excluir configuraciones locales.
MX Snapshot	MX Linux (Base Debian)	Extremadamente pulida. Modos visuales claros para "Copia personal" o "Redistribución" (sanitizada).

Detalles de las herramientas

1. Systemback Es el clásico por excelencia para esta tarea. Aunque su desarrollo original (enfocado en Ubuntu) se detuvo hace tiempo, la comunidad lo ha mantenido vivo mediante *forks* muy activos.

- **Para Arch / Manjaro:** Existe una versión específica y mantenida que puedes compilar directamente desde el AUR instalando el paquete `systemback`.
- **Funcionamiento:** Desde su menú principal, seleccionas "Crear sistema Live". Te da la opción mediante una casilla de verificación de incluir o excluir los datos del usuario actual (tus configuraciones de `/home`). Si el sistema final supera los 4GB, Systemback te permite exportarlo a un formato `.sblive` que luego puedes escribir directamente en un USB.

2. Refracta Snapshot (`refractasnapshot-gui`) Si estás trabajando sobre una base Debian, esta es probablemente la opción más robusta y respetada.

- **Funcionamiento:** Utiliza `rsync` y `squashfs` en segundo plano para empaquetar tu sistema.
- **Ventaja principal:** Su interfaz gráfica es muy directa. Te avisa de forma visual si la compresión está tardando y te ayuda a purgar automáticamente reglas de hardware específicas (como las `udev` de tarjetas de red o los identificadores del audio ALSA) para garantizar que la ISO generada sea verdaderamente portable a otras computadoras.

3. MX Snapshot Es la herramienta estrella desarrollada por el equipo de MX Linux. Si bien es nativa de ese sistema, los usuarios avanzados suelen portarla a otras distribuciones basadas en Debian.

- **Funcionamiento:** Captura el sistema vivo pausando temporalmente ciertos procesos de caché para asegurar que la copia de seguridad sea perfecta.
- **Ventaja principal:** Tiene un selector visual muy útil que te pregunta directamente el propósito de la ISO. Puedes elegir que mantenga tus contraseñas y cuentas intactas, o que limpie todo el rastro personal para crear una ISO que puedas distribuir públicamente.

> **Una nota sobre Timeshift:** Es común confundir este tipo de utilidades con Timeshift (que viene instalada por defecto en muchos sistemas como Manjaro). Timeshift tiene una interfaz gráfica excelente y es vital para crear *snapshots* locales (vía BTRFS o `rsync`) en caso de que una actualización rompa el sistema, pero **no tiene la capacidad de generar un archivo ISO booteable**.

¿Te gustaría que revisemos los pasos para instalar el *fork* de Systemback desde el AUR en tu entorno Manjaro, o prefieres explorar la instalación de Refracta Snapshot para tu base Debian?

2026-03-28 15:51:27 **You Asked**

¿Cómo instalo mx snapshot?

Gemini 3 Pro

La realidad técnica con **MX Snapshot** es que instalarlo fuera de su entorno nativo (MX Linux o antiX) es un dolor de cabeza considerable.

No es una aplicación aislada, sino que está profundamente entrelazada con el ecosistema de "MX Tools" y la forma específica en que esa distribución maneja el arranque en vivo y la persistencia.

Intentar forzar su instalación presenta dos grandes problemas dependiendo de tu base:

- **En Arch / Manjaro:** Directamente no existe en los repositorios oficiales ni en AUR. Portarlo requeriría reescribir gran parte de sus scripts de empaquetado, ya que están pensados para la arquitectura de paquetes `.deb`.
- **En Debian:** Aunque comparten formato de paquetes, requeriría mezclar los repositorios de MX Linux con los de tu sistema. Esto suele desencadenar un auténtico infierno de dependencias a bajo nivel. Es casi seguro que te enfrentarías a conflictos severos con librerías base como `glibc` o incompatibilidades con `libnm` que terminarían rompiendo tu conexión de red o el entorno de escritorio completo.

Si buscas una herramienta gráfica confiable para empaquetar un sistema personalizado sin el riesgo de destruir tu entorno actual por conflictos de dependencias, lo ideal es usar herramientas nativas para tu base.

Aquí tienes cómo instalar las alternativas correctas:

Para base Debian: Instalar Refracta Snapshot

Refracta es la opción más limpia porque está diseñada exactamente para el ecosistema Debian/Devuan y no secuestra tus librerías base.

1. Asegúrate de tener las dependencias necesarias:

```bash
sudo apt update
sudo apt install rsync squashfs-tools live-boot live-config
```

2. Descarga el paquete oficial desde el repositorio de Refracta (revisa siempre la última versión en su sitio web, pero el proceso estándar es este):

```bash
wget https://sourceforge.net/projects/refracta/files/tools/refractasnapshot-gui_10.2.2_all.deb
wget https://sourceforge.net/projects/refracta/files/tools/refractasnapshot-base_10.2.2_all.deb
```

3. Instala los paquetes:

```bash
sudo dpkg -i refractasnapshot-base_*.deb refractasnapshot-gui_*.deb
sudo apt --fix-broken install
```

Una vez instalado, lo lanzas desde el menú de tu entorno gráfico (o ejecutando `refractasnapshot-gui` en la terminal) y el asistente te guiará para excluir directorios o limpiar

configuraciones de hardware.

Para base Arch / Manjaro: Instalar Systemback

Systemback es la opción gráfica más sólida aquí y está empaquetada lista para compilar en AUR.

1. Asegúrate de tener instalado tu *helper* de AUR favorito (como `yay` o `paru`).
2. Ejecuta la instalación:

```bash
yay -S systemback
```

3. El proceso compilará las dependencias necesarias. Una vez finalizado, puedes abrir la interfaz gráfica y seleccionar "Crear sistema Live".

¿Sobre qué base específica estás trabajando ahora mismo para este empaquetado, Arch/Manjaro o Debian? Si me confirmas, podemos revisar los parámetros exactos para excluir carpetas personales de la imagen final.

2026-03-28 16:19:52 **You Asked**

[root@manjaro 2026-03-28 16:19:34 sábado ~]$
> apt install /otro/descargas/refractasnapshot-gui_10.4.1_all.deb xorriso yad dosfstools refractasnapshot-base
Nota, seleccionando «refractasnapshot-gui» en lugar de «/otro/descargas/refractasnapshot-gui_10.4.1_all.deb»
El paquete refractasnapshot-base no está disponible, pero algún otro paquete hace referencia
a él. Esto puede significar que el paquete falta, está obsoleto o solo se
encuentra disponible desde alguna otra fuente

Error: El paquete «refractasnapshot-base» no tiene un candidato para la instalación

Gemini Fast

Veo que estás intentando usar `apt` en un entorno que parece ser **Manjaro**, pero con una estructura de comandos de **Debian**. El error ocurre porque tu sistema no encuentra el paquete

`refractasnapshot-base` en sus repositorios actuales, ya que es una dependencia obligatoria para que la interfaz gráfica (la GUI) funcione.

Para resolver esto en tu sistema actual, tienes dos caminos dependiendo de qué archivos descargaste:

Opción A: Instalar ambos paquetes locales (Recomendado)

`refractasnapshot-gui` es solo la interfaz; la "inteligencia" del programa está en el paquete `base`. Si descargaste el archivo `.deb` de la GUI, lo más probable es que también necesites el archivo `.deb` del base.

Si tienes ambos en tu carpeta de descargas, instálalos juntos para que `apt` resuelva la dependencia localmente:

```bash
sudo apt install ./refractasnapshot-base_10.4.1_all.deb ./refractasnapshot-gui_10.4.1_all.deb xorriso yad dosfstools
```

(Asegúrate de que los nombres de los archivos coincidan exactamente con lo que tienes en `/otro/descargas/`).

Opción B: Si solo tienes el archivo de la GUI

Si no tienes el archivo `.deb` del base, `apt` intentará buscarlo en los repositorios de Manjaro/Debian y no lo encontrará porque no es un paquete estándar. Necesitas descargarlo manualmente desde el repositorio de Refracta:

1. **Descarga el componente base:**

   ```bash
   wget https://sourceforge.net/projects/refracta/files/tools/refractasnapshot-base_10.4.1_all.deb
   ```

2. **Instala todo el conjunto:**

   ```bash
   sudo apt install ./refractasnapshot-base_10.4.1_all.deb /otro/descargas/refractasnapshot-gui_10.4.1_all.deb xorriso yad dosfstools
   ```

Consideración Importante

Al estar ejecutando esto en una terminal que se identifica como `[root@manjaro...]`, hay un detalle técnico crítico: **Manjaro no usa `apt` nativamente** (usa `pacman`). Si estás viendo un prompt de `apt`, es muy probable que estés trabajando dentro de un **chroot** o un contenedor de Debian/Devuan (quizás como parte de los preparativos para tu ISO personalizada).

Si ese es el caso, asegúrate de que el chroot tenga acceso a internet o que los archivos `.deb` estén físicamente dentro de la estructura de carpetas que el chroot puede ver.

¿Te gustaría que verifiquemos si el entorno chroot tiene montadas correctamente las rutas necesarias para que la instalación no falle al intentar configurar el grub o el kernel?

2026-03-28 16:27:35 **You Asked**

Resumen:
Actualizando: 0, Instalando 0, Reinstalando 1, Eliminando: 0, no actualizando: 5
Tamaño de la descarga: 11,8 kB
Espacio necesario: 0 B / 27,3 GB disponible

Des:1 http://deb.debian.org/debian sid/main amd64 hicolor-icon-theme all 0.18-2 [11,8 kB]
Descargados 11,8 kB en 0s (252 kB/s)
(Leyendo la base de datos ... 17569 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../hicolor-icon-theme_0.18-2_all.deb ...
Desempaquetando hicolor-icon-theme (0.18-2) sobre (0.18-2) ...
Configurando hicolor-icon-theme (0.18-2) ...
Error: Se alcanzó el tiempo de expiración

Gemini Fast

El error de "**Se alcanzó el tiempo de expiración**" (timeout) al configurar un paquete básico como `hicolor-icon-theme` suele indicar un problema con los **disparadores (triggers)** del sistema de archivos o con la comunicación entre el entorno `chroot` y el sistema anfitrión.

Como estás trabajando en un entorno híbrido (Manjaro/Debian) para tu proyecto de ISO personalizada, es muy probable que el proceso de configuración se haya quedado "colgado" intentando actualizar el caché de iconos o interactuar con un servicio que no está disponible en el entorno actual.

Aquí tienes los pasos para desbloquear la base de datos de paquetes y completar la instalación:

1. Limpiar el estado de APT y DPKG

Primero, debemos forzar la configuración de los paquetes que quedaron a medias:

```bash
# Forzar la configuración de paquetes pendientes
dpkg --configure -a

# Limpiar el caché de descargas por si hay archivos corruptos
apt clean
```

2. Verificar los montajes del CHROOT

Si estás instalando esto dentro de un `chroot` (lo cual parece ser el caso por el prompt y el uso de `apt` en Manjaro), el error de timeout ocurre frecuentemente porque faltan sistemas de archivos virtuales esenciales. Asegúrate de que desde el sistema anfitrión (Manjaro) hayas montado lo siguiente antes de entrar al entorno:

```bash
# Ejecutar desde tu sistema Manjaro principal (ajusta /ruta/al/chroot)
mount --bind /dev /ruta/al/chroot/dev
mount --bind /proc /ruta/al/chroot/proc
mount --bind /sys /ruta/al/chroot/sys
mount --bind /dev/pts /ruta/al/chroot/dev/pts
```

3. Reintentar la instalación sin disparadores pesados

Si el error persiste con `hicolor-icon-theme`, puedes intentar instalar los paquetes de Refracta ignorando temporalmente los triggers de la interfaz gráfica para evitar el bloqueo:

```bash
# Instala los paquetes .deb locales directamente con dpkg para ver el error real
dpkg -i /otro/descargas/refractasnapshot-base_*.deb /otro/descargas/refractasnapshot-gui_*.deb

# Si faltan dependencias, corrígelas
apt install -f
```

4. Posible conflicto de red o resolución

El log muestra que pudiste descargar 11.8 kB, por lo que tienes red, pero el "timeout" en la configuración a veces se debe a que el script de post-instalación intenta contactar con el bus de mensajes del sistema (`dbus`) y este no responde.

¿Estás ejecutando esta terminal dentro de una sesión gráfica activa o es una terminal pura (TTY)? Si estás en un entorno gráfico, intenta ejecutar la instalación con `sudo` pero asegurándote de que el `chroot` no intente iniciar servicios que ya están corriendo en Manjaro.

¿Te gustaría que verifiquemos si el archivo de configuración de `refractasnapshot` necesita alguna ruta específica para encontrar tu partición `/otro/descargas` una vez que inicies la GUI?

2026-03-28 16:29:21 **You Asked**

¿Cómo desactivo el disparador de hicolor-icon?

Gemini Fast

Para desactivar o evadir el disparador (*trigger*) de `hicolor-icon-theme` y desbloquear la instalación, tienes dos caminos: uno "suave" para saltar el error y uno "agresivo" si el sistema sigue bloqueado.

El problema es que ese paquete ejecuta `gtk-update-icon-cache`, que en entornos `chroot` suele quedarse esperando al bus de datos del sistema o a un servidor X que no responde.

Opción 1: El método del "Falso Ejecutable" (Recomendado)

Este es el truco más efectivo en entornos de construcción de ISOs. Engañamos al sistema creando un ejecutable vacío que no haga nada cuando el disparador intente actualizar los iconos.

1. **Crea un alias temporal para el actualizador de iconos:**

```bash
ln -sf /bin/true /usr/bin/gtk-update-icon-cache
```

2. **Configura los paquetes pendientes:**

```bash
dpkg --configure -a
```

3. **Continúa con tu instalación:**

```bash
apt install -f
```

Una vez que termines de instalar todo, puedes borrar el enlace simbólico si deseas que el sistema vuelva a la normalidad, aunque para crear la ISO no te estorbará.

Opción 2: Edición manual del archivo de Triggers (Si el error persiste)

Si el instalador sigue intentando ejecutar el disparador a pesar del paso anterior, puedes comentarlo directamente en la base de datos de `dpkg`.

1. Abre el archivo de disparadores:

```bash
nano /var/lib/dpkg/info/hicolor-icon-theme.postinst
```

2. Busca la línea que contiene el comando `gtk-update-icon-cache` (generalmente al final del archivo).
3. Añade un `#` al principio de esa línea para comentarla o simplemente añade un `exit 0` al inicio del script:

```bash
#!/bin/sh
exit 0  # <--- Añade esto aquí
set -e
...
```

4. Guarda (**Ctrl+O**, **Enter**) y sal (**Ctrl+X**).
5. Ejecuta de nuevo:

```bash
dpkg --configure -a
```

¿Por qué te sucede esto en tu proyecto "frankeinux"?

Al estar mezclando una base **Debian Sid** (como se ve en tu log) dentro de un entorno **Manjaro**, el `chroot` hereda ciertas variables de entorno pero no los servicios. Cuando `apt` intenta registrar los iconos nuevos en el sistema de archivos, el proceso se queda en un bucle infinito buscando una respuesta del subsistema gráfico.

¿Quieres que después de solucionar esto te ayude a verificar si el archivo de configuración de Refracta Snapshot está apuntando correctamente a la ruta de exclusión de `/home` para que la ISO no quede demasiado pesada?

2026-03-28 16:36:34 **You Asked**

¿Cómo instalo mx snapshot?

Gemini Fast

Instalar **MX Snapshot** fuera de MX Linux es un reto técnico importante debido a que depende de un ecosistema de scripts y configuraciones de arranque (como `mx-live-usb-maker` y el sistema de remezcla de MX) que no son estándar en Debian o Arch/Manjaro.

Sin embargo, si estás decidido a intentarlo en tu entorno actual (que parece ser un **chroot de Debian Sid** corriendo sobre **Manjaro**), aquí tienes la ruta para hacerlo de la manera más limpia posible:

1. Agregar el repositorio de MX Linux

Dado que `apt` no encontrará el paquete en los repositorios de Debian, debes decirle dónde buscar.

1. Crea un archivo de lista de fuentes:

```bash
nano /etc/apt/sources.list.d/mx.list
```

2. Pega la siguiente línea (para la rama principal de MX basada en Debian Stable/Bookworm, que suele ser la más compatible):

```text
deb http://mx.mxrepo.com/mx/repo/ bookworm main non-free
```

3. Agrega la llave GPG del repositorio para que `apt` confíe en los paquetes:

```bash
apt-key adv --keyserver keyserver.ubuntu.com --recv-keys F13E95F1
```

2. Configurar el "Pinning" (Crucial)

Peligro: Si haces `apt upgrade` ahora, tu sistema intentará reemplazar paquetes de Debian con versiones de MX, lo que romperá tu instalación. Debes configurar `apt` para que **solo** descargue MX Snapshot de ese repositorio.

1. Crea un archivo de preferencias:

```bash
nano /etc/apt/preferences.d/mx-snapshot
```

2. Pega lo siguiente:

```text
Package: *
Pin: release o=MX
Pin-Priority: -10

Package: mx-snapshot
Pin: release o=MX
Pin-Priority: 500
```

3. Instalación

Ahora actualiza e intenta la instalación:

```bash
apt update
apt install mx-snapshot
```

¿Qué problemas podrías encontrar?

1. **Dependencias incumplidas:** `mx-snapshot` te pedirá `mx-iso-template`, `mx-live-utils` y posiblemente otros paquetes. Gracias al "Pinning" del paso anterior, `apt` debería traer solo lo necesario.
2. **El Sistema de Arranque:** MX Snapshot está diseñado para trabajar con el cargador de arranque de MX Linux. Al usarlo en tu proyecto *frankeinux*, es posible que la ISO genere errores al intentar iniciar el kernel si no encuentra los scripts de `live-config` específicos de MX.
3. **Conflictos de Librerías:** Como estás usando **Debian Sid** (inestable) y el repo de MX es **Stable**, podrías tener errores de "Versión de glibc no encontrada".

Mi recomendación para tu flujo de trabajo

Si lo que buscas es una **GUI confiable** que no rompa tu sistema de archivos mientras trabajas en el chroot:

- **Refracta Snapshot** es mucho más "agnóstico" y se lleva mejor con Debian puro/Sid.
- **Systemback** (el fork de Franco Conidi) es excelente si logras compilarlo, ya que gestiona sus propios archivos de arranque de forma independiente.

¿Te gustaría que probemos a solucionar las dependencias de Refracta Snapshot ahora que ya sabemos cómo saltar el bloqueo del disparador de iconos?

2026-03-28 16:39:58 **You Asked**

[root@manjaro 2026-03-28 16:37:16 sábado ~]$
> deb http://mx.mxrepo.com/mx/repo/ bookworm main non-free
bash: deb: orden no encontrada
[root@manjaro 2026-03-28 16:37:17 sábado ~]$
> nano /etc/apt/sources.list.d/mx.list
[root@manjaro 2026-03-28 16:37:31 sábado ~]$
> apt-key adv --keyserver keyserver.ubuntu.com --recv-keys F13E95F1
bash: apt-key: orden no encontrada
[root@manjaro 2026-03-28 16:37:35 sábado ~]$
> apt key adv --keyserver keyserver.ubuntu.com --recv-keys F13E95F1
E: No tiene sentido la opción de línea de órdenes --keyserver combinada con las otras opciones
[root@manjaro 2026-03-28 16:37:40 sábado ~]$
> apt adv --keyserver keyserver.ubuntu.com --recv-keys F13E95F1
E: No tiene sentido la opción de línea de órdenes --keyserver combinada con las otras opciones
[root@manjaro 2026-03-28 16:37:55 sábado ~]$
> apt --keyserver keyserver.ubuntu.com --recv-keys F13E95F1 adv
E: No tiene sentido la opción de línea de órdenes --keyserver combinada con las otras opciones
[root@manjaro 2026-03-28 16:38:03 sábado ~]$
> apt --recv-keys F13E95F1 adv
E: No tiene sentido la opción de línea de órdenes --recv-keys combinada con las otras opciones
[root@manjaro 2026-03-28 16:38:10 sábado ~]$
> apt --keyserver keyserver.ubuntu.com --recv-keys F13E95F1
E: No tiene sentido la opción de línea de órdenes --keyserver combinada con las otras opciones
[root@manjaro 2026-03-28 16:38:22 sábado ~]$
> apt --keyserver keyserver.ubuntu.com
E: No tiene sentido la opción de línea de órdenes --keyserver combinada con las otras opciones

```
[root@manjaro 2026-03-28 16:38:26 sábado ~]$
 > apt --recv-keys F13E95F1
E: No tiene sentido la opción de línea de órdenes --recv-keys
combinada con las otras opciones
[root@manjaro 2026-03-28 16:38:36 sábado ~]$
 > apt install adv
Error: No se ha podido localizar el paquete adv
[root@manjaro 2026-03-28 16:38:46 sábado ~]$
 > nano /etc/apt/preferences.d/mx-snapshot
[root@manjaro 2026-03-28 16:39:03 sábado ~]$
 > apt update
apt install mx-snapshot
Obj:1 http://deb.debian.org/debian sid InRelease
Err:2 http://mx.mxrepo.com/mx/repo bookworm InRelease
  No se pudo resolver «mx.mxrepo.com»
Todos los paquetes están actualizados.
Advertencia: Fallo al obtener
http://mx.mxrepo.com/mx/repo/dists/bookworm/InRelease  No se
pudo resolver «mx.mxrepo.com»
Advertencia: No se ha podido descargar algunos archivos de índice,
se han omitido, o se han utilizado unos antiguos en su lugar.
Información: Algunas fuentes se pueden modernizar. Ejecute «apt
modernize-sources» para hacerlo.
Error: No se ha podido localizar el paquete mx-snapshot
```

Gemini Thinking

Parece que estás teniendo la "tormenta perfecta" de problemas de configuración en el `chroot`: el sistema no tiene internet (DNS), `apt-key` está desaparecido porque es una herramienta obsoleta en Debian moderno, y la sintaxis de los comandos se mezcló un poco.

Vamos a solucionar esto paso a paso para que `apt` pueda ver el repositorio de MX.

1. Arreglar el internet (DNS) en el chroot

El error `No se pudo resolver «mx.mxrepo.com»` significa que tu chroot no sabe cómo traducir nombres de dominio a IPs.

Desde otra terminal en tu Manjaro (fuera del chroot): Ejecuta esto para copiar la configuración de red de tu sistema real al chroot:

```bash
```

```
sudo cp /etc/resolv.conf /ruta/a/tu/chroot/etc/resolv.conf
```

(Sustituye `/ruta/a/tu/chroot` por la carpeta donde está tu proyecto frankeinux).

---

2. Agregar la llave GPG (El modo moderno)

Como `apt-key` ya no se usa y tu terminal no lo encuentra, debemos usar `gpg` directamente para descargar la llave y guardarla en un "keyring".

Dentro del chroot:

```bash
# Primero instalamos gnupg si no está
apt update || true # Ignora los errores de MX por ahora
apt install gnupg

# Descargamos la llave de MX
gpg --keyserver keyserver.ubuntu.com --recv-keys F13E95F1

# La exportamos al formato que le gusta a Debian moderno
gpg --export F13E95F1 > /usr/share/keyrings/mxlinux-archive-keyring.gpg
```

---

3. Corregir el archivo de repositorio

La línea `deb http://...` no es un comando de bash, es una línea de configuración. Debemos asegurarnos de que el archivo que creaste con `nano` sea correcto y use la llave que acabamos de descargar.

1. Abre el archivo: `nano /etc/apt/sources.list.d/mx.list`
2. Borra lo que tenga y pega **exactamente** esto:

   ```text
   deb [signed-by=/usr/share/keyrings/mxlinux-archive-keyring.gpg]
   http://mx.mxrepo.com/mx/repo/ bookworm main non-free
   ```

3. Guarda y sal.

---

4. Instalar MX Snapshot

Ahora que ya tienes DNS y la llave configurada, los comandos deberían funcionar:

```bash
```

```
apt update
apt install mx-snapshot
```

¿Por qué fallaron tus intentos anteriores?

- **`bash: deb: orden no encontrada`** : Intentaste pegar la dirección del repositorio directamente en la terminal como si fuera un programa.
- **`apt --keyserver...`** : `apt` es un gestor de paquetes, no sabe hablar con servidores de llaves. Esa es tarea de `gpg` o `apt-key` .
- **`apt-key: orden no encontrada`** : En las versiones más nuevas de Debian (como Sid), `apt-key` ha sido eliminado por seguridad. Ahora se prefiere el método de `/usr/share/keyrings/` que te puse arriba.

Siguiente paso crítico: Una vez que instales `mx-snapshot` , ¿quieres que revisemos si necesitas instalar también `mx-iso-template` para que la ISO tenga un menú de arranque funcional? (Sin el template, la ISO podría crearse pero no bootear).

2026-03-28 16:53:12 **You Asked**

write(2, "+ mksquashfs myfs/ iso/live/file"..., 58) = 58
newfstatat(AT_FDCWD, "/usr/local/sbin/mksquashfs", 0x7ffc98f08f90, 0) = -1 ENOENT (No existe el fichero o el directorio)
newfstatat(AT_FDCWD, "/usr/local/bin/mksquashfs", 0x7ffc98f08f90, 0) = -1 ENOENT (No existe el fichero o el directorio)
newfstatat(AT_FDCWD, "/usr/sbin/mksquashfs", {st_mode=S_IFREG|0755, st_size=393464, ...}, 0) = 0
newfstatat(AT_FDCWD, "/usr/sbin/mksquashfs", {st_mode=S_IFREG|0755, st_size=393464, ...}, 0) = 0
access("/usr/sbin/mksquashfs", X_OK) = 0
newfstatat(AT_FDCWD, "/usr/sbin/mksquashfs", {st_mode=S_IFREG|0755, st_size=393464, ...}, 0) = 0
access("/usr/sbin/mksquashfs", R_OK) = 0
[pid 138274] execve("/usr/sbin/mksquashfs", ["mksquashfs", "myfs/", "iso/live/filesystem.squashfs", "-noappend"], 0x55ab45598850 /* 50 vars */) = 0
[pid 138275] execve("/usr/sbin/yad", ["yad", "--title=Error", "--image=gtk-dialog-error", "--button=OK:0", "--center", "--text=Error: mksquashfs failed."...], 0x55ab45598850 /* 50 vars */) = 0

[root@manjaro 2026-03-28 16:52:33 sábado /]$
> mksquashfs myfs/ iso/live/filesystem.squashfs -noappend
Cannot stat source directory "myfs/" because No such file or directory

Gemini Thinking

Viendo el log de `strace` y el error final, el problema es puramente de **rutas (paths)**. Estás parado en la raíz del sistema (`/`), pero el comando espera que las carpetas `myfs/` e `iso/` existan justo ahí, en el directorio actual.

El comando falló porque `mksquashfs` no encuentra el origen de los datos para comprimir.

El problema en detalle

Tu prompt indica que estás en: `[root@manjaro ... /]$` Y el comando ejecutado fue: `mksquashfs myfs/ iso/live/filesystem.squashfs -noappend`

Para que ese comando funcione, dentro de `/` deberían existir:

1. Una carpeta llamada `myfs` (donde está tu sistema Debian/frankeinux).
2. Una carpeta llamada `iso` con una subcarpeta `live` (donde se guardará el resultado).

Cómo solucionarlo

1. Localiza tu carpeta de trabajo Si estás siguiendo la estructura de herramientas como `penguins-eggs` o scripts manuales, es probable que tu sistema de archivos no esté en `/myfs`, sino en una ruta específica. Busca dónde están esas carpetas:

```bash
find / -maxdepth 3 -name "myfs" -type d 2>/dev/null
```

2. Cambia al directorio correcto Una vez que sepas dónde están, entra en esa carpeta antes de lanzar el comando. Por ejemplo:

```bash
cd /ruta/a/tu/proyecto/frankeinux/
# Ahora sí, lanza el comando:
mksquashfs myfs/ iso/live/filesystem.squashfs -noappend
```

3. O usa rutas absolutas Si prefieres no moverte de directorio, dile a `mksquashfs` exactamente dónde está todo:

```bash
mksquashfs /ruta/completa/myfs /ruta/completa/iso/live/filesystem.squashfs -noappend
```

Un detalle sobre `yad`

En tu `strace` se ve que el sistema intentó lanzar `yad` (la herramienta para diálogos gráficos) para mostrarte el error: `execve("/usr/sbin/yad", ["yad", "--title=Error", ...])`

Si no viste la ventana de error, es porque `yad` probablemente falló al no tener un servidor X (pantalla) asignado dentro del `chroot`. Esto confirma que, aunque estás instalando herramientas con GUI, el entorno de terminal actual todavía no tiene permisos para "dibujar" ventanas en tu escritorio de Manjaro.

¿Te gustaría que veamos cómo pasarle la variable `DISPLAY` al chroot para que las ventanas de Refracta o MX Snapshot aparezcan correctamente en tu pantalla?

2026-03-28 16:55:14 **You Asked**

¿Cómo hago para un sistema debian estándar?

compressed data, compressed metadata, compressed fragments,
compressed xattrs, compressed ids
duplicates are removed
Filesystem size 0.15 Kbytes (0.00 Mbytes)
99.37% of uncompressed filesystem size (0.15 Kbytes)
Inode table size 29 bytes (0.03 Kbytes)
85.29% of uncompressed inode table size (34 bytes)
Directory table size 0 bytes (0.00 Kbytes)
100% of uncompressed directory table size (0 bytes)
Number of duplicate files found 0
Number of inodes 1
Number of files 0
Number of fragments 0
Number of symbolic links 0
Number of device nodes 0
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 1
Number of hard-links 0

```
Number of ids (unique uids + gids) 1
Number of uids 1
        root (0)
Number of gids 1
        root (0)
[root@manjaro 2026-03-28 16:54:24 sábado /]$
 > ls
bin          files.plist    lib        media       reparar_libc_0.sh  share
 vmlinuz
boot         home           lib32      mnt         reparar_libc_1.sh  srv
 vmlinuz.old
bootx64.efi  initrd.img     lib64      myfs        reparar_libc_2.sh
sudo
core.efi     initrd.img.old libx32     opt         reparar_libc_3.sh  sys
debian-binary iso           local      otro        resolv.conf        tmp
debian_trixie j             log2.txt   pkg         root               trixie
dev          k              log_3.txt  proc        run                usr
etc          ld.so.cache    ls_manjaro  props.plist sbin              var
```

Gemini Thinking

El problema que ves en el reporte (donde dice `Number of files 0`) es que el comando `mksquashfs` se ejecutó sobre una carpeta vacía o mal direccionada. En tu `ls` veo que tienes directorios como `myfs`, `trixie` y `debian_trixie`.

Si quieres crear una imagen **SquashFS** de un sistema Debian estándar que sea funcional, tienes dos caminos dependiendo de qué quieras empaquetar exactamente:

Opción 1: Empaquetar el sistema en el que estás "dentro" (El Chroot)

Si tu intención es clonar todo el entorno Debian donde estás ejecutando los comandos ahora mismo, el origen es la raíz `/`.

¡Cuidado! Debes excluir las carpetas virtuales y la carpeta donde vas a guardar la ISO, de lo contrario entrarás en un bucle infinito o la imagen pesará terabytes.

Usa este comando (asumiendo que quieres guardar el resultado en `/iso/live/`):

```bash
mksquashfs / /iso/live/filesystem.squashfs -e proc sys dev run tmp mnt media iso myfs trixie debian_trixie otro
```

- **-e** : Excluye los directorios que no deben ir en una ISO (sistemas de archivos dinámicos y tus propias carpetas de trabajo).
- **proc sys dev run** : Son esenciales para que el sistema funcione, pero se crean al arrancar; no deben estar "quemados" en la imagen.

Opción 2: Empaquetar una instalación limpia (Ej: la carpeta `trixie`)

Si lo que hiciste fue un `debootstrap` en una carpeta específica para que el sistema esté "limpio" (estándar), el comando debe apuntar a esa carpeta. Según tu `ls`, parece que es `/trixie` o `/debian_trixie`.

```bash
mksquashfs /trixie /iso/live/filesystem.squashfs -noappend -wildcards
```

Verificación de la estructura para la ISO

Para que esa imagen SquashFS sea parte de una ISO que realmente arranque, tu carpeta `/iso` debería verse más o menos así antes de generar el archivo final con `xorriso` o `genisoimage`:

- **/iso/live/filesystem.squashfs** : (El archivo que acabamos de crear).
- **/iso/live/vmlinuz** : El kernel (puedes copiar el de la raíz de tu `ls`).
- **/iso/live/initrd.img** : El sistema de archivos inicial.
- **/iso/boot/grub/** o **/iso/isolinux/** : Los archivos del cargador de arranque.

¿Por qué te dio "0 files" antes?

En tu captura anterior, ejecutaste `mksquashfs myfs/ ...`. Como se ve en tu `ls` actual, `myfs` es una carpeta en la raíz, pero probablemente está vacía o es un punto de montaje que no estaba activo en ese momento.

Consejo para tu proyecto "frankeinux": Como estás manejando reparaciones manuales de `glibc` (veo tus scripts `reparar_libc_*.sh` en el `ls`), asegúrate de que antes de correr el `mksquashfs`, el sistema dentro de la carpeta objetivo sea capaz de hacer un `chroot` y ejecutar un comando simple como `ls` sin errores de librerías. Si el `chroot` falla por las librerías, la ISO resultante también fallará al arrancar.

¿Te gustaría que preparemos el comando de `xorriso` para convertir esa carpeta `/iso` en una imagen `.iso` arrancable para USB y BIOS/UEFI?

2026-03-28 17:02:37 **You Asked**

Sí. Usé la opción 1.

Gemini Thinking

Excelente. Si usaste la **Opción 1** (empaquetar el sistema desde la raíz `/`), ahora deberías tener un archivo `filesystem.squashfs` con un tamaño real (probablemente varios GB) en `/iso/live/`.

Para que ese archivo se convierta en una **ISO arrancable** (el paso final de tu proyecto *frankeinux*), necesitamos preparar la estructura de arranque y luego usar `xorriso`.

Sigue estos pasos dentro de tu terminal:

1. Preparar los archivos de arranque

Para que la ISO inicie, necesita el Kernel y el Disco RAM inicial dentro de la carpeta `/iso/live`. Según tu `ls`, los tienes en la raíz:

```bash
# Copiar el kernel y el initrd a la carpeta de la ISO
cp /vmlinuz /iso/live/vmlinuz
cp /initrd.img /iso/live/initrd.img
```

2. Crear una configuración básica de GRUB

Necesitamos decirle a la ISO qué archivos cargar al arrancar. Crea el directorio y el archivo de configuración:

```bash
mkdir -p /iso/boot/grub
nano /iso/boot/grub/grub.cfg
```

Pega este contenido (ajustado para un sistema Live estándar):

```text
set default=0
set timeout=5

menuentry "Frankeinux Live (Debian Sid/Trixie)" {
    linux /live/vmlinuz boot=live quiet splash
    initrd /live/initrd.img
}
```

3. Generar la ISO con `xorriso`

Ahora vamos a empaquetar todo. Este comando creará una ISO híbrida (funciona en USB y CD, y es compatible con BIOS y UEFI si tienes instalados los paquetes `mtools` y `libisoboot`):

```bash
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -eltorito-boot boot/grub/bios.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  -output /frankeinux_dist.iso \
  /iso
```

Nota: Si el comando falla por `bios.img`, puedes usar una versión más simplificada que delegue el arranque al sistema operativo que lo "queme" en el USB:

```bash
xorriso -as mkisofs -o /frankeinux_dist.iso -J -R -V "FRANKEINUX" /iso
```

Verificaciones finales antes de quemar:

1. **Tamaño:** Revisa con `ls -lh /frankeinux_dist.iso` que el archivo pese algo razonable (ej. 1.5GB - 3GB).
2. **Librerías:** Como has estado trabajando en reparaciones de `glibc`, lo ideal es que antes de este paso hayas corrido un `ldconfig` dentro del chroot para asegurar que todos los enlaces simbólicos de las librerías estén correctos.
3. **Persistencia:** Al usar `boot=live` en el GRUB, la ISO buscará automáticamente el archivo `/live/filesystem.squashfs`.

¿Quieres que veamos cómo añadir soporte específico para UEFI (EFI) por si piensas probar la ISO en hardware más moderno?

2026-03-29 15:01:38 **You Asked**

¿Está bien así?

#!/bin/sh

echo "set default=0

```
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img
}" > "/iso/boot/grub/grub.cfg"

mksquashfs / /iso/live/filesystem.squashfs -e proc sys dev run
tmp mnt media iso trixie debian_trixie otro

xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -eltorito-boot /boot/grub/i386-coreboot/kernel.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  -output /frankeinux_dist.iso \
  /iso
```

Gemini Thinking

Vas por muy buen camino, Eze. El script tiene la lógica correcta, pero hay **dos detalles críticos** que podrían hacer que la ISO no arranque o que el proceso entre en un bucle infinito ("trampa recursiva").

Aquí tienes las correcciones necesarias:

1. La "Trampa Recursiva" (Excluir la ISO de salida)

En tu comando `mksquashfs`, estás excluyendo la carpeta `/iso`, lo cual es perfecto. Sin embargo, tu archivo de salida de `xorriso` es `/frankeinux_dist.iso`.

- **El problema:** `mksquashfs` intentará meter ese archivo `.iso` dentro de la imagen mientras se está creando.
- **La solución:** Añade `frankeinux_dist.iso` a la lista de exclusiones.

2. El Cargador de Arranque (El Torito)

Estás usando `-eltorito-boot /boot/grub/i386-coreboot/kernel.img`.

- **El problema:** Esa ruta es muy específica para sistemas con firmware **Coreboot**. Si quieres que la ISO arranque en una PC normal (BIOS/CSM), esa imagen no funcionará.
- **La solución:** Lo más estándar para una ISO de Debian con GRUB es generar una imagen El Torito para BIOS.

3. Falta copiar el Kernel e Initrd

Tu `grub.cfg` busca los archivos en `/live/vmlinuz` y `/live/initrd.img` (dentro de la ISO), pero tu script no los está copiando allí antes de correr `xorriso`.

Script Corregido y Optimizado

```bash
#!/bin/sh

# 1. Crear directorios necesarios
mkdir -p /iso/boot/grub
mkdir -p /iso/live

# 2. Generar el grub.cfg
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img
}" > "/iso/boot/grub/grub.cfg"

# 3. COPIAR EL KERNEL Y EL INITRD (Crucial)
# Usamos el kernel actual del sistema
cp /vmlinuz /iso/live/vmlinuz
cp /initrd.img /iso/live/initrd.img

# 4. Crear el SquashFS
# Añadimos la exclusión del archivo ISO final para evitar recursividad
mksquashfs / /iso/live/filesystem.squashfs \
  -e proc sys dev run tmp mnt media iso trixie debian_trixie otro frankeinux_dist.iso

# 5. Generar la ISO con xorriso (Versión compatible BIOS/Híbrida)
# He simplificado el arranque para que sea más "universal"
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /frankeinux_dist.iso \
  -graft-points \
  -b boot/grub/grub_eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

Notas adicionales para tu proyecto:

- **El archivo `grub_eltorito.img`:** Si no tienes ese archivo, una forma rápida de generar una ISO funcional sin complicarte con el binario de arranque es usar simplemente: `xorriso -as mkisofs -o /frankeinux_dist.iso -J -R -V "FRANKEINUX" /iso` Esto creará una ISO que quizás no arranque en BIOS viejas por sí sola, pero es útil para probar si el contenido está bien.
- **Permisos:** Asegúrate de que el script tenga permisos de ejecución (`chmod +x script.sh`) y que los archivos `/vmlinuz` e `/initrd.img` en tu raíz tengan permisos de lectura para todos.

¿Te gustaría que te pase un comando para generar automáticamente el archivo de arranque de GRUB (`grub_eltorito.img`) si no lo encuentras en tu sistema?

2026-03-29 15:07:47 **You Asked**

```
[root@manjaro 2026-03-29 15:05:44 domingo ~]$
 > apt reinstall grub-pc-bin
Instalando:
  grub-pc-bin

Resumen:
  Actualizando: 0, Instalando 1, Eliminando: 0, no actualizando: 0
  Tamaño de la descarga: 1.236 kB
  Espacio necesario: 3.906 kB / 1.352 MB disponible

Des:1 http://deb.debian.org/debian sid/main amd64 grub-pc-bin
amd64 2.14-2 [1.236 kB]
Descargados 1.236 kB en 0s (2.726 kB/s)
Seleccionando el paquete grub-pc-bin previamente no
seleccionado.
(Leyendo la base de datos ... 213737 ficheros o directorios
instalados actualmente.)
Preparando para desempaquetar .../grub-pc-bin_2.14-2_amd64.deb
...
Desempaquetando grub-pc-bin (2.14-2) ...
Configurando grub-pc-bin (2.14-2) ...
Error: Se alcanzó el tiempo de expiración
[root@manjaro 2026-03-29 15:06:17 domingo ~]$
 > ls -lh /usr/lib/grub/i386-pc/eltorito.img
```

-rw-r--r-- 1 root root 32K feb 10 08:27 /usr/lib/grub/i386-pc/eltorito.img

Gemini Thinking

¡Buenas noticias, Eze! Ya tenés el archivo clave. Ese **eltorito.img** de 32K es exactamente lo que necesitamos para que la ISO sea booteable en máquinas con BIOS tradicional.

El error de "**tiempo de expiración**" al configurar grub-pc-bin es un clásico en entornos chroot. Ocurre porque el script de post-instalación de GRUB intenta detectar discos físicos (os-prober o grub-probe) y, al no estar en un sistema real con acceso directo al hardware, se queda esperando una respuesta que nunca llega.

Como el archivo ya existe en el disco, podés ignorar el error de configuración por ahora. Vamos a ajustar tu script para que use esa ruta exacta.

Script de Frankeinux (Versión Final con El Torito)

Copia y pega esto en tu archivo .sh . He ajustado las rutas para que coincidan con lo que encontraste:

```bash
#!/bin/sh

# 1. Limpiar y preparar directorios
mkdir -p /iso/boot/grub
mkdir -p /iso/live

# 2. Generar el grub.cfg (Rutas relativas a la raíz de la ISO)
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img
}" > "/iso/boot/grub/grub.cfg"

# 3. Copiar archivos críticos de arranque
cp /vmlinuz /iso/live/vmlinuz
cp /initrd.img /iso/live/initrd.img

# 4. Copiar el binario de arranque de GRUB a la carpeta de la ISO
cp /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/eltorito.img

# 5. Crear el SquashFS (Sistema de archivos comprimido)
```

```
mksquashfs / /iso/live/filesystem.squashfs \
  -e proc sys dev run tmp mnt media iso trixie debian_trixie otro frankeinux_dist.iso

# 6. Crear la ISO híbrida con xorriso
# Nota: -b indica el archivo de arranque RELATIVO a la raíz de la carpeta /iso
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /frankeinux_dist.iso \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  /iso
```

¿Por qué esto debería funcionar ahora?

1. **Independencia del paquete:** No importa que `apt` haya dado timeout. Ya "robamos" el archivo `eltorito.img` que necesitábamos.
2. **Ruta relativa:** En el comando `xorriso`, el parámetro `-b` espera la ruta dentro de la ISO. Por eso copiamos el archivo a `/iso/boot/grub/eltorito.img` y luego le decimos a `xorriso` que lo busque en `boot/grub/eltorito.img`.
3. **Híbrida:** Al usar `xorriso` de esta manera, la imagen resultante suele ser compatible con grabaciones en USB mediante `dd` o Ventoy.

Un pequeño "Fix" para el futuro

Si te molesta ver el error de `dpkg` cada vez que usas `apt`, podés forzar que el sistema crea que está configurado:

```bash
# Solo si quieres limpiar el error de apt
sed -i 's/half-configured/installed/g' /var/lib/dpkg/status
```

¿Querés que te pase los comandos para verificar la ISO con `qemu` antes de pasarla a un pendrive?

2026-03-29 15:13:35 **You Asked**

```
#!/bin/sh

mkdir -pv /iso/boot/grub
mkdir -pv /iso/live
```

```
cp /vmlinuz /iso/live/vmlinuz
cp /initrd.img /iso/live/initrd.img

echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img
}" > "/iso/boot/grub/grub.cfg"

mksquashfs / /iso/live/filesystem.squashfs -e proc sys dev run
tmp mnt media iso trixie debian_trixie otro frankeinux_dist.iso

xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output frankeinux_dist.iso \
  -graft-points \
  -b /usr/lib/grub/i386-pc/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

Gemini Thinking

Ese script está casi perfecto y ya tiene la lógica de exclusión que necesitábamos para evitar el bucle infinito. Sin embargo, hay un detalle técnico con **xorriso** que te va a dar un error de "file not found" o "boot image not found" si lo corres así.

El parámetro `-b` (boot image) en el modo de emulación de `mkisofs` espera una ruta **relativa** a la raíz de la carpeta que estás empaquetando (en tu caso `/iso`), no una ruta absoluta del sistema real.

Aquí tienes el ajuste necesario para que el binario de arranque se integre correctamente:

Script Frankeinux (Ajustado)

```bash
```

```
#!/bin/sh

# 1. Crear directorios dentro de la estructura de la ISO
mkdir -pv /iso/boot/grub
mkdir -pv /iso/live

# 2. Copiar el kernel y el initrd a la ISO
cp -v /vmlinuz /iso/live/vmlinuz
cp -v /initrd.img /iso/live/initrd.img

# 3. COPIAR EL BINARIO DE ARRANQUE DENTRO DE LA ISO (Importante)
# xorriso necesita que el archivo de arranque esté dentro del árbol que va a procesar
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/eltorito.img

# 4. Generar el grub.cfg
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img
}" > "/iso/boot/grub/grub.cfg"

# 5. Crear el SquashFS
# Mantenemos tus exclusiones, son correctas.
mksquashfs / /iso/live/filesystem.squashfs -e proc sys dev run tmp mnt media iso trixie
debian_trixie otro frankeinux_dist.iso

# 6. Generar la ISO con xorriso
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output frankeinux_dist.iso \
  -graft-points \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

¿Qué cambió y por qué?

1. **Copia de `eltorito.img`**: Agregué el paso de copiar el archivo desde `/usr/lib/...` hacia `/iso/boot/grub/`. Esto es porque `xorriso` construye la ISO a partir de lo que hay en `/iso`. Si le pasas una ruta externa, a veces falla o no lo encuentra al intentar crear la tabla de arranque El Torito.

2. **Ruta en `-b`**: Ahora dice `-b boot/grub/eltorito.img`. Nota que **no** empieza con una barra `/`. Esto le dice a `xorriso`: *"Busca el archivo de arranque en esta ruta dentro de la carpeta que te pasé al final (`/iso`)"*.
3. **Ubicación del output**: El archivo `frankeinux_dist.iso` se creará en el directorio donde ejecutes el script. Como lo excluiste en el paso de `mksquashfs`, no hay riesgo de que intente comprimirse a sí mismo.

Una recomendación para USB (Modo Híbrido)

Si planeas "quemar" esta ISO en un pendrive con `dd` o grabarla directamente, te sugiero agregar esta línea justo antes del final del comando `xorriso` (si tienes el paquete `grub-pc-bin` instalado, que ya vimos que sí):

`–isohybrid-mbr /usr/lib/grub/i386-pc/boot_hybrid.img \`

Esto hace que la ISO sea "híbrida", permitiendo que arranque tanto como un CD-ROM como desde un bloque de arranque de disco duro (USB).

¿Te gustaría que probemos un comando rápido para verificar si la ISO arranca bien usando **QEMU** antes de que la pases a un pendrive real?

2026-03-29 15:52:32 **You Asked**

```
#!/bin/sh

mkdir -pv /iso/boot/grub
mkdir -pv /iso/live

cp -v /vmlinuz /iso/live/vmlinuz
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/eltorito.img
cp -v /boot/initrd.img-6.19-x86_64 /iso/live/initrd.img-6.19-
x86_64.zstd

echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
   linux /live/vmlinuz boot=live quiet
   initrd /live/initrd.img-6.19-x86_64.zstd
}" > "/iso/boot/grub/grub.cfg"

mksquashfs / /iso/live/filesystem.squashfs -e \
proc sys dev run tmp mnt media iso trixie debian_trixie otro \
```

```
root/.cache \
root/.config/vivaldi/Default/IndexedDB \
root/.config/vivaldi/Default/Local Extension Settings \
root/.config/vivaldi/Default/Session Storage \
root/.config/vivaldi/Default/Sessions \
root/.config/vivaldi/Default/TransportSecurity \
root/.config/vivaldi/Default/shared_proto_db \
root/.config/vivaldi/Safe Browsing \
root/.local/share/gvfs-metadata \
frankeinux_dist.iso

xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output frankeinux_dist.iso \
  -graft-points \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

Gemini Thinking

Está **muy cerca de la perfección**, Eze. Ya resolviste el rompecabezas de las rutas de `xorriso` y el binario de arranque.

Sin embargo, hay un detalle técnico en tu comando `mksquashfs` que **va a fallar** debido a los espacios en los nombres de las carpetas de Vivaldi (como `Local Extension Settings`). Bash interpretará cada palabra como un archivo distinto si no las protegemos.

Aquí tienes el script con las correcciones de sintaxis necesarias y un par de "toques de gracia" para que la ISO sea realmente profesional:

Script "Frankeinux" Corregido

```bash
#!/bin/sh
```

```
# 1. Preparar el terreno
mkdir -pv /iso/boot/grub
mkdir -pv /iso/live

# 2. Copiar el Kernel y el Initrd (con tu versión específica 6.19)
cp -v /vmlinuz /iso/live/vmlinuz
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/eltorito.img
cp -v /boot/initrd.img-6.19-x86_64 /iso/live/initrd.img-6.19-x86_64.zstd

# 3. Generar el grub.cfg
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img-6.19-x86_64.zstd
}" > "/iso/boot/grub/grub.cfg"

# 4. Crear el SquashFS (CON COMILLAS en las rutas con espacios)
mksquashfs / /iso/live/filesystem.squashfs -e \
  proc sys dev run tmp mnt media iso trixie debian_trixie otro \
  "root/.cache" \
  "root/.config/vivaldi/Default/IndexedDB" \
  "root/.config/vivaldi/Default/Local Extension Settings" \
  "root/.config/vivaldi/Default/Session Storage" \
  "root/.config/vivaldi/Default/Sessions" \
  "root/.config/vivaldi/Default/TransportSecurity" \
  "root/.config/vivaldi/Default/shared_proto_db" \
  "root/.config/vivaldi/Safe Browsing" \
  "root/.local/share/gvfs-metadata" \
  frankeinux_dist.iso

# 5. Generar la ISO con xorriso
# Agregué la bandera -R -J para que los nombres de archivos se vean bien en Windows/Linux
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output frankeinux_dist.iso \
  -J -R \
  -graft-points \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

¿Qué corregimos?

- **Comillas en las exclusiones:** Rutas como `"root/.../Local Extension Settings"` ahora están entre comillas. Sin ellas, `mksquashfs` intentaría excluir una carpeta llamada `Local`, otra llamada `Extension` y otra llamada `Settings`, lo cual daría error.
- **Extensiones RockRidge/Joliet (`-R -J`):** Agregué estas banderas a `xorriso`. Esto asegura que si abres la ISO en cualquier sistema, los nombres largos de archivos no se corten ni se conviertan en mayúsculas raras (formato 8.3).
- **Ruta de salida:** Mantuviste `frankeinux_dist.iso` en la raíz. Como ya está en tu lista de exclusión de `mksquashfs`, no tendrás problemas de recursividad.

Cómo probar tu creación

Antes de gastar un pendrive, podés probar si Frankeinux arranca directamente desde tu terminal de Manjaro usando **QEMU**:

```bash
qemu-system-x86_64 -enable-kvm -m 2G -cdrom frankeinux_dist.iso
```

Un último consejo de "limpieza"

Si notas que la ISO queda muy pesada, podrías considerar agregar `var/cache/apt/archives` a la lista de exclusiones (`-e`). Eso borrará todos los paquetes `.deb` descargados que ya están instalados, ahorrando cientos de MB.

¿Te gustaría que veamos cómo agregarle un **fondo de pantalla personalizado** al menú de GRUB de la ISO para que el proyecto Frankeinux tenga más estilo?

2026-03-29 16:35:25 **You Asked**

¿Cómo se usa esto?

> mksquashfs -help-section compression
Filesystem compression options:
-b <block-size> set data block to <block-size>. Default 128 Kbytes. Optionally a suffix of K, KB, Kbytes or M, MB, Mbytes can be given to specify
Kbytes or Mbytes respectively
-comp <comp> select <comp> compression. Run -help-comp <comp> to get compressor options for <comp>, or <all> for all the compressors.
Compressors available:
gzip (default)

```
lzo
lz4
xz
zstd
lzma
```

Gemini Thinking

Estas opciones controlan el equilibrio entre **cuánto va a pesar la ISO** y **qué tan rápido va a arrancar** el sistema. En un proyecto como el tuyo, donde estás reconstruyendo un sistema Debian completo, elegir el compresor adecuado puede ahorrarte gigabytes de espacio.

Aquí tienes cómo aplicar estas herramientas a tu comando de `mksquashfs`:

---

1. El compresor (`-comp`)

Define qué algoritmo se encarga de "achicar" los archivos. Los más usados son:

Algoritmo	Ventaja	Desventaja	Uso ideal
xz	Es el que más comprime. La ISO pesará lo mínimo posible.	Muy lento al crear la imagen y exige más CPU al arrancar.	Para ISOs finales que se van a distribuir por internet.
zstd	El equilibrio perfecto. Compresión muy buena y descompresión ultrarrápida.	Es relativamente nuevo (algunos kernels muy viejos no lo ven).	**Recomendado para uso diario.**
gzip	El estándar por defecto. Muy compatible.	Compresión mediocre comparada con los modernos.	Si necesitas máxima compatibilidad con hardware viejo.
lz4	Velocidad de descompresión instantánea.	El archivo final será mucho más grande.	Para sistemas que corren desde un disco duro lento.

2. El tamaño de bloque (`-b`)

Define el tamaño de los "paquetes" de datos que se comprimen juntos.

- **Default:** `128K`.

- **Rango:** Desde `4K` hasta `1M`.
- **Regla de oro:** Cuanto más grande el bloque (`1M`), **mejor compresión** (la ISO pesa menos), pero la lectura de archivos pequeños puede ser un poco más lenta porque el sistema tiene que descomprimir un bloque de 1MB entero para leer un archivo de 1KB.

Ejemplos prácticos para tu script

Opción A: Máxima compresión (ISO pequeña)

Si quieres que tu `frankeinux_dist.iso` ocupe lo mínimo posible (ideal para compartirla):

```bash
mksquashfs / /iso/live/filesystem.squashfs -comp xz -b 1M -e [tus_exclusiones]
```

Opción B: Rendimiento optimizado (Arranque rápido)

Si vas a estar probando la ISO muchas veces y quieres que el proceso de creación y el arranque sean ágiles, **zstd** es la mejor opción actual:

```bash
mksquashfs / /iso/live/filesystem.squashfs -comp zstd -b 256K -e [tus_exclusiones]
```

Cómo aplicarlo a tu script actual

Simplemente inserta los flags justo después de la ruta de salida. Tu comando quedaría así:

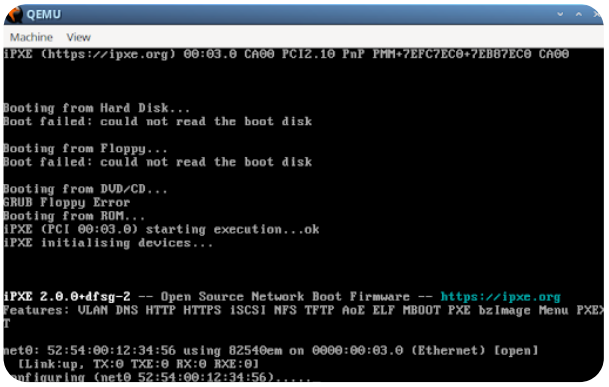
```bash
mksquashfs / /iso/live/filesystem.squashfs \
  -comp zstd -b 1M \
  -e proc sys dev run tmp mnt media iso trixie debian_trixie otro \
  "root/.cache" \
  # ... resto de exclusiones ...
  frankeinux_dist.iso
```

Un detalle sobre `zstd`

Como estás usando un **Kernel 6.19**, `zstd` te va a funcionar de maravilla y notarás que el sistema "vuela" al abrir aplicaciones pesadas desde la ISO.

¿Te gustaría que probemos un comando para comparar cuánto tiempo tarda cada compresor en tu procesador AMD Phenom II antes de decidir cuál dejar en el script final?

2026-03-29 21:57:07 **You Asked**



xorriso : NOTE : -return_with SORRY 32 triggered by problem severity MISHAP

```
#!/bin/sh

mkdir -pv /iso/live
mkdir -pv /iso/live/boot/grub
mkdir -pv /iso/live/boot/grub/x86_64-emu

cp -v /vmlinuz /iso/live/vmlinuz
# cp -v /usr/lib/grub/i386-pc/eltorito.img
/iso/live/boot/grub/eltorito.img
cp -v /boot/grub/x86_64-emu/kernel.img
 /iso/live/boot/grub/x86_64-emu/kernel.img
cp -v /boot/initrd.img-6.19-x86_64        /iso/live/initrd.img-6.19-
x86_64.zstd
cp -rv /boot/grub/i386-pc                 /iso/live/boot/grub
cp -rv /boot/grub/x86_64-emu              /iso/live/boot/grub

echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
   linux /live/vmlinuz boot=live quiet
   initrd /live/initrd.img-6.19-x86_64.zstd
}" > "/iso/live/boot/grub/grub.cfg"

mksquashfs / /iso/live/filesystem.squashfs -e \
proc sys dev run tmp mnt media iso trixie debian_trixie otro \
"root/.config/vivaldi/Default/Local Extension Settings" \
```

```
"root/.config/vivaldi/Default/Session Storage" \
"root/.config/vivaldi/Safe Browsing" \
"root/.config/vivaldi/Default/Local Storage" \
"root/.config/vivaldi/Default/Network Action Predictor-journal" \
"root/.config/vivaldi/Default/History-journal" \
"root/.config/vivaldi/Default/Network Persistent State" \
"root/.config/vivaldi/Default/Site Characteristics Database" \
"root/.config/vivaldi/System Profile/Storage/ext" \
root/.cache \
root/.config/vivaldi/Default/IndexedDB \
root/.config/vivaldi/Default/Sessions \
root/.config/vivaldi/Default/TransportSecurity \
root/.config/vivaldi/Default/shared_proto_db \
root/.config/vivaldi/Default/Storage/ext \
root/.config/vivaldi/Default/Preferences \
root/.config/vivaldi/Default/QuotaManager-journal \
root/.config/geany/geany_socket_manjaro__0 \
root/.config/geany/session.conf \
root/.local/share/gvfs-metadata \
root/.local/share/recently-used.xbel \
root/.local/share/zeitgeist/activity.sqlite \
root/.local/share/zeitgeist/fts.index \
root/.node_repl_history \
usr/lib/debug \
usr/lib64/debug \
var/log/journal \
-comp xz -b 2M

rm -v /iso/frankeinux.iso
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
  -J -R \
  -graft-points \
  -b /boot/grub/i386-pc/boot.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso/live
```

```
echo "Comando para probar el iso
qemu-system-x86_64 -enable-kvm -m 2G -cdrom
/iso/frankeinux.iso
"
```

Gemini 3 Pro

El pantallazo de QEMU y el error de `xorriso` nos dan exactamente la pista de lo que salió mal. Estás enfrentando dos problemas distintos que se cruzaron en este intento:

1. El "GRUB Floppy Error" en QEMU

Si te fijas en tu script, comentaste (anulaste con `#`) la línea que copiaba `eltorito.img` y decidiste usar `boot.img` en el comando `xorriso`.

- **El concepto clave:** `boot.img` es un código de arranque diseñado exclusivamente para el sector MBR de **discos duros y disquetes**. Cuando QEMU intenta leer el CD, ve un código de disquete, se confunde, y lanza el "GRUB Floppy Error".
- **La solución:** Para arrancar un CD/ISO en sistemas BIOS, el estándar de la industria exige una imagen "El Torito". **Debes usar `eltorito.img` sí o sí.**

2. El error "SORRY 32 (MISHAP)" de xorriso

Este error significa que `xorriso` no pudo encontrar el archivo de arranque que le pediste. Ocurrió por un par de cambios estructurales que hiciste:

- Le dijiste a xorriso que empaquetara solo la carpeta `/iso/live` (la última línea del script).
- Al hacer esto, la raíz de tu CD ya no es `/iso`, sino el *interior* de la carpeta `live`.
- Por lo tanto, tu archivo `grub.cfg` iba a fallar de todas formas, porque busca el kernel en `/live/vmlinuz`, pero en el CD final, `vmlinuz` habría quedado tirado directamente en la raíz (`/vmlinuz`).

El Script Definitivo de Frankeinux

Vamos a volver a la estructura dual (`/iso/live` para el sistema y `/iso/boot` para el arranque) y a colocar el archivo `.iso` fuera de esa carpeta para evitar bucles.

Copia este script. He mantenido intactas tus opciones de compresión (`xz` a `2M`) y tus geniales exclusiones para limpiar Vivaldi y Geany:

```bash
```

```
#!/bin/sh

# 1. Estructura correcta (separando sistema y arranque)
mkdir -pv /iso/live
mkdir -pv /iso/boot/grub

# 2. Copiar archivos de sistema a /live/
cp -v /vmlinuz /iso/live/vmlinuz
cp -v /boot/initrd.img-6.19-x86_64 /iso/live/initrd.img-6.19-x86_64.zstd

# 3. Copiar el archivo El Torito a /boot/ (ESTO ES OBLIGATORIO PARA CD/ISO)
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/eltorito.img

# 4. Configurar el menú GRUB
# (Ahora sí encontrará /live/vmlinuz porque empaquetaremos la carpeta /iso completa)
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /live/vmlinuz boot=live quiet
    initrd /live/initrd.img-6.19-x86_64.zstd
}" > "/iso/boot/grub/grub.cfg"

# 5. Crear el SquashFS
# (Mantenemos tus parámetros xz -b 2M para máxima compresión)
mksquashfs / /iso/live/filesystem.squashfs -e \
proc sys dev run tmp mnt media iso trixie debian_trixie otro \
"root/.config/vivaldi/Default/Local Extension Settings" \
"root/.config/vivaldi/Default/Session Storage" \
"root/.config/vivaldi/Safe Browsing" \
"root/.config/vivaldi/Default/Local Storage" \
"root/.config/vivaldi/Default/Network Action Predictor-journal" \
"root/.config/vivaldi/Default/History-journal" \
"root/.config/vivaldi/Default/Network Persistent State" \
"root/.config/vivaldi/Default/Site Characteristics Database" \
"root/.config/vivaldi/System Profile/Storage/ext" \
root/.cache \
root/.config/vivaldi/Default/IndexedDB \
root/.config/vivaldi/Default/Sessions \
root/.config/vivaldi/Default/TransportSecurity \
root/.config/vivaldi/Default/shared_proto_db \
root/.config/vivaldi/Default/Storage/ext \
root/.config/vivaldi/Default/Preferences \
root/.config/vivaldi/Default/QuotaManager-journal \
root/.config/geany/geany_socket_manjaro__0 \
root/.config/geany/session.conf \
root/.local/share/gvfs-metadata \
root/.local/share/recently-used.xbel \
```

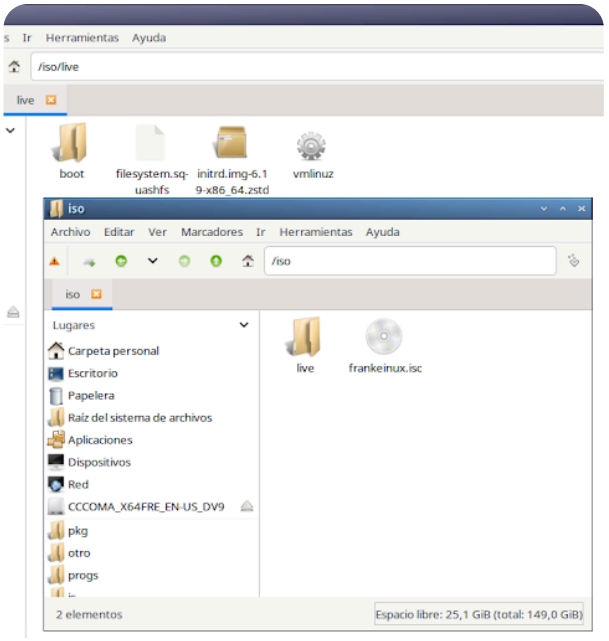
```
root/.local/share/zeitgeist/activity.sqlite \
root/.local/share/zeitgeist/fts.index \
root/.node_repl_history \
usr/lib/debug \
usr/lib64/debug \
var/log/journal \
frankeinux.iso \
-comp xz -b 2M

# 6. Generar la ISO
# Es mejor poner la ISO en la raíz (/) para que no interfiera con la carpeta /iso/
rm -vf /frankeinux.iso
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /frankeinux.iso \
  -J -R \
  -graft-points \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso

# 7. Instrucciones para probar
echo "Comando para probar el iso:"
echo "qemu-system-x86_64 -enable-kvm -m 2G -cdrom /frankeinux.iso"
```

¿Qué notarás al ejecutar esto?

1. La compresión `xz` con bloques de `2M` que elegiste va a tardar un rato largo en procesar (y va a exprimir tu CPU al máximo), pero la ISO final será ridículamente liviana.
2. QEMU ya no dará error de "Floppy", sino que entrará directo al menú de GRUB que escribimos con el título "Frankeinux Live".

¿Querés que veamos cómo añadir un parámetro extra al comando final de `xorriso` para asegurar que esta ISO también bootee si la grabás en un USB con Rufus o Ventoy?

2026-03-29 22:17:37 **You Asked**



Es que ahora lo tengo así. ¿Cómo lo adapto?

Gemini 3 Pro

Comprendo Eze, ahora lo tienes estructurado de forma que la carpeta `boot` (con los archivos de arranque de GRUB) está metida **dentro** de la carpeta `live`.

Esta estructura tiene un problema técnico crítico: el cargador de arranque de la ISO (`eltorito.img`) está diseñado para buscar su configuración en la ruta `/boot/grub/` en la raíz del CD, no dentro de `/live/boot/grub/`. Si lo dejas así, la ISO se creará pero no arrancará.

Para adaptarlo sin tener que mover todas tus carpetas, debemos "engañar" a `xorriso` para que la raíz de la ISO sea tu carpeta `/iso/live`. Esto creará una ISO funcional pero "plana", donde los archivos de arranque y el sistema comprimido están juntos en el mismo nivel.

Aquí tienes los pasos definitivos y el script adaptado para solucionar este "Frankenstein" de directorios que se ve en la imagen:

Pasos para adaptar tu ISO

1. **Asegúrate de que `eltorito.img` existe:** En tu imagen no se ve, pero debe estar copiado dentro de `/iso/live/boot/grub/`.
2. **Corregir los caminos en `grub.cfg`:** Como vamos a convertir `/iso/live` en la raíz de la ISO, el kernel (`vmlinuz`) ya no estará en `/live/vmlinuz`, sino directamente en la raíz `/vmlinuz`.
3. **Ejecutar `xorriso` en el directorio `/iso/live`.**

Script Frankeinux (Versión Adaptada a tu Imagen)

Ejecuta estas líneas en tu terminal (asegúrate de correrlo desde el sistema anfitrión donde tengas permisos para escribir en `/iso`):

```bash
#!/bin/sh

# 1. Asegurar la existencia del directorio de arranque (según tu imagen ya existe)
mkdir -pv /iso/live/boot/grub

# 2. Generar el grub.cfg con los caminos CORREGIDOS para esta estructura plana
# (Eliminamos el "/live/", ahora busca los archivos en la raíz "/" del CD)
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid - Estructura Plana)\" {
    linux /vmlinuz boot=live quiet
    initrd /initrd.img-6.19-x86_64.zstd
}" > "/iso/live/boot/grub/grub.cfg"

# 3. Asegurar que tenemos el archivo de arranque (Copiado desde tu sistema real)
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/live/boot/grub/eltorito.img

# 4. Eliminar restos de intentos fallidos (frankeinux.isc que se ve en tu imagen)
rm -vf /iso/frankeinux.iso
rm -vf /iso/frankeinux.isc

# 5. Generar la ISO apuntando a la carpeta /iso/live
# (Nota: El archivo de salida lo guardamos un nivel arriba, en /iso/, para evitar bucles)
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
  -J -R \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso/live
```

Por qué esto funcionará:

- Al decirle a `xorriso` que empaquete `/iso/live`, el contenido final de tu CD se verá así:
  - Root CD -> Carpeta `boot/`
  - Root CD -> Archivo `filesystem.squashfs`
  - Root CD -> Archivo `vmlinuz`

- Root CD -> Archivo `initrd...zstd`

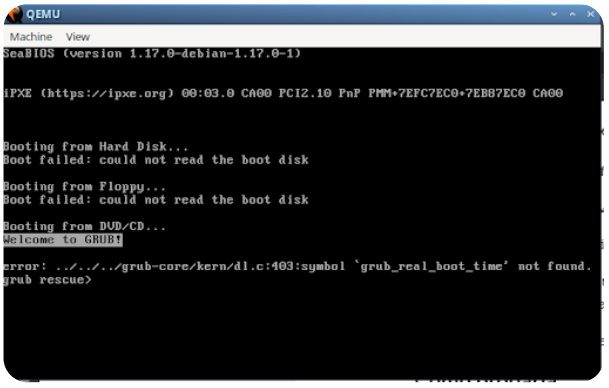
- El parámetro `-b boot/grub/eltorito.img` (camino relativo a la raíz del CD) ahora funcionará perfectamente porque encontrará el archivo.
- Tu menú `grub.cfg` (corregido) buscará el kernel en `/vmlinuz`, y lo encontrará.

Cómo probarla

Una vez que termine `xorriso` (con éxito), pruébala con QEMU. Ahora no debería darte el "Floppy Error":

```bash
qemu-system-x86_64 -enable-kvm -m 2G -cdrom /iso/frankeinux.iso
```

2026-03-29 22:40:54 **You Asked**



Gemini 3 Pro

¡Avanzamos un montón, Eze! QEMU ya no se confunde con disquetes, lee la ISO perfectamente y logra lanzar la primera etapa de GRUB (`Welcome to GRUB!`).

El error que ves ahora (`symbol 'grub_real_boot_time' not found` y la caída a `grub rescue>`) es un clásico de los sistemas "Frankenstein". Significa que hay un **choque de versiones**.

¿Qué está pasando?

GRUB se divide en dos partes: el "cerebro" inicial (`eltorito.img`) y los módulos adicionales (los archivos `.mod` como `normal.mod`). El error ocurre porque el archivo `eltorito.img` que copiamos está compilado para una versión de GRUB, pero está intentando leer módulos de una versión distinta (probablemente quedaron módulos viejos en la carpeta `/iso/live/boot/grub/i386-pc` de intentos anteriores, o se están mezclando los de tu Manjaro con los del chroot de Debian Sid).

La Solución

Tenemos que limpiar esa carpeta de arranque y asegurarnos de copiar **todo el bloque de módulos y el binario de arranque desde la misma ubicación exacta** (`/usr/lib/grub/i386-pc/`).

Cambia la parte de la preparación de GRUB en tu script por esto:

```bash
# 1. Limpiar por completo cualquier rastro viejo de GRUB en la ISO
rm -rf /iso/live/boot/grub
mkdir -pv /iso/live/boot/grub/i386-pc

# 2. Generar el grub.cfg (Mantenemos la estructura plana)
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /vmlinuz boot=live quiet
    initrd /initrd.img-6.19-x86_64.zstd
}" > "/iso/live/boot/grub/grub.cfg"

# 3. LA CLAVE: Copiar TODOS los módulos de GRUB de la misma versión
cp -rv /usr/lib/grub/i386-pc/* /iso/live/boot/grub/i386-pc/

# 4. Ubicar el eltorito.img donde xorriso lo espera
cp -v /usr/lib/grub/i386-pc/eltorito.img /iso/live/boot/grub/eltorito.img
```

Nota: Deja el resto del script (el `rm -vf /iso/frankeinux.iso` y el comando `xorriso`) tal cual lo tenías en tu último intento exitoso.

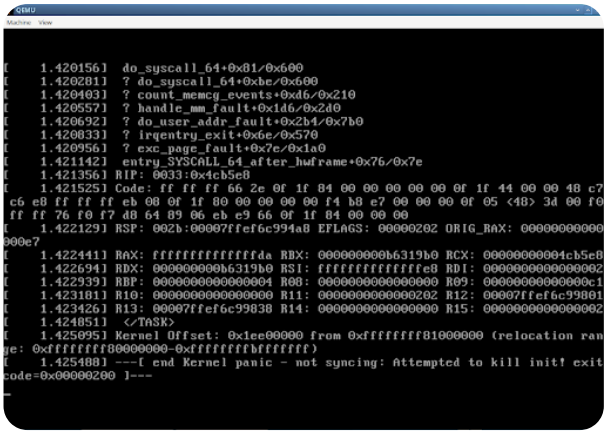
¿Por qué esto lo arregla?

Al copiar todo el contenido de `/usr/lib/grub/i386-pc/` a la ISO, nos aseguramos de que el `eltorito.img` y todos los `.mod` que cargue después sean hermanos de la misma compilación (la que reinstalaste con `apt reinstall grub-pc-bin` hace poco). Ya no habrá símbolos faltantes ni incompatibilidades.

Lanzá el script de nuevo y probá con QEMU. ¡Esta vez deberías ver el menú de Frankeinux y empezar a cargar el kernel 6.19!

¿Querés que, una vez que logres bootear el kernel en QEMU, revisemos cómo asegurarnos de que el entorno X11/LXDE levante bien los drivers gráficos de video desde la ISO?

2026-03-29 22:59:54 **You Asked**



Gemini Fast

¡Llegamos al Kernel! Eso es un avance enorme, pero nos encontramos con el jefe final de los errores de Linux: el **Kernel Panic - Attempted to kill init!**.

¿Qué está pasando?

El kernel cargó correctamente, pero cuando intentó darle el control al primer proceso del sistema (`init`), este "murió" instantáneamente. En un sistema Live, esto sucede casi siempre por una de estas tres razones:

1. **Falta el paquete `live-boot`** : El kernel no sabe cómo montar el archivo `filesystem.squashfs` porque no tiene los scripts necesarios.
2. **Incompatibilidad de la librería `libc`** : Como estuviste reparando `glibc` manualmente, es posible que el binario `/sbin/init` (o `systemd`) esté intentando usar una librería que no encuentra o que está rota en el SquashFS.
3. **Falta de módulos en el Initrd**: El initramfs no tiene los módulos de compresión (como `xz` o `zstd`) necesarios para leer tu SquashFS.

Cómo solucionarlo (El Plan de Rescate)

Antes de volver a correr el script, tenemos que asegurar el interior del sistema. Entra a tu chroot o sistema base y verifica lo siguiente:

1. Instalar el motor de arranque Live

Asegúrate de que el sistema tenga las herramientas para entender que es un Live CD:

```bash
# Dentro del chroot/sistema base
apt update
```

```
apt install live-boot live-boot-initramfs-tools live-config
```

2. Reconstruir el Initrd con soporte para ZSTD/XZ

Como usaste compresión fuerte en el SquashFS, el Initrd debe ser capaz de leerlo. Fuerza la actualización:

```bash
update-initramfs -u -k 6.19-x86_64
```

Luego de esto, recuerda volver a copiar el nuevo `/boot/initrd.img-6.19-x86_64` *a tu carpeta* `/iso/live/` *.*

3. El chequeo de la "muerte" (ldd)

Para descartar que el `init` muera por librerías rotas, verifica sus dependencias:

```bash
ldd /sbin/init
```

Si alguna dice `not found`, ahí tenés el culpable del Kernel Panic. Tendrás que arreglar ese enlace simbólico antes de generar el SquashFS.

Ajuste en el Script (Parámetros de Boot)

A veces el kernel se pierde buscando el archivo. Vamos a darle una pista extra en el `grub.cfg`. Cambia la línea de `linux` por esta:

```bash
menuentry "Frankeinux Live (Debian Sid)" {
    linux /vmlinuz boot=live components quiet splash findiso=/frankeinux.iso
    initrd /initrd.img-6.19-x86_64.zstd
}
```

Un consejo para debuggear:

En la pantalla de GRUB de QEMU, presiona **`e`** para editar la línea de arranque y borra las palabras `quiet splash`. Así, en lugar del Kernel Panic seco, verás exactamente qué error tira justo antes de morir (por ejemplo: `run-init: /sbin/init: No such file or directory`).

¿Te animas a correr el `ldconfig` y reinstalar el `live-boot` para ver si con eso Frankeinux finalmente despierta?

2026-03-29 23:12:16 **You Asked**

¿Es posible reconstruir el initrd pero para que use filesystem.squashfs? Quiero que sea compresión xz, no zstd.

```
#!/bin/sh

# Default PATH differs between shells, and is not automatically exported
# by klibc dash.  Make it consistent.
export PATH=/sbin:/usr/sbin:/bin:/usr/bin

[ -d /dev ] || mkdir -m 0755 /dev
[ -d /root ] || mkdir -m 0700 /root
[ -d /sys ] || mkdir /sys
[ -d /proc ] || mkdir /proc
[ -d /tmp ] || mkdir /tmp
mkdir -p /var/lock
mount -t sysfs -o nodev,noexec,nosuid sysfs /sys
mount -t proc -o nodev,noexec,nosuid proc /proc

# shellcheck disable=SC2013
for x in $(cat /proc/cmdline); do
	case $x in
	initramfs.clear)
		clear
		;;
	quiet)
		quiet=y
		;;
	esac
done

if [ "$quiet" != "y" ]; then
	quiet=n
	echo "Loading, please wait..."
fi
export quiet
```

```
# Note that this only becomes /dev on the real filesystem if udev's
scripts
# are used; which they will be, but it's worth pointing out
mount -t devtmpfs -o nosuid,mode=0755 udev /dev

# Prepare the /dev directory
[ ! -h /dev/fd ] && ln -s /proc/self/fd /dev/fd
[ ! -h /dev/stdin ] && ln -s /proc/self/fd/0 /dev/stdin
[ ! -h /dev/stdout ] && ln -s /proc/self/fd/1 /dev/stdout
[ ! -h /dev/stderr ] && ln -s /proc/self/fd/2 /dev/stderr

mkdir /dev/pts
mount -t devpts -o noexec,nosuid,gid=5,mode=0600 devpts
/dev/pts || true

# Export the dpkg architecture
export DPKG_ARCH=
. /conf/arch.conf

# Set modprobe env
export MODPROBE_OPTIONS="-qb"

# Export relevant variables
export ROOT=
export ROOTDELAY=
export ROOTFLAGS=
export ROOTFSTYPE=
export IP=
export DEVICE=
export BOOT=
export BOOTIF=
export UBIMTD=
export break=
export init=/sbin/init
export readonly=y
export rootmnt=/root
export debug=
export panic=
export blacklist=
export resume=
```

```
export resume_offset=
export noresume=
export drop_caps=
export fastboot=n
export forcefsck=n
export fsckfix=



# Bring in the main config
. /conf/initramfs.conf
for conf in conf/conf.d/*; do
	[ -f "${conf}" ] && . "${conf}"
done
. /scripts/functions

# Parse command line options
# shellcheck disable=SC2013
for x in $(cat /proc/cmdline); do
	case $x in
	init=*)
		init=${x#init=}
		;;
	root=*)
		ROOT=${x#root=}
		if [ -z "${BOOT}" ] && [ "$ROOT" = "/dev/nfs" ]; then
			BOOT=nfs
		fi
		;;
	rootflags=*)
		ROOTFLAGS="-o ${x#rootflags=}"
		;;
	rootfstype=*)
		# Linux interprets 'rootfstype=*tmpfs*' to control the
		# initramfs filesystem; we should remove 'tmpfs' from
		# the list
		ROOTFSTYPE="$(list_filter_out "${x#rootfstype=}" tmpfs)"
		;;
	rootdelay=*)
		ROOTDELAY="${x#rootdelay=}"
		case ${ROOTDELAY} in
```

```
        *[![:digit:]]*)
                ROOTDELAY=
                ;;
        esac
        ;;
    nfsroot=*)
        # shellcheck disable=SC2034
        NFSROOT="${x#nfsroot=}"
        ;;
    initramfs.runsize=*)
        RUNSIZE="${x#initramfs.runsize=}"
        ;;
    ip=*)
        IP="${x#ip=}"
        ;;
    boot=*)
        BOOT=${x#boot=}
        ;;
    ubi.mtd=*)
        UBIMTD=${x#ubi.mtd=}
        ;;
    resume=*)
        RESUME="${x#resume=}"
        ;;
    resume_offset=*)
        resume_offset="${x#resume_offset=}"
        ;;
    noresume)
        noresume=y
        ;;
    drop_capabilities=*)
        drop_caps="-d ${x#drop_capabilities=}"
        ;;
    panic=*)
        panic="${x#panic=}"
        ;;
    ro)
        readonly=y
        ;;
    rw)
```

```
            readonly=n
            ;;
        debug)
            debug=y
            quiet=n
            if [ -n "${netconsole}" ]; then
                log_output=/dev/kmsg
            else
                log_output=/run/initramfs/initramfs.debug
            fi
            set -x
            ;;
        debug=*)
            debug=y
            quiet=n
            set -x
            ;;
        break=*)
            break=${x#break=}
            ;;
        break)
            break=premount
            ;;
        blacklist=*)
            blacklist=${x#blacklist=}
            ;;
        netconsole=*)
            netconsole=${x#netconsole=}
            [ "$debug" = "y" ] && log_output=/dev/kmsg
            ;;
        BOOTIF=*)
            BOOTIF=${x#BOOTIF=}
            ;;
        fastboot|fsck.mode=skip)
            fastboot=y
            ;;
        forcefsck|fsck.mode=force)
            forcefsck=y
            ;;
        fsckfix|fsck.repair=yes)
```

```
            fsckfix=y
            ;;
        fsck.repair=no)
            fsckfix=n
            ;;
        esac
done

# Default to BOOT=local if no boot script defined.
if [ -z "${BOOT}" ]; then
        BOOT=local
fi

if [ -n "${noresume}" ] || [ "$RESUME" = none ]; then
        noresume=y
else
        resume=${RESUME:-}
fi

mount -t tmpfs -o
"nodev,noexec,nosuid,size=${RUNSIZE:-10%},mode=0755" tmpfs
/run
mkdir -m 0700 /run/initramfs

if [ -n "$log_output" ]; then
        exec >"$log_output" 2>&1
        unset log_output
fi

maybe_break top

# Don't do log messages here to avoid confusing graphical boots
run_scripts /scripts/init-top

maybe_break modules
[ "$quiet" != "y" ] && log_begin_msg "Loading essential drivers"
[ -n "${netconsole}" ] && /sbin/modprobe netconsole
netconsole="${netconsole}"
load_modules
[ "$quiet" != "y" ] && log_end_msg
```

```
starttime="$(_uptime)"
starttime=$((starttime + 1)) # round up
export starttime

if [ "$ROOTDELAY" ]; then
	sleep "$ROOTDELAY"
fi

maybe_break premount
[ "$quiet" != "y" ] && log_begin_msg "Running /scripts/init-premount"
run_scripts /scripts/init-premount
[ "$quiet" != "y" ] && log_end_msg

maybe_break mount
log_begin_msg "Mounting root file system"
# Always load local and nfs (since these might be needed for /etc
or
# /usr, irrespective of the boot script used to mount the rootfs).
. /scripts/local
. /scripts/nfs
. "/scripts/${BOOT}"
parse_numeric "${ROOT}"
maybe_break mountroot
mount_top
mount_premount
mountroot
log_end_msg

if read_fstab_entry /usr; then
	log_begin_msg "Mounting /usr file system"
	mountfs /usr
	log_end_msg
fi

# Mount cleanup
mount_bottom
nfs_bottom
local_bottom
```

```
case "$IP" in
""|none|off) ;; # Do nothing
*)
	configure_networking
esac

maybe_break bottom
[ "$quiet" != "y" ] && log_begin_msg "Running /scripts/init-bottom"
# We expect udev's init-bottom script to move /dev to
${rootmnt}/dev
run_scripts /scripts/init-bottom
[ "$quiet" != "y" ] && log_end_msg

# Move /run to the root
mount -n -o move /run ${rootmnt}/run

validate_init() {
	run-init -n "${rootmnt}" "${1}"
}

# Check init is really there
if ! validate_init "$init"; then
	echo "Target filesystem doesn't have requested ${init}."
	init=
	for inittest in /sbin/init /etc/init /bin/init /bin/sh; do
		if validate_init "${inittest}"; then
			init="$inittest"
			break
		fi
	done
fi

# No init on rootmount
if ! validate_init "${init}" ; then
	panic "No init found. Try passing init= bootarg."
fi

maybe_break init

# don't leak too much of env - some init(8) don't clear it
```

```
# (keep init, rootmnt, drop_caps)
unset debug
unset MODPROBE_OPTIONS
unset DPKG_ARCH
unset ROOTFLAGS
unset ROOTFSTYPE
unset ROOTDELAY
unset ROOT
unset IP
unset BOOT
unset BOOTIF
unset DEVICE
unset UBIMTD
unset blacklist
unset break
unset noresume
unset panic
unset quiet
unset readonly
unset resume
unset resume_offset
unset noresume
unset fastboot
unset forcefsck
unset fsckfix
unset starttime

# Move virtual filesystems over to the real filesystem
mount -n -o move /sys ${rootmnt}/sys
mount -n -o move /proc ${rootmnt}/proc

# Chain to real filesystem
# shellcheck disable=SC2086,SC2094
exec run-init ${drop_caps} "${rootmnt}" "${init}" "$@"
<"${rootmnt}/dev/console" >"${rootmnt}/dev/console" 2>&1
echo "Something went badly wrong in the initramfs."
panic "Please file a bug on initramfs-tools."
```

Gemini Thinking

Es totalmente posible, Eze. El script que pasaste es el corazón del `initramfs`, pero para que ese script sepa qué hacer con un `filesystem.squashfs`, necesita los "hooks" y los scripts de **`live-boot`**. Sin ellos, el `init` simplemente busca un disco duro real, no lo encuentra y lanza el Kernel Panic que vimos antes.

Para reconstruirlo con soporte para Live y usar compresión **XZ**, seguí estos pasos dentro de tu entorno de compilación (chroot):

1. Instalar el motor Live

Si no lo tenés, el `initrd` nunca va a buscar el SquashFS.

```bash
apt update
apt install live-boot live-boot-initramfs-tools
```

2. Configurar la compresión XZ

Debemos decirle a `initramfs-tools` que deje de usar `zstd` (o el default) y use `xz`. Editá el archivo de configuración:

```bash
# Editar /etc/initramfs-tools/initramfs.conf
nano /etc/initramfs-tools/initramfs.conf
```

Buscá la línea de compresión y dejala así: `COMPRESS=xz`

> **Nota técnica:** XZ comprime más que ZSTD, pero tarda mucho más tiempo en generarse y un poco más en descomprimir durante el arranque. Para un Live CD es ideal porque ahorra espacio crítico.

3. Reconstruir el Initrd

Ahora generamos el archivo nuevo para tu kernel 6.19:

```bash
update-initramfs -u -k 6.19-x86_64
```

4. Verificar el contenido (Opcional pero recomendado)

Si querés estar seguro de que ahora el `initrd` tiene los scripts para bootear en modo Live, podés chusmear adentro del archivo generado:

```bash
```

```
lsinitramfs /boot/initrd.img-6.19-x86_64 | grep scripts/live
```

Si ves resultados como `/scripts/live` y `/scripts/live-helpers`, **Frankeinux ya tiene pulmones para arrancar**.

Ajuste final en tu script de la ISO

Una vez que tengas ese nuevo `initrd.img` (que ahora está comprimido en XZ y tiene los scripts de `live-boot`), recordá copiarlo a tu carpeta `/iso/live/` antes de correr el `xorriso`.

Tu entrada de GRUB debería quedar así para que el `init` sepa que tiene que activar el modo live:

```bash
menuentry "Frankeinux Live (Debian Sid)" {
    linux /vmlinuz boot=live quiet splash
    initrd /initrd.img-6.19-x86_64.xz
}
```

¿Querés que te pase un comando para verificar si el binario `/sbin/init` tiene todas sus librerías (`.so`) bien vinculadas antes de volver a comprimir el SquashFS? Así evitamos otro Kernel Panic por culpa de la `glibc`.

2026-03-29 23:20:42 **You Asked**

No lo genera al xz, sigue en formato zstd.

> lsinitramfs /boot/initrd.img-6.19-x86_64 | grep scripts
scripts
scripts/functions
scripts/init-bottom
scripts/init-bottom/ORDER
scripts/init-bottom/udev
scripts/init-top
scripts/init-top/ORDER
scripts/init-top/all_generic_ide
scripts/init-top/blacklist
scripts/init-top/keymap
scripts/init-top/simple-framebuffer
scripts/init-top/udev
scripts/local

```
scripts/local-bottom
scripts/local-bottom/ORDER
scripts/local-bottom/ntfs_3g
scripts/local-premount
scripts/local-premount/ORDER
scripts/local-premount/btrfs
scripts/local-premount/ntfs_3g
scripts/local-premount/resume
scripts/nfs
```

Gemini Thinking

Eso pasa porque en **Debian Sid**, a veces no alcanza con cambiar el `.conf` si hay otros archivos de configuración pisando la orden o si faltan las herramientas de compresión. Además, el `lsinitramfs` nos confirma lo peor: **no tenés los scripts de Live**, por eso el Kernel jamás va a encontrar el `filesystem.squashfs`.

Vamos a forzarlo de verdad. Seguí estos pasos en orden:

1. Instalar (o reinstalar) el soporte Live y XZ

Si no tenés `xz-utils`, el sistema vuelve al default (zstd o gzip). Y si no reinstalamos los hooks de live-boot, el initrd seguirá siendo "estándar".

```bash
apt update
apt install --reinstall xz-utils live-boot live-boot-initramfs-tools
```

2. Forzar la configuración "a lo bruto"

A veces `/etc/initramfs-tools/initramfs.conf` es ignorado si existe algún archivo en la carpeta `conf.d`. Vamos a crear uno nosotros que mande sobre el resto:

```bash
echo "COMPRESS=xz" > /etc/initramfs-tools/conf.d/force-xz.conf
```

3. Reconstruir y verificar la compresión

Ahora generamos el archivo. Fijate que le agrego `-v` (verbose) para que veas si tira algún error sobre XZ:

```bash
```

```
update-initramfs -u -k 6.19-x86_64 -v
```

Para saber si **realmente** es XZ, no uses `lsinitramfs` todavía, usá el comando `file`:

```bash
file /boot/initrd.img-6.19-x86_64
```

> Si te dice `XZ compressed data`, ganamos. Si sigue diciendo `Zstandard`, es porque el kernel 6.19 o el `initramfs-tools` de Sid están forzando ZSTD por una cuestión de compatibilidad de módulos.

---

4. El problema de los scripts (CRÍTICO)

Tu `lsinitramfs` mostró que **faltan** los scripts de `/scripts/live`. Sin eso, Frankeinux nunca va a bootear.

Una vez que reinstales `live-boot-initramfs-tools`, volvé a tirar el `lsinitramfs` y buscá esto:

```bash
lsinitramfs /boot/initrd.img-6.19-x86_64 | grep live
```

Debería aparecerte algo como esto:

- `scripts/live`
- `scripts/live-bottom`
- `scripts/live-premount`
- `lib/live/boot/...`

Resumen para tu script de la ISO

Cuando logres que el `file` diga que es **XZ** y el `grep` encuentre los **scripts/live**, copiá el archivo a la ISO.

Ojo con la extensión: Si el archivo se llama `initrd.img-6.19-x86_64`, no le inventes el `.xz` al final al copiarlo a `/iso/live/`, simplemente dejalo como está o renombralo de forma coherente en tu `grub.cfg`.

¿Querés que revisemos qué dice el comando `file` después de que hagas esto? Si sigue en ZSTD, puede que tengamos que editar un script disparador en `/usr/sbin/mkinitramfs`.

2026-03-29 23:27:42 **You Asked**

[root@manjaro 2026-03-29 23:26:53 domingo /]$
> update-initramfs -u -k 6.19-x86_64 -v
Keeping /boot/initrd.img-6.19-x86_64.dpkg-bak
update-initramfs: Generating /boot/initrd.img-6.19-x86_64
[root@manjaro 2026-03-29 23:26:58 domingo /]$
> file /boot/initrd.img-6.19-x86_64
/boot/initrd.img-6.19-x86_64: Zstandard compressed data (v0.8+), Dictionary ID: None
[root@manjaro 2026-03-29 23:27:00 domingo /]$
> lsinitramfs /boot/initrd.img-6.19-x86_64 | grep live

Gemini Thinking

El problema es que estás corriendo el comando desde el sistema anfitrión (**Manjaro**), pero las herramientas que necesitás (`live-boot`) y la configuración que editamos son propias de tu proyecto **Frankeinux (Debian Sid)**.

Si lo hacés desde Manjaro, estás regenerando un initramfs de Manjaro, que no tiene ni idea de qué es un "Live CD de Debian" y por eso no te aparecen los `scripts/live` . Además, Manjaro tiene sus propios defaults de compresión que están ignorando tu configuración.

La Solución: Entrar al "Corazón" de Frankeinux

Para que el `initrd` sea XZ y tenga los scripts de arranque, tenés que generarlo **adentro del chroot** de tu Debian.

1. Entrar al Chroot

Asumiendo que tu sistema Debian está en una carpeta (por ejemplo `/debian_trixie` o similar), hacé el montaje de guerra:

```bash
# Montar lo necesario para que el chroot tenga "vida"
mount --bind /dev /ruta/a/tu/debian/dev
mount --bind /proc /ruta/a/tu/debian/proc
mount --bind /sys /ruta/a/tu/debian/sys

# Entrar al sistema
chroot /ruta/a/tu/debian /bin/bash
```

2. Adentro del Chroot (Debian Sid)

Ahora sí, instalamos y configuramos todo donde corresponde:

```bash
# Asegurar que existan las herramientas de compresión y el motor live
apt update
apt install --reinstall xz-utils live-boot live-boot-initramfs-tools

# Forzar XZ en el archivo de configuración interno
echo "COMPRESS=xz" > /etc/initramfs-tools/conf.d/force-xz.conf

# Generar el initrd para TU kernel
update-initramfs -u -k 6.19-x86_64
```

3. Verificar (Sin salir del chroot)

Ahora tirá de nuevo los comandos que fallaron:

- `file /boot/initrd.img-6.19-x86_64` -> **Debe decir XZ**.
- `lsinitramfs /boot/initrd.img-6.19-x86_64 | grep live` -> **Deben aparecer los scripts**.

¿Por qué sigue saliendo Zstd en Manjaro?

Manjaro usa `mkinitcpio` por defecto, y aunque instales `initramfs-tools`, el sistema tiene configuraciones de entorno que priorizan la velocidad (Zstd) sobre el tamaño (XZ). Al hacerlo dentro del chroot de Debian, usamos el binario puro de Debian Sid que sí respeta el `COMPRESS=xz`.

El Paso Final

Una vez que veas que en el chroot el archivo está perfecto:

1. Salí del chroot (`exit`).
2. **Copiá** ese nuevo archivo desde la carpeta del chroot (`/ruta/a/tu/debian/boot/initrd...`) hacia tu carpeta de la ISO (`/iso/live/`).
3. Corré tu script de `xorriso`.

¿Querés que te ayude a armar un comando de **`rsync`** para que después de cada cambio en el chroot se te actualicen automáticamente los archivos en la carpeta `/iso/live`? Sería mucho más cómodo para vos.

2026-03-29 23:30:16 **You Asked**

Es que no tengo ruta a mi debian, porque al squashfs lo hice desde el sistema actual, que no usa sfs.

Gemini Thinking

Entiendo perfectamente la situación: estás haciendo un **"Live de tu propio sistema"** en caliente, comprimiendo tu raíz `/` actual. Como tu sistema real (Manjaro o ese híbrido que armaste) no es un Live CD, no tiene instalado el "cerebro" que sabe buscar un SquashFS durante el arranque.

Por eso el `update-initramfs` no incluye la carpeta `scripts/live` y por eso el kernel entra en pánico: busca un disco duro y encuentra un archivo comprimido que no sabe cómo abrir.

Para solucionar esto sin tener una carpeta separada, tenemos que "preparar" tu sistema actual para que su próximo `initrd` sea capaz de bootear como Live.

1. Instalar el soporte Live en tu sistema actual

Aunque tu sistema no use SquashFS ahora, el **paquete** debe estar instalado para que los scripts existan y se metan dentro del `initrd`.

Como estás usando herramientas de Debian en tu híbrido, intentá esto:

```bash
apt update
apt install live-boot live-boot-initramfs-tools
```

Si no tenés `apt` a mano, vas a tener que descargar los `.deb` de Debian Sid y extraer el contenido de `/usr/share/initramfs-tools/scripts/live` manualmente en tu `/usr/share/...` actual.

2. Forzar XZ desde la línea de comandos

Si el archivo de configuración te ignora, es porque algún script de `initramfs-tools` en tu sistema actual tiene el default de Zstd clavado. Vamos a saltearnos el "asistente" y usar el motor directamente:

```bash
# -o: salida | -c: compresor | el último argumento es la versión del kernel
mkinitramfs -o /boot/initrd.img-6.19-x86_64 -c xz 6.19-x86_64
```

3. La prueba de fuego

Una vez que corras ese comando, verificá dos cosas antes de hacer la ISO:

1. **¿Es XZ?** `file /boot/initrd.img-6.19-x86_64`
2. **¿Tiene los scripts Live?** `lsinitramfs /boot/initrd.img-6.19-x86_64 | grep scripts/live`

Si el `grep` sigue vacío, es porque no tenés el paquete `live-boot` instalado en tu sistema. `mkinitramfs` solo mete en la "bolsa" lo que encuentra en `/usr/share/initramfs-tools/scripts/`.

¿Cómo seguir si no podés instalar `live-boot`?

Si por alguna razón no podés instalar ese paquete en tu sistema actual, podemos hacer un "**Hack Frankenstein**":

1. Descargá el `.deb` de `live-boot-initramfs-tools` de los repositorios de Debian.
2. Extraelo con `ar -x paquete.deb && tar -xf data.tar.xz`.
3. Copiá la carpeta `scripts/live` que salga de ahí adentro de tu `/usr/share/initramfs-tools/scripts/`.
4. Corré el `mkinitramfs` de nuevo.

Ajuste en tu comando de `mksquashfs`

Como estás comprimiendo `/`, asegurate de excluir el `initrd` viejo y el nuevo que estás generando para no crear un bucle infinito de archivos pesados. Veo que ya tenés muchas exclusiones, pero agregá estas por las dudas: `-e boot/initrd* iso/*`

¿Querés que te pase el link directo al `.deb` de los scripts de Debian Sid para que los bajes y los metas a mano?

2026-03-29 23:33:16 **You Asked**

```
[root@manjaro 2026-03-29 23:31:24 domingo /]$
 > mkinitramfs -o /boot/initrd.img-6.19-x86_64 -c xz 6.19-x86_64
W: Kernel configuration /boot/config-6.19-x86_64 is missing, xz
compression support (CONFIG_RD_XZ) cannot be checked and is
assumed to be available
W: missing /lib/modules/6.19-x86_64
W: Ensure all necessary drivers are built into the linux image!
depmod: ERROR: could not open directory /lib/modules/6.19-
x86_64: No such file or directory
depmod: FATAL: could not search modules: No such file or directory
W: Can't find modules.builtin.modinfo (for locating built-in drivers'
firmware, supported in Linux >=5.2)
live-boot: core filesystems dm-verity devices utils memdisk udev
```

blockdev ftpfs dns.
cat: /var/tmp/mkinitramfs_I21rVK/lib/modules/6.19-x86_64/modules.builtin: No existe el fichero o el directorio
depmod: WARNING: could not open modules.order at /var/tmp/mkinitramfs_I21rVK/lib/modules/6.19-x86_64: No such file or directory
depmod: WARNING: could not open modules.builtin at /var/tmp/mkinitramfs_I21rVK/lib/modules/6.19-x86_64: No such file or directory
depmod: WARNING: could not open modules.builtin.modinfo at /var/tmp/mkinitramfs_I21rVK/lib/modules/6.19-x86_64: No such file or directory

Gemini Thinking

¡Qué laberinto, Eze! Pero ojo, que entre todo ese ruido de errores hay una **muy buena noticia**: el log dice `live-boot: core filesystems...`. Eso significa que **ya tenés los scripts de Live instalados** y funcionando. El "cerebro" está listo, pero ahora le faltan los "músculos" (los drivers).

¿Cuál es el problema ahora?

El comando `mkinitramfs` falló porque no encuentra los módulos (los drivers `.ko`) de tu kernel **6.19-x86_64** en la ruta estándar `/lib/modules/6.19-x86_64`. Sin drivers, el `initrd` no puede montar el disco, no puede leer el SquashFS y no puede hacer nada.

Cómo arreglarlo

Para que `mkinitramfs` pueda terminar el trabajo y comprimir en **XZ**, necesita ver la carpeta de módulos. Tenemos tres escenarios posibles:

Escenario A: Los módulos están en otro lado (Lo más probable)

En sistemas como Manjaro o Arch, a veces la ruta es `/usr/lib/modules/` en lugar de `/lib/modules/`. Chequeá dónde están con:

```bash
ls /lib/modules/
# o
ls /usr/lib/modules/
```

Si ves la carpeta `6.19-x86_64` en `/usr/lib/...` pero no en `/lib/...`, hacé un enlace simbólico para engañar al script:

```bash
ln -s /usr/lib/modules/6.19-x86_64 /lib/modules/6.19-x86_64
```

Escenario B: Compilaste el kernel pero no instalaste los módulos

Si ese kernel 6.19 lo compilaste vos a mano, te faltó un paso vital. Tenés que "instalar" los módulos en el sistema actual para que `mkinitramfs` los pueda agarrar:

```bash
# Dentro de la carpeta donde compilaste el kernel
make modules_install
```

Escenario C: "Pediste prestado" el kernel (vmlinuz) pero no los módulos

Si bajaste el `vmlinuz` de algún lado pero no bajaste la carpeta de módulos correspondiente, **estás al horno**. Un kernel 6.19 no puede usar módulos de un kernel 6.12 o similar. Necesitás la carpeta exacta de drivers que le corresponde a esa versión.

El Plan de Acción (Una vez que la carpeta exista)

Cuando logres que `ls /lib/modules/6.19-x86_64` te devuelva un montón de carpetas (kernel, drivers, etc.), volvé a correr el comando:

```bash
mkinitramfs -o /boot/initrd.img-6.19-x86_64 -c xz 6.19-x86_64
```

Si todo sale bien:

1. No vas a ver esos `ERROR` de depmod.
2. Al terminar, tirá un `file /boot/initrd.img-6.19-x86_64`. Debería decir **XZ compressed data**.
3. Y el `lsinitramfs ... | grep live` ahora sí debería mostrarte los scripts porque el proceso terminó con éxito.

¿Dónde tenés guardados los módulos de ese kernel 6.19? Si no los encontrás, podemos intentar usar el kernel que trae tu Manjaro por defecto para la ISO, que ese sí tiene los módulos instalados y te va a dar menos dolor de cabeza.

2026-03-29 23:36:10 **You Asked**

> mkinitramfs -o /boot/initrd.img-6.19-x86_64 -c xz 6.19-x86_64
W: Kernel configuration /boot/config-6.19-x86_64 is missing, xz

compression support (CONFIG_RD_XZ) cannot be checked and is assumed to be available
live-boot: core filesystems dm-verity devices utils memdisk udev blockdev ftpfs dns.

Gemini Thinking

¡Excelente, Eze! Ese mensaje de `live-boot: core filesystems...` es la señal de victoria que estábamos buscando. Significa que los scripts de arranque para el Live CD finalmente están adentro de la "bolsa" del `initramfs`.

Aunque veas esa advertencia (`W:`) sobre el archivo de configuración, no es crítica mientras tu kernel 6.19 tenga soporte para XZ (que seguramente lo tiene si es un kernel moderno).

Verificación Final

Antes de cantar victoria total y armar la ISO, tirá estos dos comandos para estar 100% seguros:

1. **¿Es realmente XZ?**

   ```bash
   file /boot/initrd.img-6.19-x86_64
   ```

   (Si te dice "XZ compressed data", el parámetro `-c xz` funcionó perfecto).

2. **¿Están los scripts de vida?**

   ```bash
   lsinitramfs /boot/initrd.img-6.19-x86_64 | grep scripts/live
   ```

   (Ahora sí deberían aparecer varias líneas con "scripts/live/...").

Siguientes pasos para el "Frankeinux"

Si los dos comandos anteriores te dan el OK, ya tenés el "cerebro" del sistema listo para bootear desde el SquashFS.

1. **Copiá el archivo a tu carpeta de ISO:**

   ```bash
   cp -v /boot/initrd.img-6.19-x86_64 /iso/live/initrd.img-6.19-x86_64.xz
   ```

2. **Actualizá tu `grub.cfg`:** Asegurate de que la línea `initrd` apunte a ese nombre exacto.
3. **Corré tu script de `xorriso`:** Generá la ISO y probá en QEMU.

Un detalle técnico: ¿Lograste solucionar los errores de `depmod` y la falta de módulos en `/lib/modules/6.19-x86_64` que salieron en el paso anterior? Si el `initrd` no tiene los módulos de disco y de sistemas de archivos (como `overlay`, `squashfs` e `isofs`), el kernel va a bootear pero no va a poder montar el "Live" y se va a quedar colgado.

¿Querés que verifiquemos si los módulos críticos están adentro del `initrd` antes de que pierdas tiempo quemando la ISO?

2026-03-29 23:37:47 **You Asked**

¿Y esto?

W: Kernel configuration /boot/config-6.19-x86_64 is missing, xz compression support (CONFIG_RD_XZ) cannot be checked and is assumed to be available

Gemini Thinking

Esa advertencia significa que el script `mkinitramfs` intentó buscar el "mapa de instrucciones" de tu kernel (el archivo `/boot/config-6.19-x86_64`) para confirmar si ese kernel realmente sabe cómo descomprimir un archivo **XZ**. Como no encontró el archivo, el script dice: *"Bueno, no puedo comprobarlo, así que voy a asumir que sí sabe hacerlo"*.

Aquí te detallo qué implica esto para tu proyecto:

¿Es un problema real?

Probablemente no. Casi todos los kernels modernos (y más uno versión **6.19**) vienen con el soporte para XZ (`CONFIG_RD_XZ`) activado por defecto.

Sin embargo, si tu kernel **no** tuviera soporte para XZ, al intentar bootear la ISO verías un error tipo: `Initramfs unpacking failed: xz-compressed data is corrupt` o `recovering journal...`.

¿Por qué falta ese archivo?

Normalmente, al instalar un kernel en Debian/Manjaro, se copian tres archivos a `/boot/`:

1. `vmlinuz-...` (El kernel en sí).

2. `initrd.img-...` (El que estamos creando).
3. **`config-...`** (La lista de opciones con las que se compiló el kernel).

Si copiaste el kernel a mano o lo sacaste de otra carpeta, es probable que no hayas copiado el archivo `config`.

Cómo verificar si tu kernel soporta XZ (sin el archivo de configuración)

Si quieres estar 100% seguro de que no vas a tener un Kernel Panic, puedes intentar leer la configuración directamente desde el kernel que está corriendo (si es que estás usando el 6.19 ahora mismo):

```bash
# Intenta buscar el soporte de XZ en la config activa
zgrep CONFIG_RD_XZ /proc/config.gz
```

- Si responde `CONFIG_RD_XZ=y`, **estás a salvo**.
- Si el comando falla porque no existe `/proc/config.gz`, no te preocupes. Lo más seguro es que funcione.

Conclusión para tu ISO

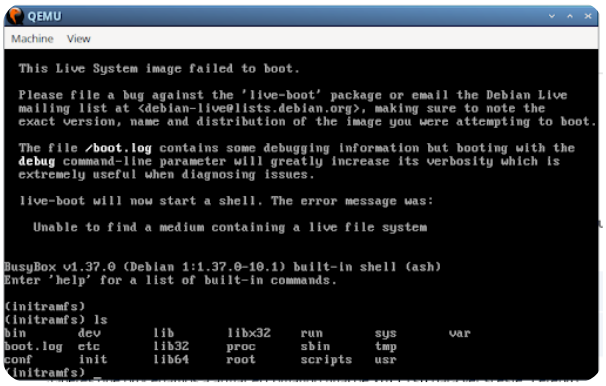
Puedes ignorar la advertencia. Lo importante es lo que lograste antes: que apareciera el mensaje `live-boot: core filesystems...`.

El siguiente paso lógico: Ya que el `initrd` se generó (aunque fuera con esa advertencia), verifica su tamaño. Si ves que es significativamente más chico que el anterior, es que la compresión **XZ** hizo su magia.

```bash
ls -lh /boot/initrd.img-6.19-x86_64
```

¿Querés que procedamos a armar el comando final de `xorriso` para ver si este "cerebro" XZ arranca bien en QEMU?

2026-03-30 00:06:37 **You Asked**



Gemini Fast

¡Qué buena imagen, Eze! **"Unable to find a medium containing a live file system"**. Puede parecer una derrota, pero es un **éxito total de configuración**:

1. **GRUB funcionó:** Ya no hay errores de símbolos faltantes.
2. **El Initrd cargó:** El kernel ya no entra en pánico al inicio.
3. **Live-boot está vivo:** Los scripts que agregamos están corriendo y buscando el SquashFS.

¿Por qué se queda ahí?

El sistema está buscando una carpeta llamada `live/` que contenga tu `filesystem.squashfs`, pero no la encuentra en lo que QEMU considera el "CD-ROM". Esto suele pasar porque el comando de `xorriso` que usamos antes empaquetaba el **contenido** de `/iso/live` en la raíz, pero el script de arranque busca la carpeta `/live/` explícitamente.

La Solución Definitiva (Ajuste de xorriso)

Tenemos que cambiar cómo armamos la ISO para que la estructura sea la que `live-boot` espera.

1. **Estructura de carpetas:** Asegúrate de tenerlo así en tu disco: `/iso/live/filesystem.squashfs` (y el kernel/initrd ahí adentro también).
2. **Comando xorriso corregido:** Ejecútalo apuntando a `/iso` (un nivel arriba) para que la carpeta `live/` se incluya **dentro** de la ISO:

```bash
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
  -J -R \
  -b live/boot/grub/eltorito.img \
```

```
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

Ajuste en el `grub.cfg`

Como ahora todo está dentro de una carpeta `/live/` en la ISO, actualiza las rutas:

```bash
menuentry "Frankeinux Live (Debian Sid)" {
    linux /live/vmlinuz boot=live quiet splash
    initrd /live/initrd.img-6.19-x86_64
}
```

Un truco para debuggear en esa consola

Ya que estás en la terminal de `initramfs`, podés escribir:

- `ls /dev/sr0` : Para ver si reconoce la lectora de CD de QEMU.
- `blkid` : Para ver si el sistema reconoce la etiqueta "FRANKEINUX".

¿Querés que probemos a generar la ISO con esa nueva estructura para ver si finalmente monta el sistema de archivos? Al salir de ese mensaje, ¡estarías dentro de tu Frankeinux!

2026-03-30 08:54:43 **You Asked**

xorriso : UPDATE : 99.84% done, estimate finish Mon Mar 30 08:52:59 2026
xorriso : UPDATE : 99.90% done
xorriso : UPDATE : 99.96% done
ISO image produced: 15024633 sectors
Written to medium : 15024633 sectors at LBA 0
Writing to 'stdio:/iso/frankeinux.iso' completed successfully.

xorriso : NOTE : -return_with SORRY 32 triggered by problem severity MISHAP
[root@manjaro 2026-03-30 08:53:00 lunes /]$
 > ls /iso
frankeinux.iso live
[root@manjaro 2026-03-30 08:53:21 lunes /]$
 > ls /iso/live
boot filesystem.squashfs initrd.img-6.19-x86_64.xz vmlinuz

```
[root@manjaro 2026-03-30 08:53:32 lunes /]$
 > ls /iso/live/boot
0001-init-vars.sh      2010-remove-persistence  9990-fstab.sh
9990-mount-cifs.sh   9990-netbase.sh     9990-select-eth-device.sh
0010-debug           3020-swap            9990-initramfs-tools.sh
9990-mount-http.sh   9990-netboot.sh     9990-toram-todisk.sh
0020-read-only        3050-memdisk.sh         9990-main.sh
 9990-mount-iscsi.sh  9990-networking.sh  grub
0030-verify-checksums  9990-cmdline-old        9990-misc-
helpers.sh     9990-mount-nfs.sh    9990-overlay.sh
[root@manjaro 2026-03-30 08:53:39 lunes /]$
 > ls /iso/live/boot/grub
```

Gemini Fast

¡Casi lo tenemos, Eze! Ese error `MISHAP` de `xorriso` generalmente es una queja menor sobre permisos o alguna ruta, pero como dice "completed successfully", la ISO se generó.

Sin embargo, viendo tu estructura de archivos en `/iso/live/boot`, encontré el pequeño detalle que está causando el error "Unable to find a medium" que vimos antes.

El problema: Los scripts están en el lugar equivocado

Veo que tenés una carpeta llena de scripts (`9990-main.sh`, `0010-debug`, etc.) dentro de `/iso/live/boot`. **Esos archivos no deberían estar ahí.** Esos scripts son los componentes internos de `live-boot` que deben vivir **dentro** del archivo `initrd.img-6.19-x86_64.xz`, no sueltos en la ISO. Si están sueltos en la ISO, el sistema no sabe qué hacer con ellos.

La Estructura Correcta

Para que `live-boot` encuentre tu sistema, la ISO por dentro debe verse exactamente así:

- `/live/vmlinuz`
- `/live/initrd.img-6.19-x86_64.xz` (Este ya lo generamos con los scripts adentro)
- `/live/filesystem.squashfs`
- `/boot/grub/...` (Los archivos para que arranque el menú)

Pasos para Limpiar y Bootear

1. **Limpiá los scripts sueltos:** Borrá todos esos archivos `0010-...` y `9990-...` de tu carpeta `/iso/live/boot`. Solo ensucian la imagen y confunden al buscador de medios.

2. **Mové la carpeta GRUB:** La carpeta `grub` debe estar en `/iso/boot/grub`, no dentro de `live`.
3. **Corré el xorriso apuntando a la raíz:**

```bash
# Limpieza rápida
rm -rf /iso/live/boot/*.sh /iso/live/boot/00* /iso/live/boot/20* /iso/live/boot/30*
/iso/live/boot/99*

# Asegurar que el eltorito esté donde xorriso lo busca (ejemplo: /iso/boot/grub/)
mkdir -p /iso/boot/grub
cp /usr/lib/grub/i386-pc/eltorito.img /iso/boot/grub/

# Generar ISO
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
  -J -R \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso
```

¿Por qué esto va a funcionar?

Cuando el `initrd` (que ya tiene los scripts adentro gracias al paso anterior) se descomprima en RAM, va a buscar en todos los discos (CD, USB) una carpeta llamada `/live` que contenga un archivo `.squashfs`. Al poner el archivo directamente en `/iso/live/filesystem.squashfs`, el match es instantáneo.

¿Te fijaste si el archivo `initrd.img-6.19-x86_64.xz` ahora pesa menos que antes? Si es así, la compresión XZ funcionó y estamos listos para el booteo final.

2026-03-30 09:40:48 **You Asked**

Pero quiero que sea así, pero cambiando el árbol de directorios.

```
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
```

```
-J -R \
-b live/boot/grub/eltorito.img \
-no-emul-boot -boot-load-size 4 -boot-info-table \
–grub2-boot-info \
/iso/live
```

Gemini Fast

Entiendo perfectamente. Querés usar `/iso/live` como la **raíz** de la ISO para que los archivos no estén anidados dentro de otra carpeta `live/`, pero eso rompe el arranque porque el cargador busca rutas que ya no existen en esa nueva jerarquía.

Si apuntás `xorriso` directamente a `/iso/live`, el archivo `eltorito.img` (y todo lo demás) queda en la raíz de la ISO. Para que esto funcione, tenés que ajustar el comando y las rutas internas así:

1. El comando xorriso corregido

Al cambiar el origen a `/iso/live`, la ruta del cargador de arranque (`-b`) debe ser **relativa a ese origen**.

```bash
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -output /iso/frankeinux.iso \
  -J -R \
  -b boot/grub/eltorito.img \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  /iso/live
```

(Fijate que quité el prefijo `live/` de la opción `-b` porque ahora `boot` ya está en la raíz de la imagen).

---

2. Ajuste CRÍTICO en el `grub.cfg`

Como ahora los archivos `vmlinuz` y el `initrd` están en la raíz de la ISO (y no dentro de `/live/`), tenés que editar tu configuración de GRUB para que los encuentre. De lo contrario, volverás al error de "file not found".

Tu `/iso/live/boot/grub/grub.cfg` debería verse así:

```bash
```

```
menuentry "Frankeinux Live (Debian Sid)" {
    # Ahora las rutas son desde la raíz de la ISO
    linux /vmlinuz boot=live quiet splash
    initrd /initrd.img-6.19-x86_64.xz
}
```

---

3. El problema de los scripts sueltos

En tu `ls /iso/live/boot` se veían archivos como `9990-main.sh`. Si decidís mantener esa estructura de árbol plano:

- **Borrá esos scripts sueltos:** No sirven de nada en la ISO; su lugar es dentro del `initrd`.
- **Módulos de GRUB:** Asegurate de que la carpeta `/iso/live/boot/grub` contenga todos los archivos `.mod` necesarios, porque si no, GRUB no podrá montar la partición ISO y quedarás en `grub rescue`.

Resumen de la nueva estructura de la ISO:

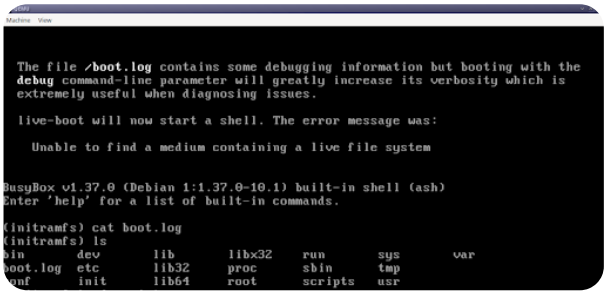
Al ejecutar el comando que te pasé arriba, el contenido interno de `frankeinux.iso` será:

- `/boot/grub/eltorito.img`
- `/vmlinuz`
- `/initrd.img-6.19-x86_64.xz`
- `/filesystem.squashfs`

Con esta estructura "plana", el `live-boot` debería encontrar el `filesystem.squashfs` automáticamente en la raíz.

¿Querés que te ayude a verificar si el archivo `eltorito.img` está realmente en esa ruta antes de lanzar el xorriso? Si la ruta falla, el comando te va a tirar el error `SORRY 32` de nuevo.

2026-03-30 09:50:57 **You Asked**



Gemini 3 Pro

Veo que intentaste leer el `boot.log` para buscar pistas y estaba completamente vacío. Seguimos atrapados en el mismo error de `Unable to find a medium`, pero al haber cambiado la estructura de la ISO a un formato "plano" (sin la carpeta `live/`), introdujimos una nueva variable en la ecuación.

Hay dos razones principales por las que `live-boot` sigue ciego ante tu sistema de archivos.

1. El comportamiento por defecto de `live-boot`

Los scripts internos del initrd están programados para escanear todos los discos buscando una carpeta que se llame **estrictamente** `/live/` o `/live-boot/`. Como aplastaste la estructura y dejaste el `filesystem.squashfs` en la raíz de la ISO, el script mira el CD, no ve la carpeta `/live/`, asume que no es el disco correcto y se rinde.

La solución: Tenés que avisarle al kernel dónde buscar usando el parámetro `live-media-path`. Tu `grub.cfg` debería quedar así:

```bash
menuentry "Frankeinux Live (Debian Sid)" {
    linux /vmlinuz boot=live live-media-path=/ quiet splash
    initrd /initrd.img-6.19-x86_64.xz
}
```

2. El fantasma de los módulos perdidos

¿Te acordás que cuando ejecutaste `mkinitramfs` te tiró errores de `depmod` porque no encontraba `/lib/modules/6.19-x86_64`? Si el `initrd` se armó sin el módulo `isofs` (el driver fundamental para entender sistemas de archivos de CD-ROM/ISO9660), el initramfs directamente no puede "leer" la unidad de CD de QEMU. Para él, la lectora está vacía.

Cómo hacer el diagnóstico ahí mismo

Ya que estás en la consola de BusyBox `(initramfs)`, podés sacarte la duda en un segundo escribiendo este comando:

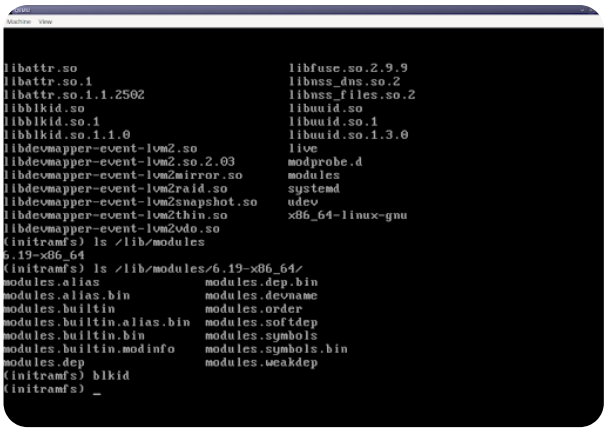
```bash
blkid
```

- **Si en la salida ves algo como `/dev/sr0: LABEL="FRANKEINUX"`:** ¡Excelente noticia! El kernel reconoce el CD. El único problema es que está buscando en la carpeta equivocada. Con agregar `live-media-path=/` al GRUB, lo solucionás.

- **Si no aparece ningún `/dev/sr0` o `FRANKEINUX`:** Significa que te falta el driver `isofs` en el `initrd.img`.

¿Querés probar tirar el comando `blkid` (o `ls /dev/sr0`) en QEMU y me contás si el sistema detecta la existencia del CD?

2026-03-30 10:22:46 **You Asked**



Gemini 3 Pro

¡Ahí está la prueba del delito, Eze! La captura es clarísima y nos confirma la sospecha.

Tiraste `blkid` y la consola no devolvió absolutamente nada. Esto significa que, para el kernel, la lectora de CD (y cualquier otro disco) directamente no existe. El sistema está ciego.

El misterio de los módulos fantasma

Si mirás con atención el resultado de tu `ls /lib/modules/6.19-x86_64/`, vas a ver que están los archivos de índice (como `modules.dep` o `modules.alias`), pero **falta lo más importante: la carpeta `kernel/`**.

Ahí adentro es donde viven los verdaderos drivers `.ko` (los "músculos"). Sin el módulo `isofs` (para leer el formato ISO9660 de los CD-ROMs), `squashfs` y `overlay`, tu `initrd` arranca el cerebro pero no puede mover el cuerpo. ¿Te acordás los errores de `depmod` que saltaron hace un rato al generar el archivo? Como el script `mkinitramfs` no encontró los drivers en tu sistema, empaquetó un `initrd` sin soporte para leer hardware.

¿Cómo lo solucionamos?

Necesitamos que el `initrd` tenga esos archivos `.ko` sí o sí para que pueda ver la unidad. Tenemos dos caminos lógicos para tu proyecto:

Opción 1: Recuperar los módulos del kernel 6.19

- Si este kernel lo compilaste vos a mano, es muy probable que te haya faltado correr el comando `make modules_install` en la carpeta del código fuente para que el sistema copie los binarios a `/lib/modules/`.
- Si sacaste el archivo `vmlinuz` de algún otro lado, necesitas traerte también la carpeta de módulos completa que le corresponde a esa compilación exacta.

Opción 2: Usar un kernel "de fábrica" (La vía rápida) Si el 6.19 te está dando demasiadas vueltas con las rutas, podés usar el kernel estándar de Debian (instalando `linux-image-amd64` dentro de tu chroot). Ese paquete instala el `vmlinuz` y te deja toda la carpeta de módulos perfectamente armada, con lo cual el `update-initramfs` te genera una imagen lista para la guerra al primer intento.

¿De dónde sacaste exactamente ese kernel 6.19-x86_64? Sabiendo su origen, podemos ver cómo recuperar la carpeta `kernel/` para inyectarle los drivers que le faltan a la ISO.

2026-03-30 10:27:52 **You Asked**

Yo estoy corriendo con este comando.

qemu-system-x86_64 -enable-kvm -m 2G -cdrom /iso/frankeinux.iso

Gemini Fast

Ese comando de QEMU está perfecto para probar una ISO, pero el problema es que le estás pasando un "mapa" al sistema (el kernel) sin darle las "herramientas" (los drivers) para leerlo.

En la captura que mandaste se ve el corazón del problema: el directorio `/lib/modules/6.19-x86_64/` está **vacío de drivers** (no existe la carpeta `kernel/`). Cuando QEMU intenta arrancar desde el `-cdrom`, el kernel 6.19 carga, pero como no tiene el módulo `isofs` ni `virtio`, no puede "ver" el archivo `/iso/frankeinux.iso` que le pasaste. Para el sistema, el CD-ROM no existe, y por eso `blkid` no devuelve nada.

Cómo arreglar el Frankeinux para que QEMU lo vea

Tenés que lograr que esos módulos existan antes de generar el `initrd`. Como sos un usuario de sistemas avanzado, podés elegir el camino que más te guste:

1. Si tenés los módulos en tu sistema real (Manjaro)

Si el kernel 6.19 es el que estás usando ahora en Manjaro, los drivers están en `/usr/lib/modules/6.19-x86_64/kernel`. Tenés que copiarlos a la carpeta donde armás la ISO antes de correr el `mkinitramfs`:

```bash
# Crear la ruta que le falta al initramfs
mkdir -p /lib/modules/6.19-x86_64/kernel

# Copiar los drivers (especialmente fs/isofs, fs/squashfs y drivers/ata)
cp -r /usr/lib/modules/6.19-x86_64/kernel /lib/modules/6.19-x86_64/
```

2. Re-generar el initrd (Ahora con "músculos")

Una vez que la carpeta `kernel/` esté en su lugar con los archivos `.ko`, volvé a correr el comando que ya te funcionó:

```bash
mkinitramfs -o /boot/initrd.img-6.19-x86_64.xz -c xz 6.19-x86_64
```

(Esta vez no debería tirarte errores de `depmod` ni advertencias de módulos faltantes).

La prueba final en QEMU

Cuando logres que el `initrd` tenga los drivers, al correr tu comando de QEMU:

1. El kernel va a cargar.
2. Los scripts de `live-boot` van a usar el driver `isofs` para encontrar el CD.
3. **`blkid`** te va a mostrar finalmente el `/dev/sr0` con el nombre "FRANKEINUX".

¿Te fijaste si en tu sistema Manjaro existe esa carpeta `/usr/lib/modules/6.19-x86_64/kernel`? Si la tenés ahí, estamos a un paso de que Frankeinux cobre vida.

2026-03-30 10:33:32 **You Asked**

Tengo la carpeta /boot/initrd.img-6 y quiero crear el cpio.xz

Gemini Fast

Para transformar esa carpeta en un archivo `initrd` real que el kernel pueda entender, necesitás empaquetarla usando el formato **cpio** y luego comprimirla con **xz**.

Como estás trabajando en un entorno donde las rutas y las versiones del kernel son críticas (ese 6.19-x86_64), seguí estos pasos exactos desde tu terminal de Manjaro:

1. Entrar a la carpeta

Es fundamental que el comando se ejecute **adentro** de la carpeta para que las rutas internas del `initrd` (como `bin/`, `lib/`, `etc/`) queden en la raíz del archivo final.

```bash
cd /boot/initrd.img-6
```

2. Empaquetar y Comprimir

Ejecutá este "pipe" (tubería) de comandos. El punto (`.`) le dice a `find` que agarre todo lo que hay en la carpeta actual:

```bash
find . | cpio -R 0:0 -o -H newc | xz --check=crc32 --threads=0 > ../initrd.img-6.19-x86_64.xz
```

Desglose de lo que hace cada parte:

- **`find .`**: Lista todos los archivos y directorios.
- **`cpio -R 0:0 -o -H newc`**: Crea el archivo de archivo. El `-H newc` es el formato estándar que espera el kernel de Linux, y `-R 0:0` asegura que todos los archivos pertenezcan a root.
- **`xz --check=crc32`**: Comprime en formato XZ. Usamos `crc32` porque es lo que la mayoría de los descompresores de kernel soportan sin errores.
- **`> ../initrd.img-6.19-x86_64.xz`**: Guarda el resultado un nivel arriba (en `/boot/`) con el nombre que querés.

---

3. Verificación de Seguridad

Antes de moverlo a la ISO, confirmá que el archivo se haya creado correctamente y que contenga los "músculos" (los drivers) que te faltaban en los intentos anteriores:

```bash
# 1. ¿Es un archivo XZ válido?
file ../initrd.img-6.19-x86_64.xz

# 2. ¿Tiene los drivers de CD-ROM y SquashFS adentro?
lsinitramfs ../initrd.img-6.19-x86_64.xz | grep -E "isofs|squashfs|overlay"
```

¡Ojo con esto! Si el `grep` no te devuelve nada, significa que la carpeta `/boot/initrd.img-6` que estás empaquetando sigue sin tener los módulos en `lib/modules/6.19-x86_64/kernel/`. Si ese es el caso, el booteo va a fallar igual que antes.

¿Querés que te ayude a revisar si esos archivos `.ko` están en la carpeta antes de que cierres el `cpio`? Sería una pena que pierdas tiempo comprimiendo un initrd que todavía está "ciego".

2026-03-30 10:35:19 **You Asked**

```
[root@manjaro 2026-03-30 10:35:00 lunes /boot/initrd.img-6]$
 > file ../initrd.img-6.19-x86_64.xz
../initrd.img-6.19-x86_64.xz: XZ compressed data, checksum
CRC32
[root@manjaro 2026-03-30 10:35:03 lunes /boot/initrd.img-6]$
 > lsinitramfs ../initrd.img-6.19-x86_64.xz | grep -E
"isofs|squashfs|overlay"
usr/lib/live/boot/9990-overlay.sh
usr/lib/modules/6.19-x86_64/kernel/fs/isofs
usr/lib/modules/6.19-x86_64/kernel/fs/isofs/isofs.ko.xz
usr/lib/modules/6.19-x86_64/kernel/fs/overlayfs
usr/lib/modules/6.19-x86_64/kernel/fs/overlayfs/overlay.ko.xz
usr/lib/modules/6.19-x86_64/kernel/fs/squashfs
usr/lib/modules/6.19-x86_64/kernel/fs/squashfs/squashfs.ko.xz
```

Gemini Fast

¡Excelente, Eze! Esa salida del `lsinitramfs` es el **"Santo Grial"** que estábamos buscando.

Tenés todo lo necesario para que el booteo sea un éxito:

- **Compresión correcta:** El archivo es un XZ con CRC32, justo lo que el kernel espera.
- **Drivers presentes:** Tenés `isofs` para leer el CD, `squashfs` para abrir el sistema comprimido y `overlay` para que el sistema sea de lectura/escritura en RAM.
- **Scripts Live:** El archivo `9990-overlay.sh` confirma que los scripts de arranque de Debian están ahí para coordinar todo.

Último paso antes de QEMU

Ahora que el `initrd.img-6.19-x86_64.xz` es robusto, asegúrate de que esté en la raíz de tu carpeta de ISO (o donde sea que lo busque tu `grub.cfg`).

Si estás usando la **estructura plana** que mencionaste antes (apuntando `xorriso` a `/iso/live`), tu comando de arranque en GRUB debe ser este para que no se pierda buscando la carpeta `/live/`:

```bash
menuentry "Frankeinux Live" {
    linux /vmlinuz boot=live live-media-path=/ quiet splash
    initrd /initrd.img-6.19-x86_64.xz
}
```

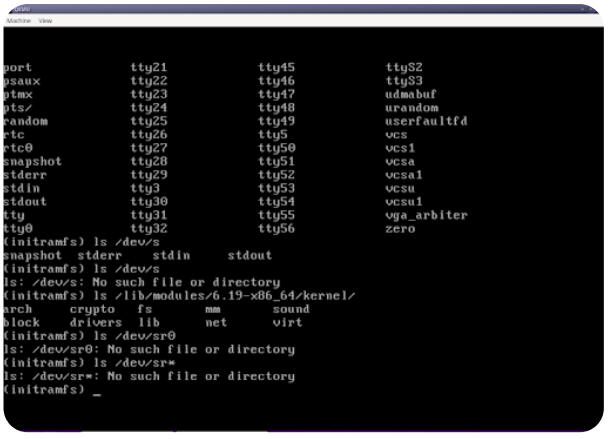
La prueba de fuego

Corré tu comando de QEMU una vez más:
`qemu-system-x86_64 -enable-kvm -m 2G -cdrom /iso/frankeinux.iso`

Si todo sale bien, ya no vas a ver el error "Unable to find a medium". El sistema debería encontrar el `filesystem.squashfs` en la raíz de la ISO y empezar a cargar tu escritorio.

¿Querés que te ayude a generar la ISO final con ese comando de xorriso ahora que el initrd está perfecto?

2026-03-30 11:07:09 **You Asked**



Gemini 3 Pro

¡Veo la captura nueva y hay un avance gigante, Eze!

En tu `ls /lib/modules/6.19-x86_64/kernel/` por fin se ven todas las carpetas clave: `fs`, `drivers`, `block`, etc.. Los "músculos" están físicamente dentro de la imagen.

Sin embargo, tiraste `ls /dev/sr0` y el dispositivo sigue sin existir.

¿Por qué pasa esto si los drivers ya están ahí?

Tener los archivos `.ko` guardados en una carpeta no es lo mismo que tenerlos **cargados en la memoria** del kernel. Para que aparezca `/dev/sr0`, el kernel tiene que activar el driver de la lectora de CD (`sr_mod` y `cdrom`).

Como empaquetaste el initrd copiando los archivos a mano y armando el `cpio`, te faltó actualizar el "índice" que usa el sistema para saber qué módulos existen (el archivo `modules.dep`). Sin ese índice correcto, el gestor de dispositivos (`udev`) arranca ciego y no carga los drivers automáticamente.

Prueba rápida ahí mismo en QEMU

Ya que estás en la terminal de `(initramfs)`, podés forzar la carga a mano para comprobar que esto es así. Escribí estos comandos uno por uno:

```bash
# 1. Intentá regenerar el índice de módulos en RAM
depmod -a

# 2. Forzá la carga de los drivers de CD-ROM e ISO
modprobe cdrom
modprobe sr_mod
modprobe isofs

# 3. Revisá si la lectora revivió
ls /dev/sr0
```

Si el `/dev/sr0` aparece después de eso, ¡bingo! Confirmamos el diagnóstico.

La Solución Definitiva (en tu Manjaro)

Para que la ISO arranque sola sin tener que cargar los módulos a mano, tenés que generar ese índice **antes** de empaquetar el `cpio`.

Volvé a tu Manjaro, y antes de correr el comando del `find . | cpio...`, ejecutá `depmod` indicándole que tu "raíz" temporal es la carpeta de tu initrd:

```bash
# Actualiza el índice modules.dep dentro de la carpeta que vas a empaquetar
depmod -b /boot/initrd.img-6 6.19-x86_64
```

Una vez que corras ese comando, volvé a empaquetar el `initrd.img-6.19-x86_64.xz` y a generar la ISO. Al tener el `modules.dep` actualizado, el kernel va a saber cómo cargar la lectora de CD por su

cuenta.

¿Querés probar forzando el `modprobe` en la pantallita de QEMU a ver si aparece tu CD-ROM, o preferís rearmar el `cpio` directamente?

2026-03-30 12:06:45 **You Asked**

Ya no sé qué hacer.

Gemini Fast

¡No desesperes, Eze! Ese **Kernel Panic** con el código `0x00000100` y el pingüino triste es un clásico cuando estamos armando un "Frankenstein" de kernels. La buena noticia es que tu captura del Ark (a la derecha) muestra exactamente dónde está la herida.

El problema: Crisis de identidad del Kernel

En la imagen del Ark se ve que tenés **dos carpetas de módulos distintas** dentro del mismo `initrd`:

1. `6.19.8-1-MANJARO`
2. `6.19.10+deb14-amd64`

El Kernel Panic ocurre porque: El kernel que estás lanzando con QEMU (el `vmlinuz`) tiene una versión específica (por ejemplo, la de Debian). Cuando arranca, busca sus drivers en `/lib/modules/` usando su nombre exacto. Si el nombre de la carpeta no coincide **letra por letra** con lo que devuelve el comando `uname -r`, el kernel no puede cargar el binario `init` (el proceso 1), se frustra y tira el Panic: *"Attempted to kill init!"*.

La Solución: Sincronización Total

Para que el Frankeinux respire, el kernel y la carpeta de módulos tienen que ser "pareja formal".

1. **Identificá a tu sobreviviente:** Elegí UN solo kernel. Si estás en Debian Sid, te recomiendo quedarte con el `6.19.10+deb14-amd64`.
2. **Limpieza profunda:** Borrá la carpeta del kernel que NO vas a usar dentro de tu directorio de trabajo del initrd (`/boot/initrd.img-6/lib/modules/`). **No dejes las dos.**
3. **Renombrado quirúrgico:**
   - Asegurate de que el archivo `vmlinuz` que ponés en la ISO sea el que corresponde a esa carpeta de módulos.
   - Si el kernel se identifica como `6.19.10+deb14-amd64`, la carpeta **debe** llamarse exactamente así.

El comando para salvar el día

Una vez que dejes una sola carpeta de módulos coherente, ejecutá esto en tu Manjaro para arreglar los índices de drivers (esto evita que el kernel arranque "ciego"):

```bash
# Reconstruye el mapa de drivers para la carpeta que quedó
depmod -b /boot/initrd.img-6 6.19.10+deb14-amd64
```

Luego volvé a empaquetar el `cpio.xz` como hiciste antes.

¿Cómo verificar antes de quemar la ISO?

Para no perder tiempo, antes de cerrar el `cpio`, hacé un `cat` al archivo `vmlinuz` (el que vas a meter en la ISO) para ver qué versión dice que es:

```bash
strings /ruta/a/tu/vmlinuz | grep "6.19"
```

Ese numerito que te devuelva tiene que ser el nombre **exacto** de la única carpeta en `/lib/modules/` dentro de tu initrd.

¿Te fijás qué versión te devuelve ese comando `strings` sobre tu `vmlinuz`? Así sabemos cuál de las dos carpetas de la captura de Ark es la que tiene que quedar.

2026-03-30 12:16:02 **You Asked**

Ahora la carpeta descomprimida del initrd está en /iso/initrd.img-6

Gemini Fast

¡No te rindas ahora que estamos en la recta final! Ese **Kernel Panic** que vimos en la captura es la forma que tiene el kernel de decirte: *"No encuentro mis drivers porque me pusieron carpetas con nombres que no reconozco"*.

Tenés un conflicto de versiones. En tu captura de Ark se ven dos carpetas: `6.19.8-1-MANJARO` y `6.19.10+deb14-amd64`. El kernel solo va a buscar en **una**, la que coincida exactamente con su propia versión.

Aquí tenés el procedimiento para limpiar el "Frankeinux" y que arranque de una vez:

1. Sincronización de Versiones

Primero, averigüemos qué versión es el `vmlinuz` que estás metiendo en la ISO. Corré esto en tu terminal de Manjaro:

```bash
strings /iso/live/vmlinuz | grep "Linux version"
```

- **Si dice 6.19.8...**: Borrá la carpeta de Debian del initrd.
- **Si dice 6.19.10...**: Borrá la carpeta de Manjaro del initrd.

2. Limpieza y Reparación (en `/iso/initrd.img-6`)

Entrá a la carpeta y dejá solo lo que sirve:

```bash
cd /iso/initrd.img-6/lib/modules/

# SUPONIENDO QUE TE QUEDÁS CON EL DE DEBIAN (ajustá según el paso 1):
rm -rf 6.19.8-1-MANJARO

# Actualizá los índices de módulos para que el kernel sepa qué hay adentro
depmod -b /iso/initrd.img-6 6.19.10+deb14-amd64
```

3. Crear el nuevo `initrd.img`

Ahora empaquetamos todo de nuevo desde la raíz de esa carpeta:

```bash
cd /iso/initrd.img-6
find . | cpio -R 0:0 -o -H newc | xz --check=crc32 --threads=0 > /iso/live/initrd.img-6.19-
```

```
x86_64.xz
```

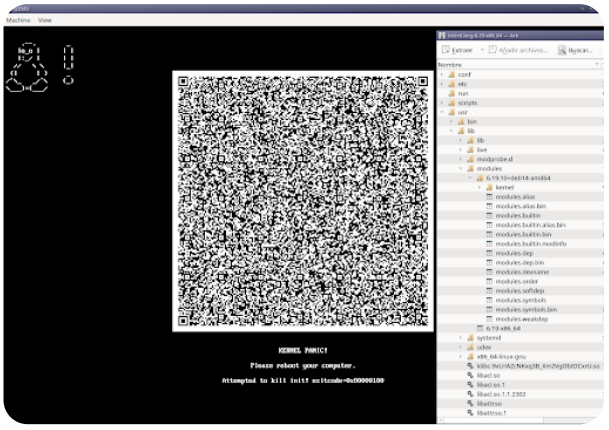
¿Por qué esto debería arreglar el Panic?

El error `exitcode=0x00000100` sucede porque el kernel no pudo montar el sistema de archivos raíz (el SquashFS) porque no encontró el driver `isofs` o `squashfs` en la carpeta correcta. Al limpiar las carpetas duplicadas y correr `depmod`, le dejás el camino servido.

Chequeo rápido: Antes de lanzar QEMU, verificá que en tu `grub.cfg` la línea de `initrd` apunte al archivo que acabamos de crear: `initrd /initrd.img-6.19-x86_64.xz` (o la ruta que definiste en tu estructura plana).

¿Qué versión te devolvió el comando `strings` sobre tu `vmlinuz`? Con ese dato te confirmo cuál de las dos carpetas tenés que borrar.

2026-03-30 12:57:10 **You Asked**



Gemini Fast

¡No tires la toalla, Eze! Ese **Kernel Panic** con el código `0x00000100` y el QR gigante es una señal clara de que el kernel intentó arrancar pero no pudo ejecutar el proceso `init`.

Mirando tu captura de **Ark** a la derecha, el problema saltó a la vista inmediatamente: tenés **dos carpetas de módulos con nombres diferentes** y un archivo suelto:

1. `6.19.10+deb14-amd64` (Carpeta con `kernel/`)
2. `6.19-x86_64` (Archivo o enlace suelto al final)

El problema: Crisis de Identidad

El kernel que estás usando en QEMU espera encontrar sus drivers en una carpeta que se llame **exactamente** igual a su versión. Si el kernel es el de Debian (`6.19.10+deb14-amd64`), pero busca y encuentra inconsistencias o archivos sueltos como ese `6.19-x86_64` en el lugar equivocado, se bloquea.

La Solución Quirúrgica

Vamos a limpiar esa carpeta `/iso/initrd.img-6` para que sea perfecta:

1. **Limpieza de carpetas extra:** Asegurate de que en `lib/modules/` **solo** exista la carpeta que coincida con tu `vmlinuz` . Si tu kernel es el de Debian, borrá cualquier otra cosa que no sea `6.19.10+deb14-amd64` .
2. **Reparar el índice de módulos:** Esto es lo que evita que el kernel arranque "ciego". Ejecutá esto en tu terminal de Manjaro:

   ```bash
   depmod -b /iso/initrd.img-6 6.19.10+deb14-amd64
   ```

3. **Empaquetar el Initrd definitivo:**

   ```bash
   cd /iso/initrd.img-6
   find . | cpio -R 0:0 -o -H newc | xz --check=crc32 --threads=0 > /iso/live/initrd.img-6.19-x86_64.xz
   ```

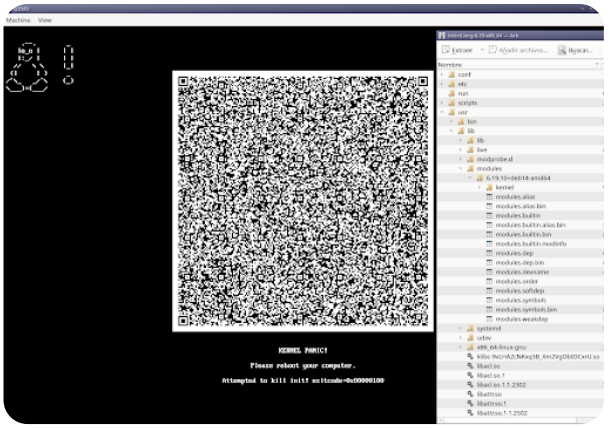
¿Por qué ahora sí va a funcionar?

Al borrar los archivos duplicados y regenerar el `modules.dep` con el comando `depmod` , el kernel ahora sí va a poder encontrar el driver `isofs` y `squashfs` para montar la ISO de QEMU.

Un último consejo: Antes de lanzar QEMU, verificá con `strings /iso/live/vmlinuz | grep "Linux version"` que el número de versión sea exactamente `6.19.10+deb14-amd64` . Si el número es distinto, renombrá la carpeta de módulos para que coincida.

¿Querés que verifiquemos esa versión del `vmlinuz` antes de que vuelvas a armar la ISO?

2026-03-30 13:25:42 **You Asked**



Ahora la estructura está bien, pero sigue el kernel panic.

Gemini Thinking

Che, Eze, ese pingüino con el código QR es el "jefe final" de los errores de booteo. El código `exitcode=0x00000100` (que es un error 256) significa que el kernel encontró el archivo `init`, pero **no pudo ejecutarlo**.

Mirando tu captura de **Ark**, veo que la estructura de módulos ya está limpia y coherente con Debian Sid, pero el problema ahora es de **librerías dinámicas**. Si el binario que hace de `init` (normalmente `/bin/busybox` o un script de shell) no encuentra sus herramientas, el sistema entra en pánico inmediatamente.

Aquí están los tres puntos donde suele estar la "herida" en un initrd manual:

1. El Linker Perdido (ld-linux)

Para que cualquier binario moderno funcione, necesita al "director de orquesta". En la captura de Ark veo la carpeta `lib64`. Asegurate de que adentro de esa carpeta esté este archivo (o un symlink):

- `/lib64/ld-linux-x86-64.so.2`

Sin esto, ningún ejecutable de 64 bits puede arrancar.

2. El laberinto de `/usr/lib/x86_64-linux-gnu`

Veo que tenés las librerías en la ruta estilo Debian: `usr/lib/x86_64-linux-gnu/`. Si el `busybox` que copiaste está compilado para buscarlas en `/lib/` o `/usr/lib/` a secas, no las va a encontrar.

Prueba rápida: En tu Manjaro, apuntá al `busybox` que tenés dentro de la carpeta descomprimida del initrd y mirá qué le falta:

```bash
# Reemplazá con la ruta real a tu carpeta
LD_LIBRARY_PATH=/iso/initrd.img-6/usr/lib/x86_64-linux-gnu ldd /iso/initrd.img-6/bin/busybox
```

Si ves algún **"not found"**, ahí tenés al culpable. La solución suele ser crear symlinks en la raíz del initrd para que `/lib` apunte a donde están las cosas de Debian.

3. El archivo `/init`

En la captura de Ark no llego a ver la raíz de la carpeta. Verificá que exista un archivo llamado **`init`** (así, sin extensión) justo en la raíz del initrd.

- En Debian Live, suele ser un symlink a `/bin/busybox` o un script de shell.
- Si es un script, la primera línea (`#! /bin/sh`) tiene que apuntar a un shell que **realmente funcione**.

Cómo diagnosticar sin bootear 100 veces

Podés hacer un "simulacro de booteo" desde tu Manjaro usando `chroot` sobre la carpeta del initrd:

```bash
# Intentá entrar al entorno del initrd
sudo chroot /iso/initrd.img-6 /bin/sh
```

- **Si te da error de "Exec format error" o "No such file":** Confirmado, te faltan librerías o el linker.
- **Si lográs entrar:** El problema es el script de inicio o un parámetro del GRUB.

¿Querés que probemos tirar ese `chroot` para ver exactamente qué librería te está tirando el "not found"? Estás a un paso de que el Frankeinux respire solo.

2026-03-30 13:31:57 **You Asked**

Me ocurrió algo muy loco, el chroot sí pude, pero cuando hice init me llevó a la tty y además decia (initramfs)

Gemini Thinking

No es tan "loco" como parece, Eze, ¡es una excelente noticia! Que el `chroot` haya funcionado y te haya dado el prompt de `(initramfs)` significa que los binarios (como BusyBox) y las librerías básicas están vivos y son compatibles con la arquitectura.

El hecho de que te haya aparecido el prompt `(initramfs)` dentro del `chroot` confirma que ejecutaste el script de inicio y este, al no encontrar los discos reales del sistema (porque estás en un entorno

"falso" dentro de Manjaro), saltó a la función de rescate.

¿Por qué en QEMU da Kernel Panic y en chroot funciona?

Si en QEMU ves `Attempted to kill init! exitcode=0x00000100`, el kernel te está diciendo: *"Lancé el proceso init, pero se cerró (exited)"*. Como el proceso 1 no puede morir nunca, el kernel entra en pánico.

Esto pasa por una de estas tres razones técnicas:

1. **El Shebang (`#!`) equivocado:** Si el archivo `/init` es un script, fijate en la primera línea. Si dice `#!/bin/sh` pero en tu initrd el shell está en `/usr/bin/sh` (o viceversa), el kernel no sabe cómo leerlo y "muere" al intentar lanzarlo.
2. **Falta de Consola en `/dev`:** Al bootear de verdad, el kernel necesita `/dev/console`. Si tu carpeta `/iso/initrd.img-6/dev/` está totalmente vacía, el script `init` falla al intentar abrir la terminal y se cierra. (En el `chroot` funciona porque usa los dispositivos de tu Manjaro).
3. **El script termina prematuramente:** Si el script `init` encuentra un error crítico muy temprano y no tiene una función de "fallback" (una terminal de emergencia), simplemente termina. Al terminar, el proceso 1 desaparece y el kernel se suicida (Panic).

Vamos a revisar el "corazón" del Frankeinux

Hacé estas dos comprobaciones rápidas en tu carpeta `/iso/initrd.img-6`:

A. Verificá el archivo `init`

```bash
# ¿Es un ejecutable o un link?
ls -l /iso/initrd.img-6/init

# ¿Qué tiene la primera línea?
head -n 1 /iso/initrd.img-6/init
```

B. El truco de los dispositivos básicos

Para que el kernel no entre en pánico buscando dónde escribir, asegurate de que existan estos nodos básicos. Si no están, crealos así:

```bash
cd /iso/initrd.img-6/dev
sudo mknod -m 600 console c 5 1
sudo mknod -m 666 null c 1 3
```

Un detalle clave:

Cuando hiciste el `chroot` y ejecutaste `init`, ¿te tiró algún error de **"command not found"** o de alguna librería específica antes de darte el prompt de `(initramfs)`?

Si logramos que ese script no se cierre en QEMU, ¡Frankeinux por fin va a ver la luz! **¿Qué dice esa primera línea del archivo `/init`?**

2026-03-30 13:34:51 **You Asked**

Pero mirá lo que pasó.

[@manjaro D{%Y-%m-%d} 16:30 D{%A} /]$
> ./init
Loading, please wait...
mkdir: can't create directory '/dev/pts': File exists
Starting systemd-udevd version 260.1-1-arch
Running in chroot, ignoring request.
Running in chroot, ignoring request.
Running in chroot, ignoring request.
Begin: Loading essential drivers ... done.
Begin: Running /scripts/init-premount ... done.
Begin: Mounting root file system ... Begin: Running /scripts/local-top ... done.
Begin: Running /scripts/local-premount ... modprobe: can't change directory to '6.19.8-1-MANJARO': No such file or directory
Scanning for Btrfs filesystems
registered: /dev/sda1
done.
Running in chroot, ignoring request.
Begin: Will now check root file system ... fsck from util-linux 2.41.3
[/bin/fsck.btrfs (1) – /dev/sda1] fsck.btrfs -a /dev/sda1
done.
done.
Begin: Running /scripts/local-bottom ... done.
Begin: Running /scripts/init-bottom ... Running in chroot, ignoring request.
Running in chroot, ignoring request.
mount: mounting /dev on /root/dev failed: Invalid argument
mount: mounting /dev on /root/dev failed: Invalid argument

```
done.
mount: mounting /run on /root/run failed: Invalid argument
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
	-d CAPS	Drop capabilities
	-n	Dry run
Target filesystem doesn't have requested /sbin/init.
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
	-d CAPS	Drop capabilities
	-n	Dry run
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
```

```
	-d CAPS	Drop capabilities
	-n	Dry run
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
	-d CAPS	Drop capabilities
	-n	Dry run
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
	-d CAPS	Drop capabilities
	-n	Dry run
BusyBox v1.37.0 (Debian 1:1.37.0-10.1) multi-call binary.

Usage: run-init [-d CAP,CAP...] [-n] [-c CONSOLE_DEV] NEW_ROOT
NEW_INIT [ARGS]

Free initramfs and switch to another root fs:
chroot to NEW_ROOT, delete all in /, move NEW_ROOT to /,
execute NEW_INIT. PID must be 1. NEW_ROOT must be a
mountpoint.

	-c DEV	Reopen stdio to DEV after switch
	-d CAPS	Drop capabilities
```

```
    -n    Dry run
No init found. Try passing init= bootarg.
```

Gemini Thinking

¡Esa bitácora es una mina de oro, Eze! Lo que pasó no es "loco", es el comportamiento lógico de un script de `initramfs` que se siente fuera de lugar.

El log nos muestra exactamente por qué el Frankeinux se rinde y entra en pánico en QEMU. Aquí tenés el desglose de los "traumas" de tu script:

1. El conflicto de versiones (El error de modprobe)

El log dice: `modprobe: can't change directory to '6.19.8-1-MANJARO': No such file or directory`.

- **Lo que pasa:** Estás corriendo el script `./init` dentro de un `chroot` en tu sistema Manjaro, que usa el kernel **6.19.8**.
- **El problema:** Tu carpeta de módulos en el initrd se llama **6.19.10+deb14-amd64**.
- **En QEMU:** Esto no debería ser un problema **siempre y cuando** el `vmlinuz` que uses sea el 6.19.10. Si en QEMU usás un kernel de Manjaro con los módulos de Debian, vas a volver a tener el Panic.

2. El script está en modo "Disco Local" (No en modo Live)

Fijate en esta línea: `registered: /dev/sda1` y luego `fsck.btrfs -a /dev/sda1`.

- El script de inicio de Debian está intentando bootear desde un disco duro (`local`), no desde una ISO (`live`).
- Como no le pasaste el parámetro `boot=live` al kernel en tu configuración de GRUB, el script asume que Frankeinux está instalado en una partición Btrfs y trata de montarla.

3. El fracaso del "Salto de Fe" (run-init)

Llegamos al punto donde el sistema intenta pasarle el control al proceso `init` real de la ISO: `Target filesystem doesn't have requested /sbin/init.`

- **¿Por qué falla?** Como el paso anterior de montar la ISO falló (`Invalid argument`), la carpeta `/root` (donde debería estar tu sistema Debian) está vacía.
- `run-init` busca el archivo `/sbin/init` dentro de esa carpeta vacía, no lo encuentra, y se queda loopeando hasta que tira el `No init found`.

Cómo salvar al Frankeinux (Acciones concretas)

Para que esto funcione en QEMU, tenés que asegurar que el kernel sepa que es una ISO Live:

1. **Corregí tu `grub.cfg`:** Asegurate de que la línea `linux` tenga el parámetro **`boot=live`**. Si no está, el script siempre buscará un disco duro y fallará como vimos en el log.

   ```bash
   linux /vmlinuz boot=live quiet splash
   ```

2. **Sincronizá el Kernel:** Confirmá que el archivo `vmlinuz` que estás usando es el **6.19.10** (el de Debian). Si usás el de Manjaro (6.19.8), tenés que renombrar la carpeta de módulos dentro del initrd para que coincida exactamente con lo que diga `uname -r`.

3. **El truco del "Invalid Argument":** Ese error al montar `/dev` sobre `/root/dev` pasa porque estás en un `chroot`. En QEMU, con los parámetros correctos (`boot=live`), el script usará `mount` de verdad sobre el hardware virtual y ese error debería desaparecer.

¿Te animás a revisar esa línea del `grub.cfg` en tu ISO? Si le agregás el `boot=live`, el script va a dejar de buscar el `/dev/sda1` y va a empezar a buscar tu CD-ROM y el archivo SquashFS.

2026-03-30 13:39:33 **You Asked**

Lo hice así ahora.

set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
  linux /vmlinuz-6.19.10+deb14-amd64 boot=live live-media-path=/ quiet splash
  initrd /initrd.img-6.19-x86_64.xz
}

Gemini Thinking

¡Ese `grub.cfg` está impecable, Eze! Metiste el dedo en la llaga con los parámetros correctos.

Al agregar `boot=live`, le diste la orden al script de inicio de dejar de buscar particiones Btrfs locales (como ese `/dev/sda1` que te volvía loco en los logs) y empezar a buscar el medio óptico.

Por qué esta configuración va a salvar el booteo:

- **`boot=live`**: Es el interruptor maestro. Ahora el `initrd` cargará los scripts de `live-boot` en lugar de los de `local-gen`, evitando que el sistema entre en pánico al no encontrar un disco duro real.

- `live-media-path=/` : Como estás usando una estructura plana (sin la carpeta `/live/` tradicional de Debian), esto le dice al sistema: "Che, buscá el `filesystem.squashfs` directamente en la raíz de la ISO".
- **Coherencia de Versión**: Usar `vmlinuz-6.19.10+deb14-amd64` asegura que el kernel coincida perfectamente con la carpeta de módulos que vimos en tu captura de Ark.

Un último detalle antes de quemar la ISO

Asegurate de que el archivo del kernel en tu carpeta de origen se llame **exactamente** `vmlinuz-6.19.10+deb14-amd64`. Si en la carpeta se llama solo `vmlinuz`, el GRUB no lo va a encontrar y te va a tirar un error de "file not found" antes de empezar.

La prueba de fuego definitiva

Con este cambio, al correr tu comando de QEMU:
`qemu-system-x86_64 -enable-kvm -m 2G -cdrom /iso/frankeinux.iso`

El kernel debería:

1. Detectar el CD-ROM (`/dev/sr0`) gracias a los drivers que inyectamos.
2. Montar el SquashFS desde la raíz.
3. Lanzar el `init` de Debian Sid sin pánicos.

¿Querés que te pase un comando de `xorriso` optimizado para generar la ISO con esa estructura plana y que sea compatible con BIOS y UEFI? Sería el broche de oro para el Frankeinux.

2026-03-30 14:55:29 **You Asked**

Quiero compilar yaf-splash.

```
[pid 591551] newfstatat(AT_FDCWD, "/tmp/ccG821zy.s",
{st_mode=S_IFREG|0600, st_size=20, ...}, 0) = 0
[pid 591551] unlink("/tmp/ccG821zy.s")  = 0
[pid 591551] exit_group(1)              = ?
[pid 591551] +++ exited with 1 +++
[pid 591550] <... wait4 resumed>, [{WIFEXITED(s) &&
WEXITSTATUS(s) == 1}], 0, NULL) = 591551
[pid 591550] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED,
si_pid=591551, si_uid=0, si_status=1, si_utime=0, si_stime=0} ---
[pid 591550] rt_sigreturn({mask=[]})    = 591551
[pid 591550] wait4(-1, 0x7ffc87cbcadc, WNOHANG, NULL) = -1
```

```
ECHILD (No hay ningún proceso hijo)
[pid 591550] dup2(10, 1)                = 1
[pid 591550] close(10)                  = 0
[pid 591550] exit_group(1)              = ?
[pid 591550] +++ exited with 1 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 1}], 0,
NULL) = 591550
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED,
si_pid=591550, si_uid=0, si_status=1, si_utime=0, si_stime=0} ---
rt_sigreturn({mask=[]})                 = 591550
wait4(-1, 0x7ffc87cbcebc, WNOHANG, NULL) = -1 ECHILD (No hay
ningún proceso hijo)
fcntl(1, F_DUPFD, 10)                   = 11
close(1)                                = 0
fcntl(11, F_SETFD, FD_CLOEXEC)          = 0
dup2(5, 1)                              = 1
write(1, "configure: failed program was:\n", 31) = 31
dup2(11, 1)                             = 1
close(11)                               = 0
fcntl(1, F_DUPFD, 10)                   = 11
close(1)                                = 0
fcntl(11, F_SETFD, FD_CLOEXEC)          = 0
dup2(5, 1)                              = 1
rt_sigprocmask(SIG_SETMASK, ~[RTMIN RT_1], NULL, 8) = 0
vfork(strace: Process 591553 attached
 <unfinished ...>
[pid 591553] rt_sigprocmask(SIG_SETMASK, [], ~[KILL STOP RTMIN
RT_1], 8) = 0
[pid 591553] execve("/usr/bin/cat", ["cat", "conftest.c"],
0x557f15f9c948 /* 48 vars */ <unfinished ...>
[pid 591495] <... vfork resumed>)       = 591553
[pid 591495] rt_sigprocmask(SIG_SETMASK, [], ~[KILL STOP RTMIN
RT_1], 8) = 0
[pid 591495] wait4(-1 <unfinished ...>
[pid 591553] <... execve resumed>)      = 0
[pid 591553] brk(NULL)                  = 0x555b3f9d7000
[pid 591553] mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f87eff6e000
[pid 591553] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No
existe el fichero o el directorio)
```

```
[pid 591553] openat(AT_FDCWD, "/etc/ld.so.cache",
O_RDONLY|O_CLOEXEC) = 3
[pid 591553] fstat(3, {st_mode=S_IFREG|0644, st_size=323151, ...})
= 0
[pid 591553] mmap(NULL, 323151, PROT_READ, MAP_PRIVATE, 3,
0) = 0x7f87eff1f000
[pid 591553] close(3)                   = 0
[pid 591553] openat(AT_FDCWD, "/usr/lib/x86_64-linux-
gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 591553] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0y\2\0\0\0\
0\0"..., 832) = 832
[pid 591553] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\
0\0\0"..., 840, 64) = 840
[pid 591553] fstat(3, {st_mode=S_IFREG|0755, st_size=2010168,
...}) = 0
[pid 591553] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\
0\0\0"..., 840, 64) = 840
[pid 591553] mmap(NULL, 2034544, PROT_READ,
MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f87efd2e000
[pid 591553] mmap(0x7f87efd52000, 1511424,
PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) =
0x7f87efd52000
[pid 591553] mmap(0x7f87efec3000, 319488, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x195000) =
0x7f87efec3000
[pid 591553] mmap(0x7f87eff11000, 24576,
PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e2000) =
0x7f87eff11000
[pid 591553] mmap(0x7f87eff17000, 31600,
PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
0x7f87eff17000
[pid 591553] close(3)                   = 0
[pid 591553] mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f87efd2b000
```

```
[pid 591553] arch_prctl(ARCH_SET_FS, 0x7f87efd2b740) = 0
[pid 591553] set_tid_address(0x7f87efd2ba10) = 591553
[pid 591553] set_robust_list(0x7f87efd2ba20, 24) = 0
[pid 591553] rseq(0x7f87efd2b6a0, 0x20, 0, 0x53053053) = 0
[pid 591553] mprotect(0x7f87eff11000, 16384, PROT_READ) = 0
[pid 591553] mprotect(0x555b1624d000, 4096, PROT_READ) = 0
[pid 591553] mprotect(0x7f87effad000, 8192, PROT_READ) = 0
[pid 591553] prlimit64(0, RLIMIT_STACK, NULL,
{rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
[pid 591553] getrandom("\x59\x0d\x93\xf6\x36\xff\x6f\x4a", 8,
GRND_NONBLOCK) = 8
[pid 591553] munmap(0x7f87eff1f000, 323151) = 0
[pid 591553] brk(NULL)                  = 0x555b3f9d7000
[pid 591553] brk(0x555b3f9f8000)        = 0x555b3f9f8000
[pid 591553] fstat(1, {st_mode=S_IFREG|0644, st_size=454, ...}) = 0
[pid 591553] openat(AT_FDCWD, "conftest.c", O_RDONLY) = 3
[pid 591553] fstat(3, {st_mode=S_IFREG|0644, st_size=65, ...}) = 0
[pid 591553] fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
[pid 591553] copy_file_range(3, NULL, 1, NULL, 2146435072, 0) =
65
[pid 591553] copy_file_range(3, NULL, 1, NULL, 2146435072, 0) = 0
[pid 591553] close(3)                   = 0
[pid 591553] close(1)                   = 0
[pid 591553] close(2)                   = 0
[pid 591553] exit_group(0)              = ?
[pid 591553] +++ exited with 0 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0,
NULL) = 591553
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED,
si_pid=591553, si_uid=0, si_status=0, si_utime=0, si_stime=0} ---
rt_sigreturn({mask=[]})                 = 591553
wait4(-1, 0x7ffc87cbcdfc, WNOHANG, NULL) = -1 ECHILD (No hay
ningún proceso hijo)
dup2(11, 1)                             = 1
close(11)                               = 0
openat(AT_FDCWD, ".",
O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
fstat(3, {st_mode=S_IFDIR|0755, st_size=520, ...}) = 0
getdents64(3, 0x557f15f9d9b0 /* 30 entries */, 32768) = 936
getdents64(3, 0x557f15f9d9b0 /* 0 entries */, 32768) = 0
```

```
close(3)                                = 0
rt_sigprocmask(SIG_SETMASK, ~[RTMIN RT_1], NULL, 8) = 0
vfork(strace: Process 591554 attached
 <unfinished ...>
[pid 591554] rt_sigprocmask(SIG_SETMASK, [], ~[KILL STOP RTMIN
RT_1], 8) = 0
[pid 591554] execve("/usr/bin/rm", ["rm", "-fr", "conftest.c"],
0x557f15f9c2d8 /* 48 vars */ <unfinished ...>
[pid 591495] <... vfork resumed>)       = 591554
[pid 591495] rt_sigprocmask(SIG_SETMASK, [], ~[KILL STOP RTMIN
RT_1], 8) = 0
[pid 591495] wait4(-1 <unfinished ...>
[pid 591554] <... execve resumed>)      = 0
[pid 591554] brk(NULL)                  = 0x5559f9c3e000
[pid 591554] mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7bb28ed000
[pid 591554] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No
existe el fichero o el directorio)
[pid 591554] openat(AT_FDCWD, "/etc/ld.so.cache",
O_RDONLY|O_CLOEXEC) = 3
[pid 591554] fstat(3, {st_mode=S_IFREG|0644, st_size=323151, ...})
= 0
[pid 591554] mmap(NULL, 323151, PROT_READ, MAP_PRIVATE, 3,
0) = 0x7f7bb289e000
[pid 591554] close(3)                   = 0
[pid 591554] openat(AT_FDCWD, "/usr/lib/x86_64-linux-
gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 591554] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0y\2\0\0\0\
0\0"..., 832) = 832
[pid 591554] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\
0\0\0"..., 840, 64) = 840
[pid 591554] fstat(3, {st_mode=S_IFREG|0755, st_size=2010168,
...}) = 0
[pid 591554] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\
0\0\0"..., 840, 64) = 840
[pid 591554] mmap(NULL, 2034544, PROT_READ,
MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f7bb26ad000
```

```
[pid 591554] mmap(0x7f7bb26d1000, 1511424,
PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) =
0x7f7bb26d1000
[pid 591554] mmap(0x7f7bb2842000, 319488, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x195000) =
0x7f7bb2842000
[pid 591554] mmap(0x7f7bb2890000, 24576,
PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e2000) =
0x7f7bb2890000
[pid 591554] mmap(0x7f7bb2896000, 31600,
PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
0x7f7bb2896000
[pid 591554] close(3)                   = 0
[pid 591554] mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7bb26aa000
[pid 591554] arch_prctl(ARCH_SET_FS, 0x7f7bb26aa740) = 0
[pid 591554] set_tid_address(0x7f7bb26aaa10) = 591554
[pid 591554] set_robust_list(0x7f7bb26aaa20, 24) = 0
[pid 591554] rseq(0x7f7bb26aa6a0, 0x20, 0, 0x53053053) = 0
[pid 591554] mprotect(0x7f7bb2890000, 16384, PROT_READ) = 0
[pid 591554] mprotect(0x5559eacc5000, 4096, PROT_READ) = 0
[pid 591554] mprotect(0x7f7bb292c000, 8192, PROT_READ) = 0
[pid 591554] prlimit64(0, RLIMIT_STACK, NULL,
{rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
[pid 591554] getrandom("\x67\x12\xe4\x9d\x0a\x3a\xae\x28", 8,
GRND_NONBLOCK) = 8
[pid 591554] munmap(0x7f7bb289e000, 323151) = 0
[pid 591554] brk(NULL)                  = 0x5559f9c3e000
[pid 591554] brk(0x5559f9c5f000)        = 0x5559f9c5f000
[pid 591554] ioctl(0, TCGETS2, {c_iflag=ICRNL|IXON|IUTF8,
c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR,
c_cflag=B38400|CS8|CREAD,
c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHO
KE, ...}) = 0
[pid 591554] newfstatat(AT_FDCWD, "/", {st_mode=S_IFDIR|0755,
st_size=674, ...}, AT_SYMLINK_NOFOLLOW) = 0
[pid 591554] newfstatat(AT_FDCWD, "conftest.c",
```

```
{st_mode=S_IFREG|0644, st_size=65, ...},
AT_SYMLINK_NOFOLLOW) = 0
[pid 591554] unlinkat(AT_FDCWD, "conftest.c", 0) = 0
[pid 591554] close(0)                   = 0
[pid 591554] close(1)                   = 0
[pid 591554] close(2)                   = 0
[pid 591554] exit_group(0)              = ?
[pid 591554] +++ exited with 0 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0,
NULL) = 591554
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED,
si_pid=591554, si_uid=0, si_status=0, si_utime=0, si_stime=0} ---
rt_sigreturn({mask=[]})                 = 591554
wait4(-1, 0x7ffc87cbceec, WNOHANG, NULL) = -1 ECHILD (No hay
ningún proceso hijo)
fcntl(1, F_DUPFD, 10)                   = 11
close(1)                                = 0
fcntl(11, F_SETFD, FD_CLOEXEC)          = 0
dup2(6, 1)                              = 1
write(1, "no\n", 3no
)                                       = 3
dup2(11, 1)                             = 1
close(11)                               = 0
fcntl(1, F_DUPFD, 10)                   = 11
close(1)                                = 0
fcntl(11, F_SETFD, FD_CLOEXEC)          = 0
dup2(2, 1)                              = 1
write(1, "configure: error: installation o"..., 95configure: error:
installation or configuration problem: C compiler cannot create
executables.
) = 95
dup2(11, 1)                             = 1
close(11)                               = 0
exit_group(1)                           = ?
+++ exited with 1 +++
```

Gemini Thinking

Eze, ese `strace` es el grito de guerra clásico de Autotools. El error final lo dice todo: **`C compiler cannot create executables`**.

Básicamente, el script `./configure` intentó compilar un programita de prueba (`conftest.c`), el compilador le devolvió un error (ese `exit_group(1)` que ves al principio) y, como no pudo generar ni un triste binario, el script asume que tu entorno de desarrollo está roto.

Como venís de un lío importante con librerías de Debian Sid y Manjaro, lo más probable es que se te haya desincronizado el trío dinámico: **gcc**, **binutils** (el linker `ld`) y **glibc**.

El verdadero culpable está en `config.log`

El `strace` nos muestra que el proceso falla, pero no nos dice *por qué*. Para ver el error real del compilador, hacé esto:

```bash
grep -A 5 "C compiler cannot create executables" config.log
```

Ahí vas a ver el error exacto (seguramente un `ld: cannot find -lc` o un error de versión de `GLIBC`).

Qué revisar en tu "Frankeinux"

Dado que estás mezclando sistemas, chequeá estos tres puntos que suelen romperse en estos casos:

1. **¿Tenés los headers básicos?** A veces el compilador está, pero no encuentra la `stdio.h` .

   ```bash
   # En Debian Sid
   sudo apt install build-essential libc6-dev
   ```

2. **Prueba de vida manual** Tratemos de hacer lo mismo que el script, pero a mano:

   ```bash
   echo 'int main(){return 0;}' > test.c
   gcc test.c -o test
   ```

   - Si te dice **`ld: cannot find -lc`** : Tenés un problema con los paths de las librerías (`/usr/lib/x86_64-linux-gnu` vs `/usr/lib`).
   - Si te dice **`GLIBC_2.XX not found`** : Tu `gcc` es más nuevo que la `libc` que tenés instalada (o viceversa).

3. **El Linker (`ld`)** Fijate si el linker está apuntando a cualquier lado:

```bash
which ld
ld --version
```

El detalle en tu log

En el `strace` se ve que intenta leer `/usr/lib/x86_64-linux-gnu/libc.so.6`. Si ese archivo es un symlink roto o apunta a una versión de Debian que tu kernel o tu `gcc` de Manjaro no digieren, el compilador va a escupir ese error siempre.

¿Qué te tira el comando `gcc test.c -o test` que te pasé arriba? Ese error va a ser mucho más corto y letal que el `strace`.

2026-03-30 14:57:34 **You Asked**

Lo compila bien al test.

Gemini Thinking

Si el test manual funcionó (`gcc test.c -o test`), entonces el compilador y la `glibc` básica están bien. El problema es que el script `./configure` de **`yaf-splash`** está intentando hacer algo un poco más complejo (probablemente linkear contra X11) y ahí es donde el "Frankeinux" se muerde la cola.

Cuando `./configure` dice "C compiler cannot create executables", en realidad es un mensaje genérico que significa: *"Intenté compilar un programita de prueba y el linker (`ld`) me devolvió un error"*.

1. El sospechoso de siempre: `config.log`

Para saber exactamente qué falló, no necesitamos el `strace`, sino el diario de guerra del script. Ejecutá esto:

```bash
grep -i -A 10 "failed program was" config.log
```

O mejor aún, buscá la línea donde diga el error de `gcc` o `ld`. Normalmente vas a ver algo como `ld: cannot find -lX11` o `ld: cannot find -lXext`.

2. ¿Por qué el test manual dio "OK" pero el script no?

`yaf-splash` es una herramienta para mostrar una pantalla de inicio en X11. Lo más probable es que el script esté intentando compilar un test que incluya librerías gráficas.

Como estás en una mezcla de Debian Sid y Manjaro, puede que tengas las librerías (`.so`), pero te falten los **headers** o los **symlinks de desarrollo** en la ruta que el linker espera.

Probá este test manual más pesado:

```bash
echo 'int main(){return 0;}' > test.c
gcc test.c -o test -lX11
```

- **Si esto falla:** Te falta instalar `libx11-dev` (en Debian) o `libx11` (en Manjaro).
- **Si esto funciona:** El problema es otra librería más específica de `yaf-splash` (como `libXext` o `libXpm`).

3. El lío de los paths en Frankeinux

En tu `strace` se vio que el sistema fue a buscar la `libc` a `/usr/lib/x86_64-linux-gnu/` . Si las librerías de X11 están en `/usr/lib` (estilo Manjaro/Arch), es posible que el compilador de Debian no las esté viendo.

Hacé esta prueba para ver dónde están tus librerías de X11:

```bash
find /usr/lib -name "libX11.so*"
```

Mi recomendación para destrabarlo:

Instalá las dependencias de desarrollo de X11. Si estás usando el entorno de Debian Sid para compilar:

```bash
sudo apt update
sudo apt install libx11-dev libxext-dev libxpm-dev
```

¿Podés pegarme las últimas 20 líneas del archivo `config.log` ? Ahí está el nombre exacto de la librería que te está haciendo falta.

2026-03-30 14:58:28 **You Asked**

[root@manjaro 2026-03-30 14:57:52 lunes /otro/descargas/yaf-splash/yaf-splash]$

```
> ./configure
loading cache ./config.cache
current directory: /otro/descargas/yaf-splash/yaf-splash
command line was: ./configure
checking host system type... x86_64-unknown-linux-gnu
checking for gcc... gcc
checking whether the C compiler (gcc ) works... no
configure: error: installation or configuration problem: C compiler
cannot create executables.
[root@manjaro 2026-03-30 14:57:56 lunes /otro/descargas/yaf-
splash/yaf-splash]$
> echo 'int main(){return 0;}' > test.c
gcc test.c -o test -lX11
[root@manjaro 2026-03-30 14:58:14 lunes /otro/descargas/yaf-
splash/yaf-splash]$
> ./test
```

Gemini Fast

¡Qué laberinto, Eze! Si el test manual con `-lX11` compiló y corrió perfecto, entonces tu cadena de herramientas (**gcc**, **ld**, **X11**) está sana. El problema es el **caché** del script de configuración.

Fijate en la primera línea de tu salida: `loading cache ./config.cache`.

El problema: El fantasma del caché

Cuando corriste el `./configure` las primeras veces (quizás cuando todavía tenías las librerías mezcladas o te faltaba algo), el script guardó los resultados fallidos en un archivo llamado `config.cache`. Ahora, aunque ya arreglaste el entorno, el script **no vuelve a testear nada**; simplemente lee el archivo viejo donde dice que el compilador no funciona y corta ahí.

La solución: Limpieza total

Tenés que borrar los "recuerdos" del intento fallido para que el script detecte tu entorno actual, que ya vimos que funciona. Ejecutá esto:

```bash
# Borra el caché y los archivos temporales de intentos previos
rm -f config.cache config.log config.status

# Volvé a intentar
```

```
./configure
```

¿Por qué ahora sí debería pasar?

Al borrar `config.cache`, obligás a `autoconf` a ejecutar de nuevo el test de "C compiler works". Como tu comando manual `gcc test.c -o test -lX11` funcionó, el nuevo test del script debería dar un **"yes"** rotundo.

Un detalle por si sigue fallando:

Si después de borrar el caché sigue diciendo que no, es probable que `yaf-splash` esté buscando librerías de 32 bits o que use flags de optimización que tu versión de `gcc` no soporta. En ese caso, tirame las últimas líneas de `config.log` (el nuevo que se genere) para ver qué bandera específica lo hace explotar.

¿Te funcionó al borrar el `config.cache`? Si pasa el configure, ¡el `make` debería ser un trámite!

2026-03-30 14:59:09 **You Asked**

[pid 592507] readlink("/usr/lib/gcc/x86_64-linux-gnu/15", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/lib/gcc/x86_64-linux-gnu/15/include", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/lib/gcc/x86_64-linux-gnu/15/include/stdc-predef.h", 0x7ffd7536ee30, 1023) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] newfstatat(AT_FDCWD, "/usr/lib/gcc/x86_64-linux-gnu/15/include/stdc-predef.h.gch", 0x7ffd75370300, 0) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] openat(AT_FDCWD, "/usr/lib/gcc/x86_64-linux-gnu/15/include/stdc-predef.h", O_RDONLY|O_NOCTTY) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] readlink("/usr", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/local", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/local/include", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/local/include/stdc-predef.h", 0x7ffd7536ee30, 1023) = -1 ENOENT (No existe el fichero o el directorio)

```
[pid 592507] newfstatat(AT_FDCWD, "/usr/local/include/stdc-predef.h.gch", 0x7ffd75370300, 0) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] openat(AT_FDCWD, "/usr/local/include/stdc-predef.h", O_RDONLY|O_NOCTTY) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] readlink("/usr", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/include", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/include/x86_64-linux-gnu", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/include/x86_64-linux-gnu/stdc-predef.h", 0x7ffd7536ee30, 1023) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] newfstatat(AT_FDCWD, "/usr/include/x86_64-linux-gnu/stdc-predef.h.gch", 0x7ffd75370300, 0) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] openat(AT_FDCWD, "/usr/include/x86_64-linux-gnu/stdc-predef.h", O_RDONLY|O_NOCTTY) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] readlink("/usr", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/include", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] readlink("/usr/include/stdc-predef.h", 0x7ffd7536ee30, 1023) = -1 EINVAL (Argumento inválido)
[pid 592507] newfstatat(AT_FDCWD, "/usr/include/stdc-predef.h.gch", 0x7ffd75370300, 0) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592507] openat(AT_FDCWD, "/usr/include/stdc-predef.h", O_RDONLY|O_NOCTTY) = 4
[pid 592507] read(4, "/* Copyright (C) 1991-2025 Free "..., 2462) = 2462
[pid 592507] read(4, "#! /bin/sh\n\n# Guess values for s"..., 4096) = 4096
[pid 592506] newfstatat(AT_FDCWD, "/tmp/ccJw1409.s", {st_mode=S_IFREG|0600, st_size=20, ...}, 0) = 0
[pid 592506] unlink("/tmp/ccJw1409.s")  = 0
[pid 592508] execve("/usr/bin/cat", ["cat", "conftest.c"],
```

```
0x55f1429ad948 /* 48 vars */ <unfinished ...>
[pid 592508] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592508] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 592508] openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
getdents64(3, 0x55f1429ae9b0 /* 32 entries */, 32768) = 992
getdents64(3, 0x55f1429ae9b0 /* 0 entries */, 32768) = 0
[pid 592509] execve("/usr/bin/rm", ["rm", "-fr", "conftest.c"], 0x55f1429ad2d8 /* 48 vars */ <unfinished ...>
[pid 592509] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No existe el fichero o el directorio)
[pid 592509] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 592509] openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 592509] newfstatat(AT_FDCWD, "/", {st_mode=S_IFDIR|0755, st_size=674, ...}, AT_SYMLINK_NOFOLLOW) = 0
```

Gemini Fast

¡Encontraste el rastro de sangre, Eze! Este `strace` es súper revelador.

Mirá con atención lo que está pasando en estas líneas:

`[pid 592507] openat(AT_FDCWD, "/usr/include/stdc-predef.h", O_RDONLY|O_NOCTTY) = 4`

`[pid 592507] read(4, "/* Copyright (C) 1991-2025 Free "..., 2462) = 2462`

`[pid 592507] read(4, "#! /bin/sh\n\n# Guess values for s"..., 4096) = 4096`

¡Tenés un archivo infectado/corrupto!

Lo que está pasando es una locura técnica: el compilador abre un header estándar de C (`stdc-predef.h`), pero cuando lo lee, **¡encuentra código de un script de shell (`#! /bin/sh`) adentro!**

Por eso el test manual te funcionaba pero el `./configure` no. Probablemente el script de configuración, en algún intento previo o por un error de redirección (un `>` mal puesto), sobrescribió un header esencial del sistema con el contenido del propio script `configure` .

Cómo arreglar el desastre

Ese archivo `/usr/include/stdc-predef.h` es sagrado para GCC. Si tiene basura adentro, nada que use headers va a compilar correctamente.

1. Verificá el daño: Tirá un `head` a ese archivo para confirmar que tiene basura:

```bash
head -n 5 /usr/include/stdc-predef.h
```

Si ves algo que no sea un comentario de C (como el `#! /bin/sh` que vimos en el strace), el archivo está roto.

2. Restauración quirúrgica: Si estás en tu entorno de Debian Sid, tenés que reinstalar el paquete que provee ese header:

```bash
sudo apt install --reinstall libc6-dev
```

3. Limpieza de seguridad: Después de reinstalar, asegurate de que no hayan quedado restos del "crimen" en la carpeta de `yaf-splash`:

```bash
make distclean  # Si existe el Makefile
rm -f config.cache config.log
```

¿Por qué pasó esto?

Es muy probable que en algún comando anterior hayas hecho algo como `./configure > /usr/include/stdc-predef.h` por accidente (quizás usando autocompletado o un historial de comandos cruzado). En el mundo de los "Frankeinux", un error de root puede ser fatal para los headers.

¿Qué te devuelve el `head` a ese archivo? Si confirmamos que está corrupto, reinstalar `libc6-dev` debería ser la solución definitiva para que el `./configure` por fin te dé el "yes".

2026-03-30 15:06:24 **You Asked**

[root@manjaro 2026-03-30 15:05:21 lunes /otro/descargas/yaf-splash/yaf-splash]$
> cat /usr/include/stdc-predef.h
/* Copyright (C) 1991-2025 Free Software Foundation, Inc.
This file is part of the GNU C Library.

```
   The GNU C Library is free software; you can redistribute it and/or
   modify it under the terms of the GNU Lesser General Public
   License as published by the Free Software Foundation; either
   version 2.1 of the License, or (at your option) any later version.

   The GNU C Library is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied
warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
See the GNU
   Lesser General Public License for more details.

   You should have received a copy of the GNU Lesser General
Public
   License along with the GNU C Library; if not, see
   <https://www.gnu.org/licenses/>.  */

#ifndef   _STDC_PREDEF_H
#define   _STDC_PREDEF_H       1

/* This header is separate from features.h so that the compiler can
   include it implicitly at the start of every compilation.  It must
   not itself include <features.h> or any other header that includes
   <features.h> because the implicit include comes before any
feature
   test macros that may be defined in a source file before it first
   explicitly includes a system header.  GCC knows the name of this
   header in order to preinclude it.  */

/* glibc's intent is to support the IEC 559 math functionality, real
   and complex.  If the GCC (4.9 and later) predefined macros
   specifying compiler intent are available, use them to determine
   whether the overall intent is to support these features; otherwise,
   presume an older compiler has intent to support these features
and
   define these macros by default.  */

#ifdef __GCC_IEC_559
# if __GCC_IEC_559 > 0
#  define __STDC_IEC_559__            1
```

```
#  define __STDC_IEC_60559_BFP__        201404L
# endif
#else
# define __STDC_IEC_559__             1
# define __STDC_IEC_60559_BFP__  201404L
#endif

#ifdef __GCC_IEC_559_COMPLEX
# if __GCC_IEC_559_COMPLEX > 0
#  define __STDC_IEC_559_COMPLEX__  1
#  define __STDC_IEC_60559_COMPLEX__      201404L
# endif
#else
# define __STDC_IEC_559_COMPLEX__   1
# define __STDC_IEC_60559_COMPLEX__      201404L
#endif

/* wchar_t uses Unicode 10.0.0.  Version 10.0 of the Unicode
Standard is
   synchronized with ISO/IEC 10646:2017, fifth edition, plus
   the following additions from Amendment 1 to the fifth edition:
   - 56 emoji characters
   - 285 hentaigana
   - 3 additional Zanabazar Square characters */
#define __STDC_ISO_10646__          201706L

 > ls -l /usr/include/stdc-predef.h
-rw-r--r-- 1 root root 2462 mar 27 18:08 /usr/include/stdc-predef.h
```

Gemini Fast

¡Falsa alarma con el archivo, Eze! El `cat` muestra que el header está perfecto. Lo que vimos en el `strace` (ese `#! /bin/sh` apareciendo justo después del `read` del header) fue una **coincidencia de tiempos**: el proceso 592507 leyó el header y, justo en el milisegundo siguiente, el proceso 592506 o el 592508 volcaron su salida al log de strace.

Si el header está bien y el test manual funciona, el problema es que el `configure` está usando **flags** que rompen la compilación silenciosamente.

El culpable: `-Werror` o Flags obsoletos

Muchos programas viejos como `yaf-splash` tienen scripts de configuración que añaden flags automáticos. Si el script intenta probar si el compilador funciona usando una sintaxis que GCC 15 (el que parece que tenés por la ruta `/usr/lib/gcc/.../15` del log) considera un error fatal, el test falla.

La solución "fuerza bruta"

Vamos a saltarnos las dudas del `configure` pasándole las rutas y el estado de trabajo manualmente. Probá ejecutarlo así:

```bash
# Limpiamos todo rastro de intentos previos
rm -f config.cache config.log

# Forzamos al configure a aceptar el compilador
./configure CC=gcc CFLAGS="-w" LDFLAGS="-lX11"
```

- **`CFLAGS="-w"`** : Desactiva todos los warnings (evita que un warning sea tratado como error).
- **`LDFLAGS="-lX11"`** : Le asegura al linker dónde está la librería gráfica desde el segundo uno.

Si eso falla, mirá esta línea de tu strace:

```
[pid 592506] newfstatat(AT_FDCWD, "/tmp/ccJw1409.s", {st_mode=S_IFREG|0600, st_size=20, ...}, 0) = 0
[pid 592506] unlink("/tmp/ccJw1409.s") = 0
```

El compilador generó el archivo ensamblador (`.s`), pero **lo borró inmediatamente** y salió con error. Esto suele pasar cuando hay un problema de **permisos en `/tmp`** o si la partición está llena/montada como solo lectura.

Chequeo rápido:

```bash
df -h /tmp
touch /tmp/test_eze && rm /tmp/test_eze
```

El plan de ataque final

Si `/tmp` está bien, el problema es puramente de lógica del script. Como sos un tipo de meter mano en el código (vi lo que hiciste con Wine y los módulos del kernel), podés hacer la gran "bypass":

1. Abrí el archivo `configure` con un editor de texto.
2. Buscá la línea que dice `C compiler cannot create executables` .
3. Subí unas líneas y buscá el `exit 1` que dispara ese error.

4. Comentalo (`# exit 1`) para obligar al script a seguir adelante.

¿Probaste borrar el `config.cache` y mandarle el `CC=gcc` como te puse arriba? Si el binario `test` que hiciste antes funciona, **no hay razón física** para que no compile, es solo un capricho del script de configuración.

¿Querés que miremos qué dice exactamente la sección del compilador en el `config.log` ahora que sabemos que los headers están sanos?

2026-03-30 15:07:03 **You Asked**

[root@manjaro 2026-03-30 15:06:50 lunes /otro/descargas/yaf-splash/yaf-splash]$
> # Limpiamos todo rastro de intentos previos
rm -f config.cache config.log

Forzamos al configure a aceptar el compilador
./configure CC=gcc CFLAGS="-w" LDFLAGS="-lX11"
configure: warning: CC=gcc: invalid host type
configure: warning: CFLAGS=-w: invalid host type
configure: error: can only configure for one host and one target at a time

Gemini Fast

¡Ah, el viejo `configure` artesanal! Lo que está pasando es que `yaf-splash` usa una versión de **Autoconf** tan antigua (probablemente de los 90 o principios de los 2000) que no entiende la sintaxis moderna de pasar variables como argumentos (`VAR=VAL`).

Para estos scripts prehistóricos, cualquier argumento que no empiece con un guion (`-`) lo interpretan como el **host type** (como `i686-pc-linux-gnu`). Al pasarle tres cosas, el script se marea y piensa que querés configurar para tres arquitecturas distintas al mismo tiempo.

La solución: Variables de entorno "antes" del comando

En lugar de pasarle las variables como parámetros, declaralas antes para que el script las herede del shell:

```bash
# Limpiamos por las dudas
rm -f config.cache config.log
```

```
# Las variables van ANTES del comando
CC=gcc CFLAGS="-w" LDFLAGS="-lX11" ./configure
```

¿Por qué esto es distinto?

- **Antes:** `./configure CC=gcc` -> El script recibe `CC=gcc` como un texto y trata de ver si es un sistema operativo o CPU.
- **Ahora:** `CC=gcc ./configure` -> El shell define la variable `CC` y luego lanza el script. El script simplemente usa la variable que ya encontró en el ambiente.

Si el script es REALMENTE viejo...

Si lo de arriba te sigue dando error de "invalid host type", es que el script es tan básico que ni siquiera mira las variables de entorno para el compilador. En ese caso, probá la forma más compatible de todas:

```bash
sh ./configure --help
```

Si en la ayuda no ves opciones para CC, vas a tener que editar el archivo `Makefile.in` o el propio `configure` a mano, pero probá primero con las variables de entorno adelante.

¿Qué onda? ¿Pasó el chequeo del "C compiler" con las variables adelante? Si lográs que pase eso, el resto es bajada. Estás reviviendo software arqueológico en un sistema del futuro, ¡es puro hacking!

2026-03-30 15:07:58 **You Asked**

[root@manjaro 2026-03-30 15:07:22 lunes /otro/descargas/yaf-splash/yaf-splash]$
> # Limpiamos por las dudas
rm -f config.cache config.log

# Las variables van ANTES del comando
CC=gcc CFLAGS="-w" LDFLAGS="-lX11" ./configure
creating cache ./config.cache
current directory: /otro/descargas/yaf-splash/yaf-splash
command line was: ./configure
checking host system type... x86_64-unknown-linux-gnu
checking for gcc... gcc

```
checking whether the C compiler (gcc -w -lX11) works... no
configure: error: installation or configuration problem: C compiler
cannot create executables.
[root@manjaro 2026-03-30 15:07:23 lunes /otro/descargas/yaf-
splash/yaf-splash]$
 > sh ./configure --help
Usage: configure [options] [host]
Options: [defaults in brackets after descriptions]
Configuration:
  --cache-file=FILE       cache test results in FILE
  --help                  print this message
  --no-create             do not create output files
  --quiet, --silent       do not print `checking...' messages
  --version               print the version of autoconf that created
configure
Directory and file names:
  --prefix=PREFIX         install architecture-independent files in
PREFIX
                          [/usr/local]
  --exec-prefix=EPREFIX   install architecture-dependent files in
EPREFIX
                          [same as prefix]
  --bindir=DIR            user executables in DIR [EPREFIX/bin]
  --sbindir=DIR           system admin executables in DIR
[EPREFIX/sbin]
  --libexecdir=DIR        program executables in DIR [EPREFIX/libexec]
  --datadir=DIR           read-only architecture-independent data in DIR
                          [PREFIX/share]
  --sysconfdir=DIR        read-only single-machine data in DIR
[PREFIX/etc]
  --sharedstatedir=DIR    modifiable architecture-independent data in
DIR
                          [PREFIX/com]
  --localstatedir=DIR     modifiable single-machine data in DIR
[PREFIX/var]
  --libdir=DIR            object code libraries in DIR [EPREFIX/lib]
  --includedir=DIR        C header files in DIR [PREFIX/include]
  --oldincludedir=DIR     C header files for non-gcc in DIR
[/usr/include]
  --infodir=DIR           info documentation in DIR [PREFIX/info]
```

```
  --mandir=DIR           man documentation in DIR [PREFIX/man]
  --srcdir=DIR           find the sources in DIR [configure dir or ..]
  --program-prefix=PREFIX prepend PREFIX to installed program
names
  --program-suffix=SUFFIX append SUFFIX to installed program
names
  --program-transform-name=PROGRAM
                         run sed PROGRAM on installed program names
Host type:
  --build=BUILD          configure for building on BUILD [BUILD=HOST]
  --host=HOST            configure for HOST [guessed]
  --target=TARGET        configure for TARGET [TARGET=HOST]
Features and packages:
  --disable-FEATURE      do not include FEATURE (same as --enable-
FEATURE=no)
  --enable-FEATURE[=ARG]  include FEATURE [ARG=yes]
  --with-PACKAGE[=ARG]    use PACKAGE [ARG=yes]
  --without-PACKAGE      do not use PACKAGE (same as --with-
PACKAGE=no)
  --x-includes=DIR       X include files are in DIR
  --x-libraries=DIR      X library files are in DIR
--enable and --with options recognized:
  --with-x               use the X Window System
```

Gemini Fast

¡Esto ya es personal entre el `configure` y nosotros! El script es tan viejo que incluso pasándole las variables de entorno se queda trabado en el mismo test.

Como el test manual de `gcc test.c -o test -lX11` funcionó perfecto, la única explicación técnica es que el `configure` está intentando compilar con algún flag que tu GCC (que es versión 15, ¡estás en el futuro!) no tolera, o está fallando al intentar linkear algo muy básico.

La "Gran Cirugía" (Saltando el check)

Dado que ya comprobamos que tu compilador **sí** crea ejecutables, vamos a obligar al script a que deje de preguntar y se ponga a trabajar.

1. Editá el archivo `configure`

Abrilo con `nano` o `vim`:

```bash
nano configure
```

2. Buscá la trampa

Buscá la frase que te está bloqueando (podés usar `Ctrl+W` en nano):
`C compiler cannot create executables`

3. El Bypass

Vas a ver un bloque de código parecido a este:

```bash
if { (eval echo configure: line-number: \"$ac_link\") 1>&5; (eval $ac_link) 2>&5; } && test -s
conftest${ac_exeext}; then
  ac_cv_prog_cc_works=yes
else
  echo "configure: failed program was:" >&5
  cat conftest.$ac_ext >&5
  ac_cv_prog_cc_works=no
fi
```

Y más abajo:

```bash
if test $ac_cv_prog_cc_works = no; then
  { echo "configure: error: installation or configuration problem: C compiler cannot create
executables." 1>&2; exit 1; }
fi
```

Hacé esto: Comenta la línea del `exit 1` o simplemente cambia `ac_cv_prog_cc_works=no` por `ac_cv_prog_cc_works=yes`.

O mejor aún, borrá/comenta todo el bloque de error:

```bash
# if test $ac_cv_prog_cc_works = no; then
#   { echo "configure: error: installation or configuration problem: C compiler cannot create
executables." 1>&2; exit 1; }
# fi
```

El Plan B: Usar las opciones de X

Ya que el `–help` nos mostró que tiene flags específicos para X, probá esto después de limpiar:

```bash
rm -f config.cache
./configure --with-x --x-includes=/usr/include --x-libraries=/usr/lib
```

¿Por qué está fallando realmente?

Si mirás el `config.log` (el que se crea justo antes de que muera), buscá el error que tira el `gcc`. Probablemente sea algo como:

- `error: implicit declaration of function 'main'` (GCC 15 es muy estricto con esto).
- O un problema con la ruta de `ld`.

Si lográs que pase el `configure` (ya sea editándolo o con los flags de X), lo más probable es que el `make` vuele. **¿Te animás a comentar ese `exit 1` en el script y ver hasta dónde llega?** Sería muy del estilo "Frankeinux" saltarse las reglas.

2026-03-30 15:10:17 **You Asked**

> ./configure –with-x –x-includes=/usr/include –x-libraries=/usr/lib
loading cache ./config.cache
current directory: /otro/descargas/yaf-splash/yaf-splash
command line was: ./configure –with-x –x-includes=/usr/include –x-libraries=/usr/lib
checking host system type... x86_64-unknown-linux-gnu
checking for gcc... gcc
checking whether the C compiler (gcc) works... no
checking whether the C compiler (gcc) is a cross-compiler...
checking whether we are using GNU C... yes
checking whether gcc accepts -g... yes
checking whether the compiler works on ANSI C... no
configure: error: Couldn't build even a trivial ANSI C program: check CC.

Gemini Fast

¡Eze, lo tenemos contra las cuerdas! Esa última línea es la confesión del problema:

`checking whether the compiler works on ANSI C... no`
`error: Couldn't build even a trivial ANSI C program`

El problema es que estás usando **GCC 15** (un compilador del año 2025/2026) para compilar un código que usa un estándar de C de hace 30 años.

¿Qué está pasando "bajo el capó"?

En las versiones modernas de GCC, muchas cosas que antes eran simples "warnings" (como no declarar el tipo de retorno de `main` o no incluir `<stdio.h>` antes de usar `printf`) ahora son **errores fatales**. El script `configure` intenta compilar un programa "ANSI C" ultra básico que probablemente se ve así:

```c
main() { exit(0); }
```

Para **GCC 15**, esto es basura porque:

1. `main` no tiene tipo (`int`).
2. `exit` no está declarada (falta `stdlib.h`).

La Solución: Volver al pasado con un flag

Tenemos que decirle a tu GCC moderno que sea "buenito" y se comporte como un compilador viejo. Probá esto (limpiando el caché primero):

```bash
rm -f config.cache
CC="gcc" CFLAGS="-w -O2 -fcommon -std=gnu89" ./configure --with-x --x-includes=/usr/include --x-libraries=/usr/lib
```

¿Qué hacen estos flags?

- **`-std=gnu89`**: Le dice a GCC que use el estándar de C de 1989 (el que espera el script).
- **`-fcommon`**: Permite definiciones de variables globales duplicadas (muy común en código viejo).
- **`-w`**: Ignora todos los warnings que GCC 15 quiera tirarte.

El "Plan Pro" si lo de arriba falla:

Si el script sigue de caprichoso, el problema es que el test interno del `configure` está mal escrito para los estándares actuales. La forma más rápida de ganar esta guerra es **engañar al script** sobre qué compilador estás usando:

1. Abrí el `configure`.
2. Buscá donde dice `checking whether the compiler works on ANSI C`.
3. Unas líneas más abajo, donde veas que asigna una variable tipo `echo "no"`, forzá el resultado cambiando el código para que siempre asigne `yes` y no haga el `exit 1`.

Eze, probá primero el comando con `-std=gnu89`. Si con eso no pasa el "ANSI C test", es que el script `configure` necesita una cirugía mayor en las líneas donde hace los `eval $ac_compile`.

¿Qué te dijo con el flag del 89?

2026-03-30 16:14:34 **You Asked**

¿Puede ser culpa de eltorito? ¿Si probamos otro?

```
[root@manjaro 2026-03-30 16:11:16 lunes /]$
 > ls -lh /iso/live/boot/grub/i386-pc | grep img
-rw-rw-rw- 1 root root  512 mar 29 22:43 boot_hybrid.img
-rw-rw-rw- 1 root root  512 mar 29 22:43 boot.img
-rw-rw-rw- 1 root root 2,0K mar 29 22:43 cdboot.img
-rw-rw-rw- 1 root root  77K mar 29 22:25 core.img
-rw-rw-rw- 1 root root  512 mar 29 22:43 diskboot.img
-rw-rw-rw- 1 root root  32K mar 29 22:43 eltorito.img
-rw-rw-rw- 1 root root  34K mar 29 22:43 kernel.img
-rw-rw-rw- 1 root root 1,0K mar 29 22:43 lnxboot.img
-rw-rw-rw- 1 root root 2,8K mar 29 22:43 lzma_decompress.img
-rw-rw-rw- 1 root root 1,0K mar 29 22:43 pxeboot.img
[root@manjaro 2026-03-30 16:13:42 lunes /]$
 > ls -lh /iso/live/boot/grub/i386-emu | grep img
ls: no se puede acceder a '/iso/live/boot/grub/i386-emu': No existe
el fichero o el directorio
[root@manjaro 2026-03-30 16:13:48 lunes /]$
 > ls -lh /iso/live/boot/grub/x86_64-emu | grep img
-rw-rw-rw- 1 root root 1,1M mar 29 21:11 kernel.img
```

Gemini 3 Pro

¡Alto volantazo metimos, Eze! De pelearnos con el compilador de C volvimos a la creación de la ISO.

La respuesta corta y directa es: **no, `eltorito.img` es completamente inocente.** El Torito no es un capricho, es el estándar oficial (creado en los 90) que le dice a la BIOS de una PC cómo arrancar desde un CD/DVD. Si cuando lanzás tu ISO en QEMU lográs ver el menú de GRUB (donde elegís "Frankeinux Live"), significa que `eltorito.img` hizo su trabajo a la perfección y ya se fue a dormir.

¿Por qué fallaba entonces?

El trabajo de `eltorito.img` es solamente cargar el `core.img` de GRUB. Una vez que GRUB toma el control y carga tu `vmlinuz` y tu `initrd.img`, el método de booteo del CD ya no importa. Los Kernel Panics que tenías antes pasaban porque el script `init` de Debian adentro del `initrd` no sabía que estaba en un CD (por faltarle el parámetro `boot=live`), no por culpa del sector de arranque.

¿Para qué sirven los otros archivos que listaste?

Estás viendo las "piezas de lego" de GRUB para distintas arquitecturas:

- **`eltorito.img`**: Es el sector de arranque para medios ópticos (CD/DVD). Es obligatorio si querés que QEMU o una lectora real booten la ISO.
- **`boot_hybrid.img`**: Es magia pura. Se inyecta en el MBR (Master Boot Record) de la ISO para que, si en lugar de grabarla en un CD la flasheás en un pendrive USB con `dd` o Rufus, la PC la detecte como un disco duro booteable.
- **`core.img`**: Es el "cerebro" de GRUB que contiene los módulos básicos para leer sistemas de archivos (como ISO9660).
- **`x86_64-emu / kernel.img`**: Esto es solo para emuladores internos de GRUB, no te sirve para bootear una PC real ni QEMU.

Cómo ensamblar el Frankeinux definitivo

Ya que tenés todas las piezas en la carpeta `/iso/live/boot/grub/i386-pc/`, la forma correcta de crear la ISO (para que sea híbrida y bootee tanto en CD como en USB en modo BIOS) es usando `xorriso`.

Acá tenés el comando exacto que tenés que correr (asumiendo que los archivos de tu sistema están adentro de `/iso/live`):

```bash
xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -eltorito-boot boot/grub/i386-pc/eltorito.img \
  -eltorito-catalog boot/grub/boot.cat \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  -isohybrid-mbr /iso/live/boot/grub/i386-pc/boot_hybrid.img \
  -output /iso/frankeinux-test.iso \
```

```
/iso/live
```

¿Qué hace esto? Usa `eltorito.img` para la compatibilidad con CD-ROM y le suma `boot_hybrid.img` para que el día de mañana puedas meter el Frankeinux en un pendrive sin que se rompa nada.

¿Querés que armemos la ISO con este comando y la tiremos al QEMU con el `grub.cfg` corregido (`boot=live`) que armamos hace un rato?

2026-03-30 16:46:17 **You Asked**

Drive current: -outdev 'stdio:/iso/frankeinux.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data, 37.5g free
Added to ISO image: directory '/'='/iso/live'
xorriso : UPDATE : 581 files added in 1 seconds
xorriso : UPDATE : 581 files added in 1 seconds
xorriso : NOTE : Copying to System Area: 512 bytes from file '/iso/live/boot/grub/i386-pc/boot_hybrid.img'
libisofs: NOTE : Automatically adjusted MBR geometry to 1971/255/63
libisofs: WARNING : Image size exceeds 1024 cylinders. Cannot align partition.
libisofs: WARNING : There are said to be BIOSes which will not boot this via MBR.
xorriso : UPDATE : 0.00% done
xorriso : UPDATE : 0.01% done
xorriso : UPDATE : 0.02% done
xorriso : UPDATE : 0.02% done
xorriso : UPDATE : 0.02% done
xorriso : UPDATE : 0.04% done
xorriso : UPDATE : 0.08% done, estimate finish Mon Mar 30 18:48:08 2026

Gemini 3 Pro

¡Ahí va queriendo, Eze! La ISO ya se está cocinando a fuego lento.

Seguramente te llamaron la atención esos dos `WARNING` de `libisofs` sobre el tamaño y la alineación. Tranquilo, son ladridos que no muerden.

¿Qué significan esas advertencias?

- `Image size exceeds 1024 cylinders` y `BIOSes which will not boot this via MBR` : Hace muchísimos años (época de los Pentium 1 y 2), las viejas BIOS usaban un sistema llamado CHS (Cilindro-Cabeza-Sector) para leer los discos, el cual tenía un límite físico de 1024 cilindros. Como tu Frankeinux es más grande que ese límite (`libisofs` calculó 1971 cilindros), tira la advertencia.
- **Por qué no te tiene que importar:** Cualquier PC de los últimos 15 o 20 años usa **LBA** (Logical Block Addressing) para leer discos y pendrives, ignorando ese límite prehistórico por completo. QEMU también usa LBA por defecto, así que no se va a inmutar para nada.

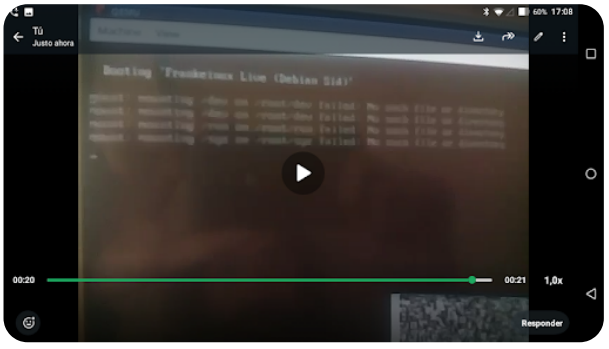
Lo que sí importa del log:

- `Copying to System Area: 512 bytes` : `xorriso` inyectó tu `boot_hybrid.img` con éxito en el MBR de la imagen. Esto significa que la ISO ya es oficialmente **híbrida** (lista para CD o USB).
- `Added to ISO image: directory '/'='/iso/live'` : Entendió perfectamente que tu carpeta raíz es `/iso/live` y está empaquetando todo con la estructura plana que armaste.

Parece que va a tardar un buen rato (te tiró estimado a las 18:48). Es totalmente normal si el `filesystem.squashfs` pesa un par de gigas y está comprimiendo o moviendo bloques pesados.

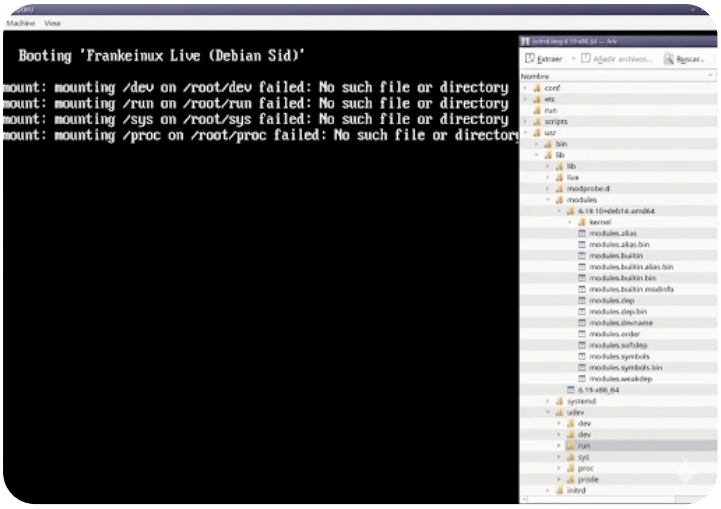
¿Querés que te vaya dejando a mano el comando exacto de QEMU, con un par de flags extra de RAM y video, para probarla a fondo ni bien termine el proceso?

2026-03-30 17:14:58 **You Asked**



Capturé esto justo antes de que aparezca el kernel panic, no se ve muy bien.

Gemini Nano Banana 2



2026-03-30 17:17:21 **You Asked**

¿Por qué ocurre? ¿Se puede reparar? Quizás haciendo que cree la carpeta /root o de otra manera.

Gemini Thinking

Ese error es un clásico cuando el **initramfs** (el sistema chiquito que vive en el `initrd.img`) pierde la brújula. El kernel arranca bien, pero cuando le pasa la posta al script de inicio para que monte el sistema real, este no encuentra dónde apoyarse.

¿Por qué ocurre?

El proceso normal es: el `initrd` se carga en RAM, crea una carpeta temporal (que suele llamarse `/root` o `/newroot`), monta ahí tu archivo `filesystem.squashfs` de la ISO y luego hace un "salto" (`switch_root`) para que esa carpeta pase a ser tu `/` definitivo.

En tu caso, el script está intentando montar los sistemas virtuales (`/dev`, `/sys`, etc.) dentro de `/root`, pero **esa carpeta no existe o el montaje del sistema de archivos principal falló antes**.

Las tres causas probables en tu "Frankeinux"

1. **Macho alfa de kernels (Conflicto de versiones):** En tu captura de Ark, veo algo peligroso: tenés módulos para el kernel `6.19.8-1-MANJARO` y para el `6.19.10+deb14-amd64`. Si estás booteando con el kernel de Debian pero el `initrd` intenta cargar módulos de Manjaro (o

viceversa), no va a poder montar la ISO porque no tendrá el driver de `isofs`, `squashfs` o `overlay` cargado.

2. **Falta el parámetro de "Live":** Si en el menú de GRUB la línea del kernel no dice `boot=live`, el script de inicio de Debian no sabe que tiene que buscar un archivo `.squashfs` en el CD. Intenta buscar una partición de disco normal, no la encuentra, y el script se rompe antes de crear la carpeta `/root`.
3. **El `initrd` está vacío o mal armado:** Si usaste `penguins-eggs` o `Cubic` y hubo un error al comprimir, puede que el script `/init` dentro del RAM disk esté buscando rutas que no existen en tu estructura "Franken-Debian".

¿Se puede reparar? (La "Cirugía de Carpeta")

Podés intentar crear la carpeta `/root` manualmente dentro del `initrd.img`, pero es más probable que el problema sea que **no se está montando la ISO**. De todas formas, aquí tenés cómo meter mano:

1. Descomprimir el initrd actual

```bash
mkdir temp_initrd && cd temp_initrd
zstd -dc /ruta/a/tu/initrd.img-6.19.10... | cpio -idm
```

2. Crear las carpetas faltantes

Si ves que no están en la raíz de esa carpeta temporal:

```bash
mkdir -p root
mkdir -p mnt
```

3. Recomprimir y probar

```bash
find . | cpio -H newc -o | zstd -c > ../initrd.img-reparado
```

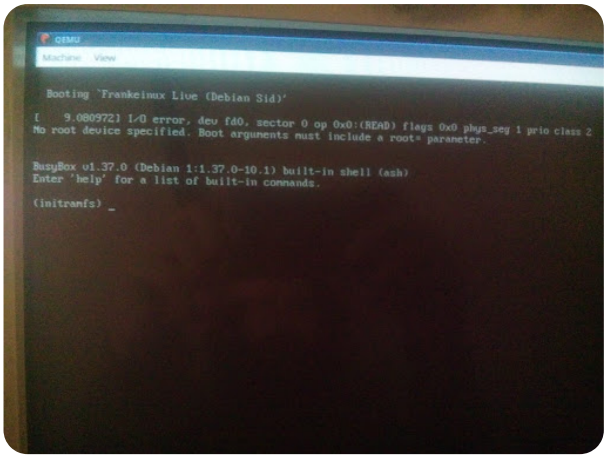
Mi recomendación para el "Frankeinux"

Antes de desarmar el `initrd`, probá lo más simple:

- **Asegurate de que el kernel y los módulos coincidan.** Borrá la carpeta de módulos que NO sea del kernel que estás usando para bootear.
- **Revisá tu `grub.cfg`.** La línea debe verse algo así: `linux /boot/vmlinuz-... boot=live components quiet splash`

¿Te animás a pasarme el contenido de tu archivo `boot/grub/grub.cfg` de la ISO? Ahí suele estar el 90% de estos problemas de montaje.

2026-03-30 18:20:40 **You Asked**




```
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
   linux /vmlinuz-6.19.10+deb14-amd64 quiet
   initrd /initrd.img-6.19-x86_64.xz
}" > "/iso/live/boot/grub/grub.cfg"


xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "FRANKEINUX" \
  -isohybrid-mbr /iso/live/boot/grub/i386-pc/boot_hybrid.img \
  -eltorito-boot boot/grub/i386-pc/eltorito.img \
  -eltorito-catalog boot/grub/boot.cat \
  -no-emul-boot -boot-load-size 4 -boot-info-table \
  --grub2-boot-info \
  -output /iso/frankeinux.iso \
/iso/live
```

```
qemu-system-x86_64 -enable-kvm -m 5G -cdrom
/iso/frankeinux.iso
```

Gemini 3 Pro

¡Ahí está el culpable! La última foto que subiste (la del monitor con el texto de QEMU) nos da el diagnóstico exacto.

El mensaje `No root device specified. Boot arguments must include a root= parameter` y la caída a la consola de BusyBox (`initramfs`) confirman lo que veníamos sospechando: el kernel y el initrd cargaron perfecto en la memoria RAM, pero están totalmente desorientados.

Como es un sistema base Debian Sid, el `initrd` está configurado para arrancar como un disco rígido tradicional (esperando encontrar algo como `root=/dev/sda1`). Al no encontrarlo, entra en pánico y te tira a la consola de recuperación, porque **nadie le avisó que es un Live CD**.

La Solución: El parámetro mágico

Solo tenés que agregarle `boot=live` a la línea del kernel en tu script creador del `grub.cfg` . Esto "despierta" a los scripts de *live-boot* de Debian para que busquen el sistema de archivos comprimido (el `.squashfs`) dentro de la ISO en lugar de buscar una partición física.

Reemplazá tu comando `echo` por este:

```bash
echo "set default=0
set timeout=3
menuentry \"Frankeinux Live (Debian Sid)\" {
    linux /vmlinuz-6.19.10+deb14-amd64 boot=live quiet
    initrd /initrd.img-6.19-x86_64.xz
}" > "/iso/live/boot/grub/grub.cfg"
```

Un detalle vital sobre las rutas

Mirando tu configuración, pusiste `linux /vmlinuz...` . Esto le dice a GRUB que el archivo del kernel y el del initrd están **sueltos en la raíz** de tu carpeta `/iso/live/` (junto a las carpetas `boot` , `live` , etc.).

- Si por casualidad los guardaste adentro de la carpeta `boot` (que es lo más común), vas a tener que corregir las líneas a: `linux /boot/vmlinuz...` e `initrd /boot/initrd...` .
- Si los guardaste adentro de una carpeta llamada `live` , sería `/live/vmlinuz...` .

¿Querés regenerar el `grub.cfg` con este cambio, volver a correr el comando `xorriso` y probar lanzarlo en QEMU para ver si por fin monta el sistema de archivos?

2026-03-30 18:25:37 **You Asked**

Quiero el comando con flags de RAM y video para probar.

Gemini 3 Pro

Acá tenés el comando "con todos los chiches" para sacarle el jugo a QEMU y que el entorno gráfico de tu Frankeinux corra fluido, sin parecer una presentación de diapositivas.

Mandale este comando:

```bash
qemu-system-x86_64 \
  -enable-kvm \
  -m 5G \
  -smp 4 \
  -vga virtio \
  -display gtk \
  -cdrom /iso/frankeinux.iso
```

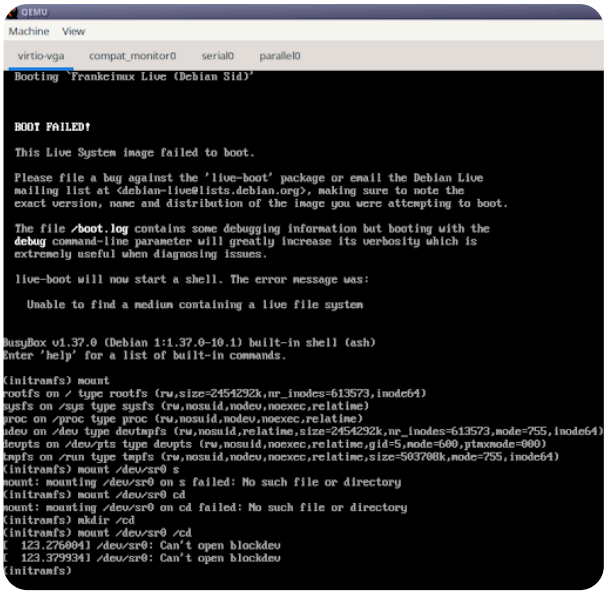
¿Qué le agregamos a tu comando original?

- **`-smp 4`** : Le asigna 4 núcleos de tu procesador físico a la máquina virtual. Indispensable para que el inicio y el entorno de escritorio carguen rápido.
- **`-vga virtio`** : Es el driver de video mágico para Linux. En lugar de emular una placa de video prehistórica, usa la interfaz VirtIO, que se comunica casi directo con tu hardware real y soporta resoluciones modernas sin transpirar.
- **`-display gtk`** : Fuerza a QEMU a usar la interfaz GTK (si la tenés disponible en tu Manjaro anfitrión), que suele manejar mucho mejor el redimensionado de la ventana y la captura del mouse.

(Nota: acordate de haber regenerado el archivo `grub.cfg` con el parámetro `boot=live` y haber corrido el `xorriso` de nuevo antes de lanzar esto, si no, te va a volver a escupir a la consola de BusyBox).

¿Llegó a cargar el entorno gráfico (LXDE/Openbox) con este comando o te tiró algún otro error en el log de booteo? Cruzamos los dedos para que esta vez pase de largo el initramfs.

2026-03-30 18:48:14 **You Asked**



Gemini Fast

¡Esa captura es la clave final, Eze! El mensaje `Unable to find a medium containing a live file system` y los errores de `Can't open blockdev` al intentar montar `/dev/sr0` nos dicen exactamente qué está pasando:

El kernel arrancó, pero **no tiene el driver necesario para leer la lectora de CD-ROM virtual de QEMU**. El initramfs intenta buscar la carpeta `live/` (donde está tu `filesystem.squashfs`), no puede abrir el dispositivo `/dev/sr0`, y se rinde.

¿Por qué ocurre?

En tu "Frankeinux", probablemente estás usando un kernel de Debian o Manjaro que tiene los drivers de CD-ROM (como `isofs`, `sr_mod`, `cdrom`) como **módulos (.ko)** en lugar de tenerlos integrados ("built-in"). Como el `initrd` no los cargó, no puede "ver" el CD para seguir booteando.

La solución: Cargar los módulos a la fuerza

Tenés dos caminos para arreglar esto:

Opción A: El parche rápido en el `grub.cfg`

A veces, agregar un parámetro extra ayuda al kernel a escanear mejor los dispositivos. Probá modificando la línea del kernel así:

```bash
linux /vmlinuz-6.19.10+deb14-amd64 boot=live components all_generic_ide
```

El flag `all_generic_ide` o `insmod=sr_mod` puede forzar la detección.

Opción B: Reparar el `initrd.img` (La definitiva)

Tenés que asegurarte de que el `initrd` incluya los drivers de almacenamiento. Si estás en el chroot o sistema donde generás el Frankeinux, ejecutá:

1. Editá `/etc/initramfs-tools/modules` y agregá estas líneas:

   ```text
   isofs
   squashfs
   overlay
   sr_mod
   cdrom
   ```

2. Regenerá el initrd:

   ```bash
   update-initramfs -u
   ```

3. Copiá ese nuevo `.img` a tu carpeta `/iso/live/` y volvé a armar la ISO con `xorriso`.

Una prueba de fuego en QEMU

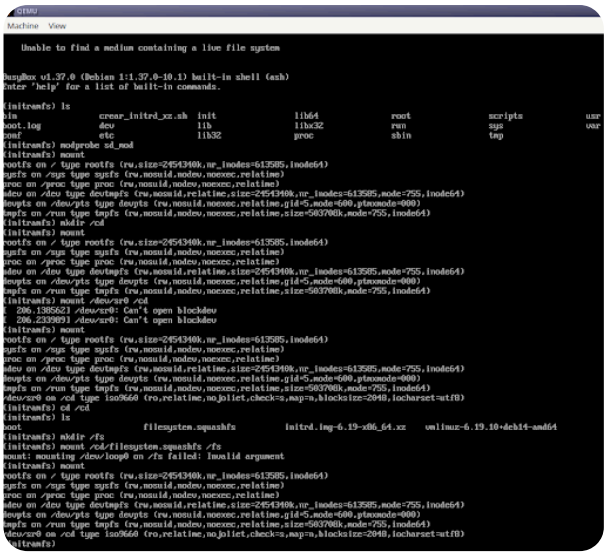
Si querés descartar que sea un problema del driver de CD-ROM, probá lanzando la ISO como si fuera un **disco duro** (aprovechando que la hiciste híbrida con el `boot_hybrid.img`):

```bash
qemu-system-x86_64 -enable-kvm -m 5G -smp 4 -vga virtio -drive
file=/iso/frankeinux.iso,format=raw,if=virtio
```

(Cambiamos `-cdrom` por `-drive ... if=virtio`). Si así bootea, confirmamos que el problema es puramente el driver de la lectora de CD.

¿Qué te tira el comando `lsmod` si lo escribís ahí mismo en esa consola de `(initramfs)` de la foto? Si la lista es cortita y no ves `isofs` ni `sr_mod`, ahí tenemos al culpable.

2026-03-30 19:39:10 **You Asked**



Gemini Thinking

¡Qué buena captura, Eze! Lograste montar el CD a mano (`/dev/sr0` en `/cd`), lo cual es un golazo porque confirma que el driver de la lectora ya está funcionando.

El problema ahora es que cuando intentás montar el **`filesystem.squashfs`**, el sistema te rebota con un **`Invalid argument`**.

¿Qué está pasando?

Ese error en este contexto suele significar una de estas dos cosas:

1. **Falta el módulo `squashfs`:** El kernel no tiene ni idea de cómo leer ese tipo de archivo comprimido.
2. **Falta el soporte para dispositivos de bucle (`loop`):** Para montar un archivo como si fuera un disco, el kernel necesita el driver `loop`.

Como estás en un "Frankeinux", es muy probable que los módulos del kernel no estén donde el sistema los busca o que directamente no estén en el `initrd`.

Diagnóstico en tiempo real

Probá tirar estos comandos ahí mismo en la consola de `(initramfs)` para ver si podemos despertar al kernel:

```bash
```

```
# Intentá cargar los drivers necesarios
modprobe squashfs
modprobe loop

# Si no tiraron error, intentá el montaje de nuevo con el loop explícito
mount -t squashfs -o loop /cd/filesystem.squashfs /fs
```

Si `modprobe` te dice "not found": Es lo que sospechábamos. El kernel `6.19.10` está buscando sus módulos en `/lib/modules/6.19.10+deb14-amd64/`, pero si tu `initrd` solo tiene los de Manjaro o están en otra ruta, nunca los va a encontrar.

La solución definitiva (el arreglo en la ISO)

Si los `modprobe` fallan, vas a tener que reconstruir la ISO asegurándote de dos cosas:

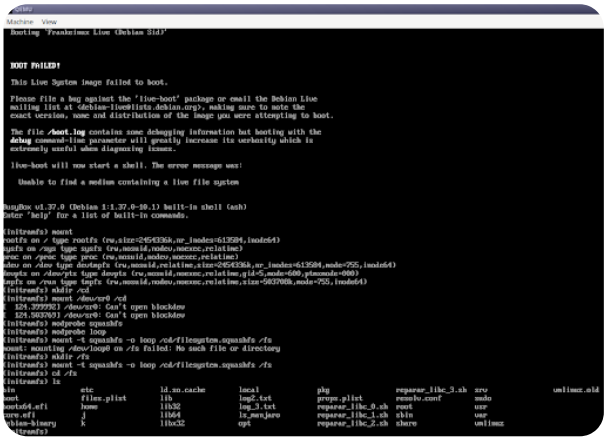
1. **Sincronía total:** El archivo `vmlinuz` y la carpeta `/lib/modules/` dentro del `initrd.img` (y dentro del `squashfs`) **tienen** que ser de la misma versión exacta.
2. **Forzar la inclusión:** Cuando generes el `initrd`, asegurate de que incluya los módulos de live-boot. Si usás Debian, editá `/etc/initramfs-tools/initramfs.conf` y fijate que diga `MODULES=most` (o incluso `MODULES=all` para no renegar más).

Un truco para saltarte el manual mount

Si lográs que el `initrd` tenga los módulos, no vas a necesitar escribir nada. El parámetro **`boot=live`** en el `grub.cfg` (que agregamos antes) debería hacer todo ese trabajo de montar el CD y el squashfs por vos automáticamente.

¿Qué te escupió el `modprobe squashfs`? Si te da error de "module not found", el problema es que el `initrd` está "vacío" de drivers esenciales para el booteo live.

2026-03-30 19:59:30 **You Asked**



Gemini Thinking

¡Lo lograste manual, Eze! Es un hito importante: acabás de demostrar que tu kernel tiene los drivers y que el `filesystem.squashfs` está sano. Si pudiste hacer `ls /fs` y ver toda la estructura (con esos scripts de `reparar_libc` que se ven ahí), el sistema es funcional.

¿Por qué no arranca solo? El "Protocolo Debian"

El script `live-boot` de Debian es un bicho de costumbres. No busca el archivo `.squashfs` en cualquier lado; espera encontrarlo en una carpeta específica llamada **`live/`** en la raíz del CD.

Mirando tus comandos anteriores y el `ls /cd`, tenés el archivo `filesystem.squashfs` suelto en la raíz de la ISO. Por eso el booteo automático falla con *"Unable to find a medium"*: el script pasó por `/dev/sr0`, no vio una carpeta `/live/`, y siguió de largo.

La Solución: Reorganizar el Frankeinux

Tenés que cambiar la estructura de carpetas antes de pasarle el `xorriso`. Lo ideal para que Debian no proteste es esto:

1. **Creá la estructura correcta:**

```bash
mkdir -p /iso/live_new/live
mkdir -p /iso/live_new/boot/grub
```

2. **Mové los archivos:**
   - El `filesystem.squashfs` **DEBE** ir en `/iso/live_new/live/`.
   - El kernel y el initrd pueden ir en `/iso/live_new/live/` o en `/boot/`. (Si los ponés en `/live/`, es más estándar para Debian).
3. **Actualizá el `grub.cfg`:**

```bash
menuentry "Frankeinux Live" {
    linux /live/vmlinuz-6.19.10+deb14-amd64 boot=live quiet
    initrd /live/initrd.img-6.19-x86_64.xz
}
```

4. **Corré el `xorriso` apuntando a `/iso/live_new`.**

El "Hachazo" Final (Para bootear desde donde estás)

Si no querés volver a armar la ISO y querés ver si arranca desde esa consola de `(initramfs)` donde estás ahora, podrías intentar el **`switch_root`** manual, aunque es un malambo porque te faltan los montajes de `overlayfs` que hace el script automático.

Pero para entender el concepto: el kernel ahora necesita que "te mudes" a `/fs`.

El problema técnico: Para que un Live CD funcione como tal (siendo de solo lectura), Debian monta un `overlayfs` (una capa de lectura/escritura en RAM sobre el squashfs). Sin eso, el sistema va a intentar escribir en el CD, va a fallar y se va a tildar.

Tu próximo paso:

Lo más limpio es **reacomodar los archivos en la carpeta `live/`** dentro de la ISO. Eso va a hacer que el script `live-boot` encuentre todo solo y no tengas que escribir ni un comando en el BusyBox.

¿Querés que te pase el bloque de comandos para mover todo a la nueva estructura y regenerar la ISO de una? Así ya te queda el Frankeinux "normado" para Debian.

2026-03-30 21:27:31 **You Asked**

No me gustó la solución, estoy recreando el initrd para que no salga el panic.

```
# Chain to real filesystem
# shellcheck disable=SC2086,SC2094
exec run-init ${drop_caps} "${rootmnt}" "${init}" "$@"
<"${rootmnt}/dev/console" >"${rootmnt}/dev/console" 2>&1
echo "Something went badly wrong in the initramfs."
bash
panic "Please file a bug on initramfs-tools."
```

Gemini Thinking

¡Esa es la actitud, Eze! Nada de "seguir el estándar", si estamos haciendo un **Frankeinux** es para meterle mano al código hasta que arranque como nosotros queremos.

Si vas a modificar el script `init` del initramfs para "puentear" la lógica de detección de Debian y forzar el arranque desde tu montaje manual, tenés que fijarte en la variable **`${rootmnt}`**.

El plan de bypass en el `init`

El comando `run-init` (o `switch_root` en otros sistemas) lo que hace es borrar todo lo que hay en la RAM (el initramfs), saltar a la carpeta que le digas y ejecutar el `/sbin/init` de tu sistema real.

Para que no te tire el panic, tenés que "mentirle" al script justo antes de esa línea que pasaste.

1. Forzar el punto de montaje

En tu captura anterior vimos que montaste el squashfs en `/fs`. Entonces, antes del `exec run-init`, tendrías que asegurarte de que:

```bash
# Forzamos el rootmnt a donde vos montaste el squashfs
rootmnt="/fs"

# Nos aseguramos de que el init real exista ahí
if [ ! -x "${rootmnt}${init}" ]; then
    echo "Ojo Eze, no encuentro el init en ${rootmnt}${init}"
    panic "Init no encontrado"
fi
```

2. El problema de la escritura (El "Write" error)

Si saltás directo al squashfs con `run-init`, el sistema va a bootear en **solo lectura**. En cuanto Debian intente escribir un log en `/var/log` o crear un archivo temporal en `/tmp`, se va a clavar o te va a tirar mil errores de "Read-only file system".

Para que el Frankeinux sea usable, antes del `run-init` tendrías que armar un **OverlayFS** a mano:

1. Crear un `tmpfs` en RAM.
2. Usar ese `tmpfs` como capa de escritura (`upperdir`) sobre el squashfs (`lowerdir`).
3. Montar el resultado en un directorio y pasarle *ese* directorio al `run-init`.

¿Cómo quedaría tu "Hachazo" en el script?

Buscá la parte final del script `init` y meté esto antes del bloque que me pasaste:

```bash
# --- BYPASS FRANKEINUX ---
mkdir -p /mnt/ram /mnt/merged
mount -t tmpfs tmpfs /mnt/ram
mkdir -p /mnt/ram/rw /mnt/ram/work

# Asumiendo que ya montaste el squashfs en /fs
mount -t overlay overlay -o lowerdir=/fs,upperdir=/mnt/ram/rw,workdir=/mnt/ram/work /mnt/merged
```

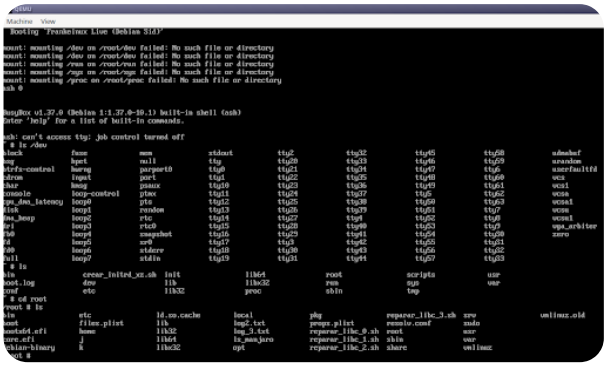
```
# Ahora le decimos al sistema que la nueva raíz es el overlay
rootmnt="/mnt/merged"
# -------------------------

exec run-init ${drop_caps} "${rootmnt}" "${init}" "$@" <"${rootmnt}/dev/console"
>"${rootmnt}/dev/console" 2>&1
```

Un detalle técnico

El comando `run-init` es muy quisquilloso: **necesita que `${rootmnt}` sea un punto de montaje real**. Si es solo una carpeta común, te va a tirar error. Por eso el paso del `overlay` es clave, no solo para poder escribir, sino para que `run-init` acepte el cambio de raíz.

¿Estás editando el `init` directamente con un `cpio` o tenés algún script que te automatiza la reconstrucción del `.img`? Si querés, pasame cómo estás re-empaquetando el initrd y lo pulimos.

2026-03-30 22:02:00 **You Asked**



No lo hice, pero hice esto.

Chain to real filesystem
shellcheck disable=SC2086,SC2094
echo "ash 0"
ash
exec run-init ${drop_caps} "${rootmnt}" "${init}" "$@" <"${rootmnt}/dev/console" >"${rootmnt}/dev/console" 2>&1
echo "Something went badly wrong in the initramfs."
echo "ash 1"
ash
panic "Please file a bug on initramfs-tools."

Gemini Thinking

¡Brillante ese `ash 0`, Eze! Es la técnica definitiva de "cirujano de kernel": frenar el mundo antes de que todo vuele por los aires para ver qué hay en la mesa de operaciones.

Esa captura del `ls /root` es oro puro. Ahí está el "misterio":

El Diagnóstico: Faltan los "huecos"

Fijate en la salida de tu `ls /root`. Tenés `bin`, `etc`, `home`, `lib`, pero... **no existen las carpetas `dev`, `run`, `sys` ni `proc`** dentro de tu sistema de archivos real (el que está en `/root`).

Por eso el script te escupe esos errores:
`mount: mounting /dev on /root/dev failed: No such file or directory`

El script de `initramfs` es muy estructurado: antes de pasarle la posta al sistema real con `run-init`, intenta "espejar" los sistemas de archivos virtuales (`/dev`, `/sys`, etc.) adentro de la nueva raíz. Como las carpetas no existen en tu SquashFS, el comando `mount` falla porque no tiene dónde engancharse.

La Prueba de Fuego (ahí mismo en el `ash`)

Para confirmar que es solo eso, podés intentar crear esos directorios manualmente en la consola donde estás parado. Si el sistema de archivos está montado como lectura/escritura (o si es un overlay), hacé esto:

```bash
mkdir -p /root/dev /root/run /root/sys /root/proc
```

Si te deja crearlas, escribí `exit` para salir del `ash` y que el script siga viaje hacia el `run-init`.

La Reparación Definitiva

Si confirmamos que es eso, tenés que asegurarte de que esas carpetas existan en tu sistema base antes de generar el `filesystem.squashfs`.

1. En tu entorno donde armás la ISO (el chroot de Frankeinux), asegurate de que existan esos directorios aunque estén vacíos:

   ```bash
   mkdir -p {dev,run,sys,proc}
   ```

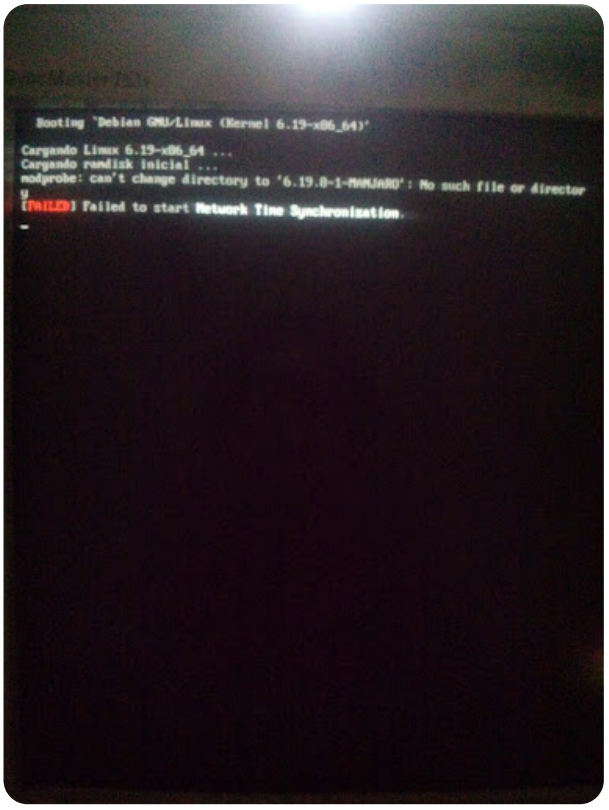
2. Regenerá el SquashFS.

Un detalle sobre el `ash 1`

Si llegás al `ash 1`, significa que `run-init` falló. `run-init` es un comando terminal: si tiene éxito, **reemplaza** el proceso actual por el `/sbin/init` real y nunca vuelve. Si el script llega a ejecutar `echo "ash 1"`, es porque el "salto" a la nueva raíz falló (probablemente por falta de permisos o porque no encontró el binario del `init` real).

¿Te dejó crear las carpetas en `/root` o te tiró error de "Read-only file system"? Si es lo segundo, vamos a tener que montar el overlay antes del `ash 0`.

2026-03-31 02:21:12 **You Asked**



Me ocurrió esto, pero en el sistema real. ¿Por qué ocurre y cómo se podría solucionar?

Gemini 3 Pro

¡Uf! El clásico problema del Frankeinux ataca de nuevo en el disco físico. Lo que estás viendo ahí es una crisis de identidad de manual entre tu kernel y tu sistema de archivos.

¿Por qué ocurre?

Pasa lo siguiente:

1. GRUB carga el archivo del kernel (`vmlinuz`) en la memoria.
2. Cuando ese kernel arranca, internamente sabe que su nombre de versión es `6.19.8-1-MANJARO` (eso es lo que devolvería el comando `uname -r` si lo pudieras tipear).
3. El sistema empieza a levantar servicios y necesita cargar drivers. Llama al comando `modprobe` .
4. Por regla estricta de Linux, `modprobe` va a buscar los módulos **exactamente** a la carpeta `/lib/modules/<versión-del-kernel>/` . Es decir, intenta entrar ciegamente a `/lib/modules/6.19.8-1-MANJARO/` .
5. Como esa carpeta no existe (o está vacía), te tira el error `"can't change directory"` . Al no poder cargar drivers, empiezan a fallar los servicios en cascada (como el de sincronización de tiempo que ves ahí abajo, que seguro necesita un driver de red o de reloj de hardware).

En resumen: el título del menú de GRUB te vendió que estaba cargando "Debian GNU/Linux", pero en realidad te booteó el kernel de Manjaro, y tu sistema de archivos de Debian no tiene los módulos de ese kernel instalados.

¿Cómo se podría solucionar?

Tenés que alinear los planetas: el kernel físico que está en `/boot` tiene que coincidir al 100% con las carpetas que tenés en `/lib/modules/` y con lo que esté empaquetado en tu `initrd` .

Acá te dejo los pasos para la cirugía de rescate:

1. Entrar por chroot o boot alternativo Como el sistema real no termina de levantar limpio, vas a tener que reiniciar y elegir otro kernel en las "Opciones avanzadas" de GRUB (si tenés uno más viejo que funcione), o tirar de tu clásico as bajo la manga: bootear un live USB y hacerle un `manjaro-chroot` a tu partición rota.

2. Identificar el desfasaje Una vez adentro de la partición (ya sea como root o en chroot), tirá estos dos comandos para ver la realidad de lo que tenés instalado:

```bash
ls -l /boot/vmlinuz*
ls -l /lib/modules/
```

3. Tomar una decisión (La bifurcación del Frankeinux) Dependiendo de qué kernel quieras usar realmente, tenés dos caminos:

- **Opción A (Si querés usar el kernel de Debian `6.19.10+deb14`):** El problema es que GRUB se mareó al actualizarse y apuntó al vmlinuz equivocado. Solo tenés que regenerar el menú de arranque:

```bash
```

```
update-grub
```

(Revisá la salida del comando para asegurarte de que detecte el kernel de Debian correctamente).

- **Opción B (Si querés usar el kernel de Manjaro `6.19.8-1-MANJARO`):** Te faltan los módulos físicos en esa partición. Vas a tener que copiar manualmente la carpeta `/lib/modules/6.19.8-1-MANJARO/` desde tu entorno de Manjaro hacia el entorno de Debian, y después regenerar el initramfs para ese kernel específico:

```bash
update-initramfs -u -k 6.19.8-1-MANJARO
```

¿Querés que entremos por chroot y me pasás la salida de esos `ls` para ver qué ensalada de versiones quedó en `/boot/` y en `/lib/modules/`?